Multicloud DevOps By

VEERA SIR

AWS Full Notes

Complete Master Study Guide

Theory • Architecture Diagrams • Class Notes • LinkedIn Notes

24+	35+	100+
Complete Sections	Architecture Diagrams	Theory Topics

Topics Covered

EC2 · IAM · VPC · S3 · RDS · Lambda · ECS · EKS · Route 53 · CloudFront
ALB · NLB · Auto Scaling · DynamoDB · API Gateway · ElastiCache
CloudWatch · Prometheus · Grafana · WAF · Config · Nginx · Dynatrace · PagerDuty
CloudFormation · SNS · SQS · EventBridge · KMS · Cognito · Secrets Manager

TABLE OF CONTENTS

1. **AWS Cloud Computing — Overview**
   - 1.1 What is Cloud Computing (IaaS / PaaS / SaaS)
   - 1.2 Regions & Availability Zones
   - 1.3 AWS Service Domains

2. Amazon EC2 — Elastic Compute Cloud
2.1 How EC2 Works — Step by Step
2.2 AMI — Amazon Machine Image
2.3 Instance Types (General, Compute, Memory, Storage, Accelerated)
2.4 EBS — Elastic Block Storage (gp3, io2, st1, sc1)
2.5 Instance Store vs EBS
2.6 Security Groups — Virtual Firewall
2.7 Key Pairs — SSH Access
2.8 ENI — Elastic Network Interface
2.9 Nginx Installation on EC2
2.10 Public IP vs Private IP vs Elastic IP

3. AWS IAM — Identity & Access Management
3.1 Root User vs IAM Users
3.2 IAM Groups
3.3 IAM Roles & STS
3.4 IAM Policies — JSON Permission Documents
3.5 Allow vs Deny — Policy Evaluation Logic
3.6 IAM Role Switching
3.7 Programmatic Access & AWS CLI
3.8 IAM Best Practices

4. AWS VPC & Networking
4.1 CIDR — IP Address Ranges
4.2 Subnets — Public, Private, Isolated
4.3 Internet Gateway (IGW)
4.4 NAT Gateway — Private Subnet Internet Access
4.5 Route Tables
4.6 Security Groups vs Network ACLs
4.7 Bastion Host (Jump Server)
4.8 VPC Peering
4.9 VPC Endpoints (Gateway & Interface)
4.10 Forward Proxy vs Reverse Proxy

5. Amazon Route 53 — DNS & Domain Service
5.1 How DNS Works — Full Step-by-Step
5.2 Hosted Zones (Public & Private)
5.3 DNS Record Types (A, AAAA, CNAME, Alias, MX, TXT)
5.4 Alias vs CNAME — Critical Difference
5.5 Routing Policies (Simple, Weighted, Latency, Failover, Geolocation, Multi-Value)
5.6 Route 53 Health Checks

VEERA

6. AWS Elastic Load Balancer (ELB)
6.1 Types of Load Balancers (ALB, NLB, GWLB, CLB)
6.2 ALB — Application Load Balancer (Layer 7)
6.3 ALB Path-Based, Host-Based, Header-Based Routing
6.4 NLB — Network Load Balancer (Layer 4)
6.5 WAF + ALB Architecture
6.6 Target Groups & Health Checks

7. AWS Auto Scaling Group (ASG)
7.1 Min / Desired / Max Capacity
7.2 AMI + Launch Template — The Foundation
7.3 Scaling Policies (Target Tracking, Step, Scheduled, Predictive)
7.4 ASG + ALB Integration
7.5 Instance Refresh — Rolling Deployments

8. ACM & CloudFront — SSL and CDN
8.1 AWS Certificate Manager (ACM) — Free SSL/TLS
8.2 CloudFront CDN — How It Works
8.3 CloudFront Distributions, Origins, Cache Behaviors, TTL
8.4 CloudFront Security (OAC, Signed URLs, Signed Cookies, Geo Restriction, WAF)

9. Amazon S3 — Simple Storage Service
9.1 Core Concepts (Buckets, Objects, Keys, Metadata)
9.2 S3 Storage Classes (Standard, IA, Intelligent-Tiering, Glacier, Deep Archive)
9.3 Versioning
9.4 Lifecycle Policies
9.5 Replication (CRR & SRR)
9.6 Security (Bucket Policy, Block Public Access, Pre-Signed URL, Encryption)
9.7 Static Website Hosting
9.8 Event Notifications & Multipart Upload

10. Amazon RDS — Relational Database Service
10.1 Supported Database Engines
10.2 Three-Tier Architecture with RDS
10.3 Multi-AZ Deployment vs Read Replicas
10.4 Backup & Point-in-Time Recovery
10.5 Backup Window & Maintenance Window
10.6 Publicly Accessible Setting — 4 Conditions
10.7 RDS Proxy — Connection Pooling

VEERA

10.8 Amazon Aurora — Cloud-Native DB
10.9 Self-Managed Database on EC2
10.10 AWS Secrets Manager

11. Amazon DynamoDB — NoSQL Database
11.1 Why NoSQL? When to Use DynamoDB
11.2 Data Model (Tables, Items, Partition Key, Sort Key, Attributes)
11.3 Indexes — GSI & LSI
11.4 Capacity Modes (On-Demand & Provisioned)
11.5 DynamoDB Accelerator (DAX)
11.6 DynamoDB Streams & Transactions

12. Amazon API Gateway
12.1 API Types (REST, HTTP, WebSocket)
12.2 Core Concepts (Resources, Methods, Stages, Usage Plans)
12.3 Endpoint Types (Edge-Optimized, Regional, Private)
12.4 Security Methods (IAM, Lambda Authorizer, Cognito, WAF)

13. AWS Lambda — Serverless Compute
13.1 Serverless vs Traditional EC2
13.2 Lambda Trigger Sources (API GW, S3, DynamoDB, SQS, SNS, CloudWatch, EventBridge)
13.3 Execution Model (Cold Start, Warm Start, Concurrency, Memory, Timeout)
13.4 Lambda Layers
13.5 Lambda Monitoring & Log Pipeline

14. Amazon ElastiCache — In-Memory Caching
14.1 The Caching Pattern — How It Works
14.2 Redis vs Memcached — Full Comparison
14.3 Use Cases (Sessions, Query Cache, Rate Limiting, Leaderboards, Pub/Sub)

15. Monitoring & Observability
15.1 Amazon CloudWatch (Metrics, Alarms, Logs, Insights)
15.2 Prometheus — Open Source Metrics & PromQL
15.3 Grafana — Dashboards & Visualization
15.4 Alertmanager — Alert Routing & Deduplication
15.5 PagerDuty — On-Call & Incident Management
15.6 Dynatrace — Full-Stack APM (OneAgent, Davis AI, Smartscape)

VEERA

16. AWS Messaging — SNS, SQS & EventBridge
16.1 Amazon SNS — Simple Notification Service (Pub/Sub, Fan-out)
16.2 Amazon SQS — Message Queue (Standard, FIFO, DLQ, Visibility Timeout)
16.3 Amazon EventBridge — Serverless Event Bus

17. Nginx — Web Server & Reverse Proxy
17.1 Nginx Reverse Proxy Architecture
17.2 Nginx Configuration (Basic Proxy, Path-Based Routing)
17.3 Security Group Setup for Nginx

18. AWS Security — WAF, Config, KMS & Cognito
18.1 AWS WAF — Web Application Firewall (Web ACL, Rules, Managed Rules)
18.2 AWS Config — Compliance Monitoring & Config Rules
18.3 Automated Compliance: Config + EventBridge + Lambda + PagerDuty
18.4 AWS KMS — Key Management Service (CMK, Envelope Encryption)
18.5 Amazon Cognito — User Pools & Identity Pools

19. AWS CloudFormation — Infrastructure as Code
19.1 Why Infrastructure as Code?
19.2 Core Concepts (Template, Stack, Change Set, StackSets, Nested Stacks)
19.3 Template Structure — YAML Example

20. AWS Containers — ECS, EKS, ECR & Fargate
20.1 Amazon ECS — Elastic Container Service
20.2 AWS Fargate — Serverless Containers
20.3 Amazon ECR — Elastic Container Registry

21. Full AWS Production Architecture
21.1 Complete Request Flow — Step by Step
21.2 High Availability at Every Layer
21.3 Security — Defense in Depth
21.4 Complete AWS Master Checklist

VEERA

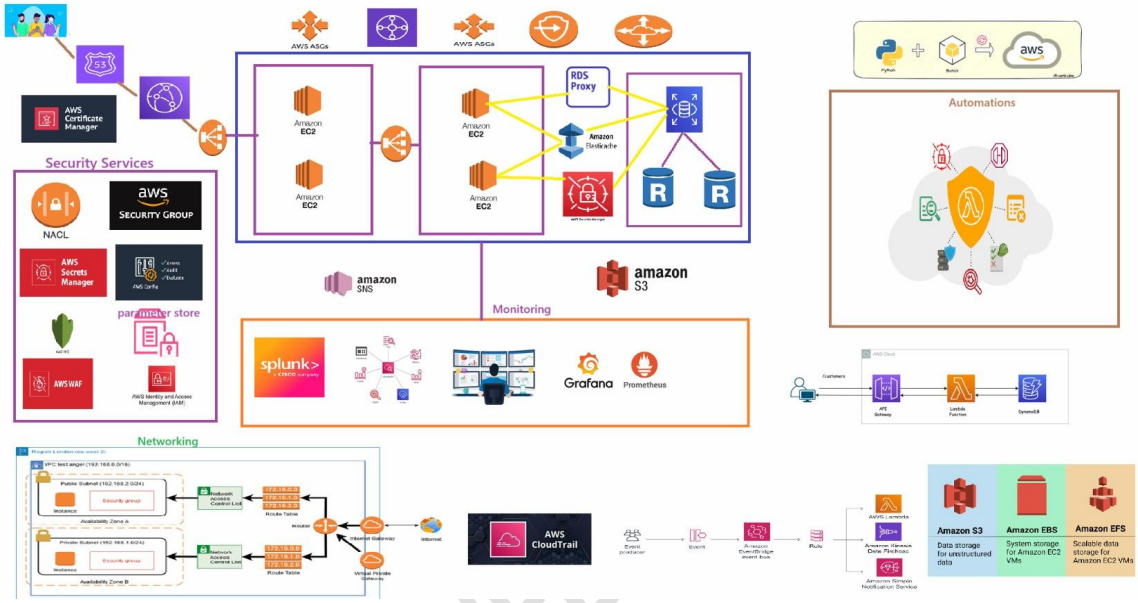
22. RDS Deep Dive — Architecture & Diagrams
22.1 DB on EC2 vs RDS — Connection via Endpoint
22.2 Multi-AZ: High Availability & Disaster Recovery
22.3 RDS with Read Replicas — Production Architecture
22.4 RDS Networking — Subnets, Public Access, All 4 Conditions
22.5 RDS Hands-On Lab Steps

23. S3 Deep Dive — Storage Classes & Replication
23.1 S3 Storage Classes — Visual Mind Map with Pricing
23.2 Storage Class Pricing Detail (per GB)
23.3 S3 Replication with Pre-Signed URLs

24. Production System — Frontend + Backend + Database
24.1 Data Consistency Problem — Why Shared Database
24.2 Complete 3-Tier Production Architecture

VEERA

1. AWS Cloud Computing — Overview



AWS Global Service Architecture — Regions, AZs, all key service categories

1.1 What is Cloud Computing?

Cloud computing means using remote servers on the internet to store, manage, and process data — instead of buying and maintaining physical servers on-premises. AWS is the world's leading cloud provider, offering 200+ services from 30+ global regions.

Model	Full Name	Who Manages Infra	AWS Example	You Manage
IaaS	Infrastructure as a Service	AWS manages hardware	EC2, VPC, S3	OS, Runtime, App, Data
PaaS	Platform as a Service	AWS manages OS+runtime	Elastic Beanstalk, RDS	App and Data only

SaaS	Software as a Service	AWS manages everything	WorkMail, Chime	Nothing — just use it

Theory Explained
Before cloud: A startup needed $50,000+ for servers, 3 months to set up, and a full IT team. With AWS: same startup launches in 1 hour, pays $50/month, scales to millions of users automatically. This is why cloud changed everything.

1.2 Regions & Availability Zones

- Region: A geographic location containing AWS data centers. Each region is fully independent. Example: ap-south-1 = Mumbai, us-east-1 = N. Virginia, eu-west-1 = Ireland.
- Availability Zone (AZ): One or more physically separate data centers inside a region, connected by high-speed private links. Each region has 2–6 AZs. Example: ap-south-1a, apsouth-1b, ap-south-1c.
- Edge Location: CloudFront CDN caching points — 400+ worldwide. Separate from regions. Used for low-latency content delivery.
- Local Zone: AWS infrastructure placed near large cities for ultra-low latency applications.

Concept	Count	Purpose	Example
Regions	30+	Geographic isolation, data sovereignty, compliance	ap-south-1 (Mumbai)
AZs per Region	2–6	HA and fault isolation within region	ap-south-1a / 1b / 1c
Edge Locations	400+	CloudFront CDN caching	Mumbai, Chennai, Hyderabad
Local Zones	30+	Ultra-low latency near cities	Los Angeles, NY, Delhi

Theory Explained
Why multiple AZs? Imagine AZ-a has a power failure. App running ONLY in AZ-a is DOWN. App running in BOTH AZ-a AND AZ-b stays up in AZ-b. Rule 1 of AWS architecture: always deploy across 2+ AZs.

1.3 AWS Service Domains

Domain	Key Services	What It Does
Compute	EC2, Lambda, ECS, EKS, Fargate	Run apps — VMs, containers, serverless
Storage	S3, EBS, EFS, Glacier	Store files, block data, file systems, archives
Database	RDS, Aurora, DynamoDB, ElastiCache	Relational, NoSQL, in-memory, data warehouse
Networking	VPC, Route 53, CloudFront, ELB	Connect, route, secure and distribute traffic
Security	IAM, KMS, WAF, Shield, Cognito	Identity, encryption, DDoS, access control
Monitoring	CloudWatch, CloudTrail, Config	Metrics, logs, audit trails, compliance
Messaging	SNS, SQS, EventBridge	Pub/sub, queuing, event-driven integration
DevTools	CodeBuild, CodeDeploy, CloudFormation	CI/CD pipelines and infrastructure as code

VEERA

Scenario-Based Interview Questions — AWS Cloud Computing — Overview

Q1: Scenario: Your startup runs on a single on-prem server. A celebrity tweeted about your app and traffic jumped 50x. The server crashed and your app was down for 6 hours. How would cloud computing have prevented this?

With AWS, you would use an Auto Scaling Group behind an ALB. ASG adds instances automatically when CPU/traffic spikes and removes them when traffic drops. The celebrity spike would have triggered scale-out within minutes. You pay only for the extra servers during the spike. On-prem, you would have needed to buy 50 servers months in advance — wasted money if the spike never comes, and too late if it does.

Q2: Scenario: Your company spent Rs 1.5 crore on 20 physical servers. 6 months later, only 3 are in use. The CFO asks why the cloud would have been better. Explain using CapEx vs OpEx.

Those 20 servers are CapEx — a huge upfront purchase whether you use them or not. On AWS, the same workload is OpEx — you pay monthly only for what you actually use. Those 3 busy servers might cost Rs 40,000/month on EC2, and the other 17 would cost Rs 0 because they would not exist. Cloud converts a Rs 1.5 crore bet into a Rs 40,000 monthly bill that scales with actual demand.

Q3: Scenario: A healthcare company says 'we cannot use cloud because it is not secure — our patient data will be on shared infrastructure.' How do you respond?

AWS follows the Shared Responsibility Model. AWS secures the physical infrastructure (data centres, hardware, hypervisor, network) — HIPAA-compliant, SOC 2, ISO 27001 certified. You secure your data inside: encryption at rest (KMS), encryption in transit (TLS), IAM access control, VPC isolation, and security groups. Your data on AWS is more isolated than most on-prem setups because VPCs are logically separated at the hypervisor level. Many hospitals and insurance companies already run on AWS with full compliance.

Q4: Scenario: Your manager asks: 'We have 3 apps — a static website, a MySQL database, and a custom C++ simulation that needs root OS access. Which cloud model (IaaS/PaaS/SaaS) fits each and why?'

Static website → S3 + CloudFront (effectively SaaS-level simplicity — zero server management). MySQL database → RDS (PaaS — AWS manages OS, patching, backups, failover; you manage schema and queries). C++ simulation needing root access → EC2 (IaaS — you get a full VM with root/admin, install your custom C++ runtime, control the OS completely). The rule: the more control you need, the more you manage; the less control you need, the more AWS manages.

Q5: Scenario: Your app runs in ap-south-1a (Mumbai AZ-a) only. At 2 AM, AZ-a has a power failure. Your app is completely down. What architectural mistake did you make and how do you fix it?

You violated Rule #1 of AWS architecture: always deploy across 2+ Availability Zones. Fix: place your ALB across ap-south-1a AND ap-south-1b, run your ASG in both AZs, and use RDS Multi-AZ. Now if AZ-a has a power failure, AZ-b serves all traffic automatically. The ALB detects the dead targets in AZ-a and routes to healthy ones in AZ-b. Cost increases slightly but downtime drops from hours to zero.

Q6: Scenario: A government agency says 'our data must never leave India — not even for backups.' Can they use AWS? What specific controls do they implement?

Yes. Deploy in ap-south-1 (Mumbai) or ap-south-2 (Hyderabad). AWS data never leaves a region unless you explicitly move it. Controls: use Service Control Policies (SCPs) in AWS Organizations to DENY all API calls outside ap-south-1/ap-south-2, disable S3 Cross-Region Replication to foreign regions, keep all RDS snapshots and backups region-local, and use KMS keys created in that region. This satisfies RBI and MEITY data localisation requirements.

Q7: Scenario: Your team built a mobile app. Initially 500 users, but Shark Tank episode airs next month and you expect 5 lakh users. You have Rs 2 lakh budget. Design the approach.

Use elastic architecture: S3 + CloudFront for the app's static content (handles millions of downloads at pennies). API Gateway + Lambda for the backend (scales from 0 to thousands of concurrent

requests automatically, zero servers). DynamoDB on-demand for the database (no connection limits, scales per-request). Total cost at 500 users: nearly Rs 0 (free tier). At 5 lakh users: scales automatically, pay-per-use, probably Rs 30-50K/month. No servers to pre-buy, no capacity planning. After the Shark Tank spike dies down, cost drops back automatically.

Q8: Scenario: Your company's AWS bill jumped from Rs 5 lakh to Rs 18 lakh in one month with no new projects. The CEO is furious. What are the first 5 things you investigate?

1) Cost Explorer → group by service and look for the spike — which service caused it?
2) Check for zombie resources: dev/test EC2 instances left running 24/7, unattached EBS volumes, idle Elastic IPs, forgotten NAT Gateways.
3) Check for a runaway process: a recursive Lambda, a misconfigured Auto Scaling Group that scaled to 100 instances, or a massive data transfer.
4) Check S3: versioning without lifecycle rules silently doubles storage monthly; incomplete multipart uploads accumulate invisibly.
5) Check if someone launched GPU/large instances for crypto mining after a credential leak — check CloudTrail for unfamiliar API calls from unknown IPs.

Q9: Scenario: Your team runs 50 EC2 instances 24/7 for production. The workload is steady and predictable. Another team runs batch jobs that can tolerate interruptions. A third team spins up dev servers only during work hours. What pricing model should each use?

Production (steady 24/

7) → Savings Plans or Reserved Instances (1-year commitment, ~40% discount). The workload is predictable, so committing saves significantly. Batch jobs (interruptible) → Spot Instances (up to 90% discount). If AWS reclaims the instance, the batch job restarts — no customer impact. Dev servers (work hours only) → On-Demand + scheduling (stop at 7 PM, start at 9 AM, off on weekends). This alone saves ~65% vs running 24/7. Common mistake: running everything On-Demand forever.

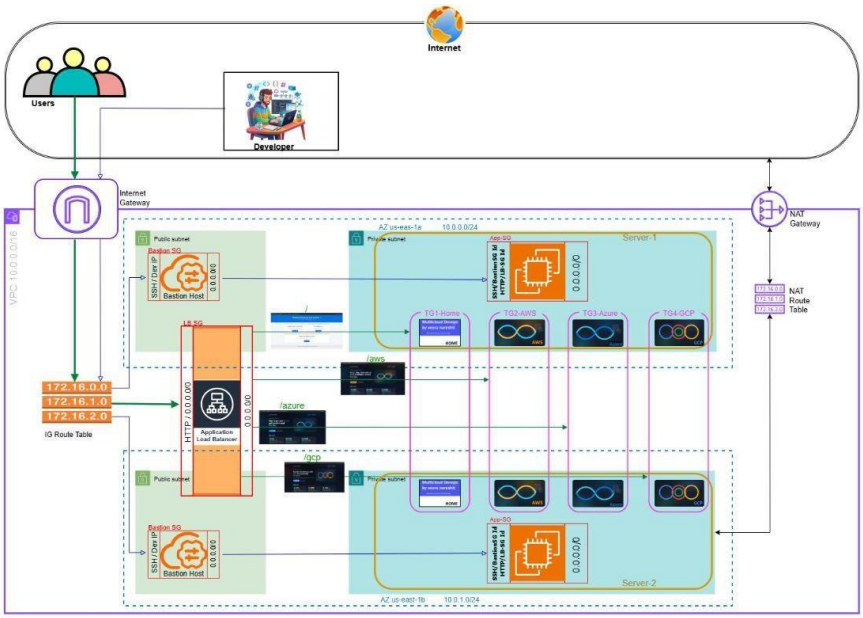
Q10: Scenario: Two teams in your company independently built apps — Team A in Mumbai region, Team B in Singapore region. Now they need to share a database. The CTO asks 'why didn't we plan this?' What went wrong and what is the correct approach from day one?

What went wrong: no cloud governance. No standardised region strategy, no IP address planning, no shared architecture review. Correct approach from day one:

1) Define a primary region (ap-south-1 for an India company).
2) Plan a company-wide CIDR allocation (10.0.0.0/16 for prod, 10.1.0.0/16 for dev — non-overlapping) so VPC peering/Transit Gateway works later.
3) Use AWS Organizations with OUs and SCPs from the start.
4) Use AWS Control Tower to enforce baseline guardrails.
5) For the current problem: use VPC Peering or Transit Gateway between regions, or replicate the database with DynamoDB Global Tables or Aurora Global Database. The lesson: cloud governance is cheaper to set up on day one than to retrofit after two teams have built incompatible silos.

2. Amazon EC2 — Elastic Compute Cloud

EC2 (Elastic Compute Cloud) provides virtual servers called instances in the cloud. It is the most fundamental compute service. Launch instances in minutes, choose any OS, control networking and security, and pay only for what you run.



1. VPC(Virtual Private Cloud) Creation by selecting CIDR.
2. Create Internet Gateway(IG) and attach it to the created VPC.
3. Create 4 Subnets in Two different Availability Zones(AZ)s (2 Subnets in one AZ and another two in other AZ).
4. Create Route Table(RT) and Route it from IG to RT(edit route) and RT to 2 Subnets(subnet association) from each AZ, so that they are Public subnets.
5. Create Nat Gateway for this VPC.
6. Route Table creation to connect from NAT to RT(edit route) and RT to Private Subnets(subnet association).
7. Create 3 Security Groups (SG)s:
   i. Load Balancer Security Group (LB SG) to inbound all HTTP requests.
   ii. Bastion host Security Group(Bastion SG) to inbound SSH from Develop IP.
   iii. Application Security Group (App SG) to inbound all HTTP requests from LB SG and SSH from Bastion SG.
8. Launch 8 EC2 servers:
   i. 4 application servers in Private subnet of us-east-1a AZ and assign the App SG.
   ii. 4 application servers in Private subnet of us-east-1b AZ and assign the App SG.
   iii. 1 Bastion server in Public subnet of us-east-1a AZ and assign the Bastion SG.
   iv. 1 Bastion server in Public subnet of us-east-1b AZ and assign the Bastion SG.
9. Login onto the Bastion host servers and copy the Key file and switch to the App servers. And the the Internet connectivity.
10. Install the Nginx server and Deploy the required Applications.
11. Create 4 path based Target Groups(TG) for each application to route the request from Load balancer to specific Applications by adding the servers.
12. Create the Load Balancer by including the two AZs (us-east-1a & us-east-1b), Assign the LB SG and the Target group of Default application.
13. Click on Add rules inside LB and chose path based routing and provide remaining 3 apps paths and respective servers.
14. Access the Applications by using the Load Balancer's DNS address.
15. Make sure all the apps are being accessing by all the servers.

EC2 Architecture: AMI → Instance Type → VPC/Subnet → Security Group → EBS Storage

2.1 How EC2 Works — Step by Step

- Step 1 — Choose AMI: Select the OS and pre-installed software template (Amazon Linux, Ubuntu, Windows Server, or a custom AMI).
- Step 2 — Choose Instance Type: Select CPU/RAM/network size. t3.micro = 2 vCPU + 1 GB RAM. m5.xlarge = 4 vCPU + 16 GB RAM.
- Step 3 — Configure Network: Choose VPC, subnet (public or private), assign public IP or not.
- Step 4 — Add Storage: Attach EBS volumes (persistent block storage). Root volume is required.

- Step 5 — Security Group: Define firewall rules — which ports accept traffic from which sources.
- Step 6 — Key Pair: SSH public/private key for Linux login. RDP password for Windows.

VEERA

- Step 7 — Launch & Connect: EC2 provisions in under 60 seconds. SSH via terminal or RDP via Remote Desktop.

Real-World Example
Startup deploys Node.js API: t3.small instance, 20GB gp3 EBS, security group allowing port 3000 from ALB only, IAM role for S3 access. App is live in under 2 minutes — no hardware purchase, no waiting.

2.2 AMI — Amazon Machine Image

An AMI is a snapshot/template containing the OS, installed software, configuration, and data needed to launch EC2. It is the 'master copy' you clone to create instances.

- AWS-Provided AMIs: Amazon Linux 2023, Ubuntu 24.04, Windows Server 2022 — ready to use.
- Marketplace AMIs: Pre-configured by vendors — WordPress, Jenkins, Deep Learning, SAP.
- Custom AMIs: You create from a running EC2 instance. Captures all your app and configuration.
- Golden AMI Pattern: Build one perfect AMI with your app pre-installed → launch 100 identical instances from it. Fast startup, consistent environment.

Key Tip
For Auto Scaling Groups, always use a custom Golden AMI with your application pre-installed. ASG launches instances in seconds (not minutes). Every instance is guaranteed identical.

2.3 Instance Types

Family	Examples	Optimized For	Use Cases
General Purpose	t3, t4g, m5, m6i	Balanced CPU+Memory+Network	Web servers, dev/test, small databases, APIs
Compute Optimized	c5, c6g, c7g	High CPU performance	Batch processing, ML inference, gaming, video encoding
Memory Optimized	r5, r6g, x1e, z1d	Large RAM (up to 24 TB)	In-memory DBs (SAP HANA), big data, real-time analytics
Storage Optimized	i3, i4i, d3	High sequential disk I/O	Data warehousing, Hadoop, Cassandra, Elasticsearch

Accelerated Computing	p3, g4dn, inf1	GPU/FPGA/ML accelerators	Deep learning training, HPC, graphics rendering

Theory Explained
Instance naming decoded: c5.2xlarge → 'c'=compute family, '5'=5th generation hardware, '2xlarge'=size (8vCPU/16GB). Always choose the latest generation — better price/performance ratio.

2.4 EBS — Elastic Block Storage

EBS provides persistent block storage for EC2. Like a hard drive attached over the network — data persists when instance stops, reboots, or terminates. AZ-specific, auto-replicated within its AZ. Max 64 TB per volume.

Type	Technology	Max IOPS	Max Throughput	Use Case
gp3 (recommended default)	SSD	16,000	1,000 MB/s	Boot volumes, web apps, dev/test, general workloads
io2 Block Express	SSD	256,000	4,000 MB/s	Mission-critical: Oracle RAC, SAP HANA, large MongoDB
st1 Throughput HDD	HDD	500 MB/s throughput	500 MB/s	Big data, Kafka, data warehouse, sequential reads
sc1 Cold HDD	HDD	250 MB/s throughput	250 MB/s	Infrequent access, cold backup, cheapest option

VEERA

2.5 Instance Store vs EBS

Not all storage is equal. Choosing wrong can cause data loss or performance bottlenecks.

Feature	EBS	Instance Store
Data persistence	Persists beyond stop/terminate/reboot	LOST on stop or terminate
Latency	Network-attached (low, ~1ms)	Physically attached (lowest, <0.1ms)

Max IOPS	256,000 (io2)	Millions (NVMe SSD)
Detach & Reattach	Yes — move between instances	No — tied to host hardware
Snapshots	Yes — backup to S3	Not supported
Use case	OS, databases, app data	Cache, tmp files, buffers, scratch space

2.6 Security Groups

A Security Group is a stateful virtual firewall at the instance level. EVERY EC2 instance must have at least one security group. Rules apply immediately — no restart needed.

- STATEFUL: Allow inbound port 80 → response traffic is automatically allowed outbound. No separate outbound rule needed for responses.
- ALLOW ONLY: Cannot write a Deny rule. To explicitly block an IP, use a Network ACL instead.
- Default: ALL inbound denied. ALL outbound allowed. You add inbound allow rules.
- Referencing other SGs: Instead of an IP range, set the source to another security group ID. Only instances with that SG can connect.

Rule	Port	Source/Dest	Purpose
Allow HTTP	80	0.0.0.0/0	Web traffic from internet
Allow HTTPS	443	0.0.0.0/0	Secure web traffic
Allow SSH	22	Your IP/32	Admin access from your laptop only
Allow DB	3306	App-Server-SG-ID	Only app servers can reach MySQL
Allow All Out	All	0.0.0.0/0	Default — allow all outbound

2.7 Key Pairs

- A key pair = public key (stored on EC2 in ~/.ssh/authorized_keys) + private key (.pem file you download once).
- SSH command: ssh -i my-key.pem ec2-user@<PUBLIC-IP>
- Default usernames: ec2-user (Amazon Linux), ubuntu (Ubuntu), Administrator (Windows).

> ⚠ **Important**
> Download your .pem file IMMEDIATELY when creating a key pair — AWS shows the private key ONCE only. If lost, the only fix is: create a new AMI from the instance → launch a new instance with a new key pair.

2.8 ENI — Elastic Network Interface

An ENI (Elastic Network Interface) is a virtual network card for EC2. Every instance gets one primary ENI automatically. You can create and attach additional ENIs.

- Each ENI has: primary private IP, optional secondary private IPs, optional public/Elastic IP, MAC address, and associated security groups.
- Primary ENI: Auto-created, cannot be detached from the instance.
- Secondary ENI: Create separately, attach/detach freely, move between instances.
- Use case — HA failover: Secondary ENI has a fixed private IP. If Instance A fails, detach ENI and attach to Instance B. Services using that private IP automatically connect to B.
- Use case — Dual-homed: One ENI in public subnet (for web traffic), another ENI in private subnet (for backend). Same instance handles both.
- Use case — Software licensing: Some licenses are MAC-address tied. Keep the MAC by moving the ENI.

VEERA

2.9 Nginx Installation on EC2

Nginx is commonly used on EC2 as a web server or reverse proxy. Install on Amazon Linux:

```
sudo yum install nginx -y          # install sudo
systemctl start nginx          # start service sudo
systemctl enable nginx         # auto-start on reboot sudo
systemctl status nginx         # verify running
```

> 💡 **Key Tip**
> After installing nginx: ensure security group allows TCP port 80 inbound from 0.0.0.0/0. Open EC2 public IP in browser → nginx welcome page confirms success.

2.10 Public IP vs Private IP vs Elastic IP

Type	Internet Accessible	Persists on Restart	Cost	Use Case

Private IP	No — VPC internal only	Yes — never changes	Free	Internal communication between AWS resources
Public IP (Dynamic)	Yes	No — changes on stop/start	Free while running	Temporary external access, dev/test only
Elastic IP (Static)	Yes	Yes — permanently assigned	Free if attached; $0.005/hr if idle	Fixed IP for DNS records, firewall whitelisting

Scenario-Based Interview Questions — Amazon EC2 — Elastic Compute Cloud

Q1: Scenario: You launched an EC2 instance with nginx installed. curl localhost works from inside the instance, but the website does not open in a browser. Walk through your debugging steps.

1) Does the instance have a public IP? If it is in a private subnet with no public IP, the browser cannot reach it.
2) Security Group: is port 80 inbound allowed from 0.0.0.0/0? This is the #1 cause — SG blocks everything inbound by default.
3) Route Table: does the subnet's route table have 0.0.0.0/0 → Internet Gateway? Without this, the subnet is not public even if it has an IGW.
4) Network ACL: is inbound port 80 AND outbound ephemeral ports 1024-65535 allowed? NACLs are stateless — this is the classic trap.
5) Is nginx bound to 0.0.0.0:80 and not 127.0.0.1:80?
6) OS firewall (iptables/firewalld) blocking?
7) Are you typing https:// when only port 80 is open?

Q2: Scenario: You stopped your EC2 instance overnight to save cost. When you started it in the morning, your DNS record and client firewall whitelist stopped working. What happened and how do you fix it permanently?

When you stop and start an EC2 instance, the auto-assigned public IP is RELEASED and a new one is assigned. Your DNS A record and the client's firewall whitelist point at the old IP, which no longer belongs to you. Fix: attach an Elastic IP (EIP). An EIP is a static public IPv4 that stays with you until you release it. Update your DNS and whitelist to the EIP once. Note: EIPs are free

while attached to a running instance, but AWS charges ~$0.005/hr when idle/unattached — and since Feb 2024, all public IPv4 addresses cost $0.005/hr.

Q3: Scenario: Your production EC2 root volume (20 GB gp3) is 95% full at 2 AM. The app will crash within the hour. Fix it without downtime.

Immediate (no reboot needed):

1) Modify the EBS volume size in the console — increase from 20 GB to 50 GB.
2) Wait for the volume to enter 'optimizing' state.
3) Extend the partition: sudo growpart /dev/nvme0n1 1.
4) Grow the filesystem: sudo resize2fs /dev/nvme0n1p1 (ext
4) or sudo xfs_growfs / (XFS). This is EBS Elastic Volumes — live resize with zero downtime. While that runs, free space immediately: clear old logs, run journalctl --vacuum-size=200M, docker system prune. Long term: move logs to CloudWatch Logs, add a separate EBS data volume, set a CloudWatch alarm on disk usage at 75%.

Q4: Scenario: Your team needs to launch 100 identical EC2 instances for a load test. Each needs Node.js, your app code, and monitoring agents. Launching each manually takes 15 minutes. How do you do this in under 5 minutes?

Use the Golden AMI pattern.

1) Launch ONE instance, install Node.js, deploy your app, configure the monitoring agent, and verify everything works.
2) Create a custom AMI from this instance (Actions → Image → Create Image).
3) Create a Launch Template referencing this AMI with the correct instance type, security group, and IAM role.
4) Launch an Auto Scaling Group with desired=100 using this template. All 100 instances boot in ~60 seconds with your app already running — no 15-minute install per instance. This is exactly how ASG achieves fast scaling.

Q5: Scenario: A developer asks: 'My ML training job needs maximum disk I/O performance. I tested gp3 but it's not fast enough. What are my options?'

Two paths:

1) io2 Block Express — provisioned IOPS SSD up to 256,000 IOPS and 4,000 MB/s. You pay for the IOPS you provision. Use this when you need both high performance AND data persistence (the data survives stops/restarts).
2) Instance Store — NVMe SSDs physically attached to the host hardware. Millions of IOPS, sub-0.1ms latency, and it is included in the instance price. BUT: data is LOST when the instance stops, terminates, or the host fails. For ML training: if you can checkpoint to S3 periodically, use an i3 or i4i instance with Instance Store for maximum speed and use S3 as the durable store.

Q6: Scenario: Your security team scanned your AWS account and found 15 EC2 instances with Security Group rules allowing SSH (port 22) from 0.0.0.0/0. Why is this dangerous and what is the fix?

0.0.0.0/0 on port 22 means every IP address on the internet can attempt SSH connections. Bots constantly scan for this and brute-force credentials. Fix immediately:

1) Restrict SSH to your office IP only: allow port 22 from 203.0.113.5/32 (your specific IP).
2) Better: remove port 22 entirely and use AWS Systems Manager Session Manager instead — it requires IAM auth, needs no open port, and logs every command to CloudTrail and S3.

3) For the Security Group rule referencing pattern: the private instances' SG should allow SSH only FROM the bastion's Security Group ID, not from any IP.

Q7: Scenario: You have 3 EC2 instances behind an ALB. Instance A handles user sessions in local memory. When Instance A goes down, 1000 users lose their sessions and are logged out. What is the root cause and the fix?

Root cause: storing sessions in local memory makes the app STATEFUL. When Instance A dies or the ALB routes a user to Instance B, the session does not exist there. Fix: externalize sessions to ElastiCache Redis. All 3 instances read/write sessions from Redis. Now any instance can serve any user, Instance A dying is invisible to users (the ALB routes to B/C, which find the session in Redis), and Auto Scaling works correctly because new instances immediately have access to all sessions. Sticky sessions is the wrong fix — it hides the problem and breaks when the stuck-to instance dies.

Q8: Scenario: Your company runs 20 m5.4xlarge instances in production. CloudWatch shows average CPU at 8% across all of them. Finance says the EC2 bill is Rs 12 lakh/month. Reduce it by at least 50%.

8% CPU on m5.4xlarge (16 vCPU, 64 GB) means massive over-provisioning. Step 1: Right-size using Compute Optimizer — an m5.4xlarge at 8% CPU probably needs m5.large (2 vCPU, 8 GB) or m5.xlarge at most. That alone is a 4-8x cost reduction per instance. Step 2: Switch to Graviton (m7g) for ~20% better price-performance. Step 3: After right-sizing, buy a 1-year Compute Savings Plan for the steady baseline — ~36% additional discount. Step 4: Schedule non-prod instances off from 8 PM to 8 AM and weekends. Combined: right-sizing (75% cut) + Savings Plan (36% on remainder) easily exceeds 50%.

Q9: Scenario: Your EC2 instance has two ENIs — one in a public subnet and one in a private subnet. A teammate asks 'why would you ever do this?' Give two real use cases.

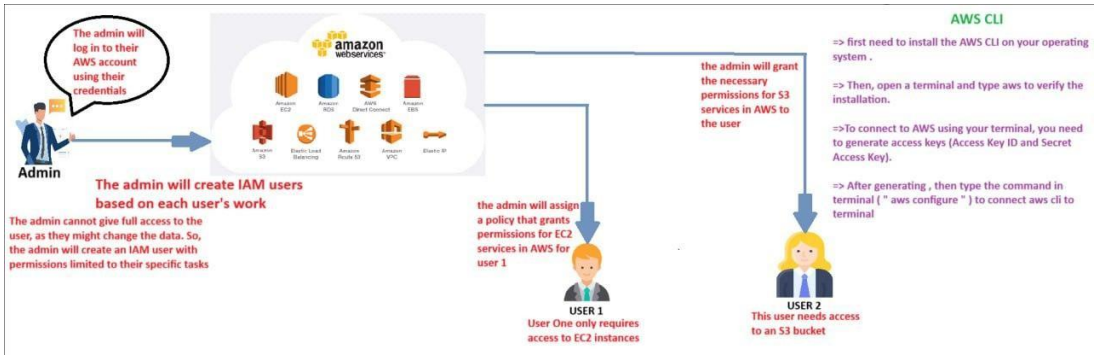
Use case 1 — Dual-homed security appliance: the public ENI receives web traffic from the internet, the private ENI connects to the backend database. The instance acts as a firewall/proxy between the two networks. Only the public ENI has a public IP; the private ENI is completely isolated from the internet. Use case 2 — Management + data separation: one ENI for production application traffic on the data subnet, and a second ENI for SSH/management access on a separate management subnet with its own security group and logging. If the management network is compromised, the data network's security group is untouched.

Q10: Scenario: You have a running production EC2 instance with critical configuration and installed software. You need to replicate this exact setup in another region for disaster recovery. What is the step-by-step process?

1) Create a custom AMI from the running instance (Actions → Image → Create Image). This captures the OS, installed software, configuration, and data.
2) Copy the AMI to the DR region (Actions → Copy AMI → select destination region).
3) In the DR region, create a Launch Template from the copied AMI with the appropriate instance type, security group, and IAM role.
4) Test by launching an instance from the template and verifying the app works.
5) For automated DR: create an ASG with min=0 in the DR region. When disaster hits, set desired=N and instances launch from the pre-copied AMI in seconds. Key: re-copy the AMI regularly as your app updates, or automate it with EventBridge + Lambda.

3. AWS IAM — Identity & Access Management

IAM (Identity and Access Management) is the security backbone of AWS. It controls WHO can access AWS resources and WHAT actions they can perform. Every single API call to AWS — console click, CLI command, SDK call — is authenticated and authorized through IAM.



IAM: Users → Groups → Roles → Policies — identity and permission structure

3.1 Root User vs IAM Users

- Root User: Created automatically when you open the AWS account. Has 100% unrestricted access. Use ONLY for: initial account setup, billing settings, account-level changes.
- IAM User: Individual accounts for people or applications. Permissions defined by attached policies. Create separate users — never share credentials.
- IAM User credentials: Console password (browser login) AND/OR Access Keys (CLI/SDK/API).

> ⚠ **Important**
> Enable MFA on root user IMMEDIATELY after account creation. If root credentials are stolen, your ENTIRE account — every region, every service, every resource — can be deleted or ransomed. MFA means stolen password alone cannot access the account.

3.2 IAM Groups

Groups are collections of IAM users. Attach policies to the GROUP — all users in the group inherit those permissions automatically. Never attach policies to individual users when groups can be used.

- Example: Create 'DBAdmins' group → attach AmazonRDSFullAccess → add 5 DBA users. All 5 get RDS access instantly. New DBA joins → add to group. DBA leaves → remove from group.
- Groups CANNOT be nested (groups within groups not supported).

- A user can belong to multiple groups (up to 10).

VEERA

3.3 IAM Roles

Roles are like user accounts but designed to be ASSUMED temporarily. They have no username or password. When assumed, AWS STS (Security Token Service) issues temporary credentials valid for 15 minutes to 12 hours.

Role Use Case	Who Assumes It	Example
EC2 Service Role	EC2 instance (automatic)	EC2 app reads/writes S3 without hardcoded keys
Lambda Execution Role	Lambda function	Lambda writes to DynamoDB and sends SNS
Cross-Account Role	IAM user in another account	Consultant switches to your account role
Federated Role	SAML/OIDC identity (AD, Google)	Employee logs in via SSO, assumes AWS role

Real-World Example

BAD — never do this: ec2.env('AWS_SECRET_KEY', 'abc123supersecret') # hardcoded in code! GOOD — always do this: Attach IAM role to EC2. AWS SDK auto-discovers credentials from instance metadata (169.254.169.254). No keys in code, no keys in environment, no keys anywhere.

3.4 IAM Policies — JSON Permission Documents

A policy is a JSON document defining permissions. Every permission in AWS is expressed as a policy. Policies are attached to users, groups, or roles.

```
{ "Version": "2012-10-17", "Statement": [ { "Effect": "Allow", "Action": ["s3:GetObject", "s3:PutObject"], "Resource": "arn:aws:s3:::mybucket/*" }, { "Effect": "Deny", "Action": "ec2:TerminateInstances", "Resource": "*" } ] }
```

- Effect: 'Allow' grants access. 'Deny' blocks access. DENY ALWAYS WINS over Allow.
- Action: The API operation. Format: service:ActionName. Example: s3:GetObject, ec2:StartInstances.
- Resource: Which AWS resource. Use full ARN or * for all. Example: arn:aws:s3:::mybucket/*.
- Condition (optional): Restrict by IP address, time, MFA status, request tags, etc.

Policy Type	Created By	Attached To	Best Use Case
AWS Managed	AWS (read-only)	Users/Groups/Roles	Quick start: AmazonS3ReadOnly, AdministratorAccess
Customer Managed	You	Users/Groups/Roles	Fine-grained custom permissions for your org
Inline	You	Single identity ONLY	Strict 1:1 — deleted when identity is deleted
Resource-Based	You	AWS Resource (S3, SQS)	Cross-account access or service-toservice

3.5 Allow vs Deny — Policy Evaluation Logic

> ⚠ **Important**
> THE MOST IMPORTANT IAM RULE: Explicit DENY always overrides ALL ALLOWs — no exceptions. Evaluation order: (1) Default DENY all. (2) Evaluate all policies. (3) Any explicit DENY found? → DENY immediately. (4) Any ALLOW found (and no DENY)? → ALLOW. (5) No match? → implicit DENY. Example: 10 policies grant ec2:*, 1 policy denies ec2:TerminateInstances → user CANNOT terminate. The single deny wins.

3.6 IAM Role Switching

- Role switching lets you temporarily assume a different role in the same or another AWS account.
- Use case: Developer role for daily work → switch to 'ProductionDeploy' role for releases → switch back. Audit trail shows exactly what was done under which role.
- Powered by STS: issues temporary access key + secret key + session token (valid 1–12 hours).

```
# CLI role switching aws sts assume-role \   --role-arn
arn:aws:iam::123456789012:role/ProductionDeploy \   --role-session-name
DeploySession
```

3.7 IAM Programmatic Access & AWS CLI

Programmatic access allows scripts, applications, and CI/CD pipelines to interact with AWS using Access Keys instead of passwords.

- Access Key ID: Public identifier, like a username. Example: AKIAIOSFODNN7EXAMPLE

- Secret Access Key: Private key, like a password. Shown ONLY ONCE at creation. Store securely.

```
# Install AWS CLI on Linux curl 'https://awscli.amazonaws.com/awscli-exe-
linuxx86_64.zip' -o awscliv2.zip unzip awscliv2.zip && sudo ./aws/install aws -
version
# Configure credentials aws configure # Prompts for: Access Key ID, Secret
Access Key, Region (e.g. ap-south-1), Output (json)
# Common AWS CLI commands aws s3 ls                          # list S3 buckets a
describe-instances          # list EC2 instances aws iam listusers
IAM users aws rds describe-db-instances       # list RDS databases
```

> ⚠ **Important**
> NEVER commit access keys to Git. Bots scan GitHub 24/7 for AWS keys. A leaked key can cost tens of thousands of dollars within hours (crypto mining bots). Use IAM roles whenever possible. If keys are leaked: go to IAM → deactivate the key → create new key IMMEDIATELY.

3.8 IAM Best Practices

- Enable MFA on root user and all privileged IAM users immediately.
- Never use root for daily operations — lock root credentials away after initial setup.
- Principle of Least Privilege: grant ONLY the minimum permissions needed, nothing more.
- Use IAM Roles for EC2, Lambda, ECS — never embed access keys in application code.
- Rotate access keys every 90 days. Audit and delete unused credentials with IAM Access Analyzer.
- Enable CloudTrail to log and audit ALL API calls across your entire AWS account.
- Use IAM groups to manage permissions — attach policies to groups, never to individual users.

VEERA

Scenario-Based Interview Questions — AWS IAM — Identity & Access Management

Q1: Scenario: A developer accidentally pushed AWS access keys to a public GitHub repo. Within 20 minutes, 200 GPU instances are mining crypto in your account. What do you do, in order?

1) IMMEDIATELY: go to IAM → deactivate and delete the compromised access key. This stops the attacker's access.

2) Terminate all rogue EC2 instances and delete any IAM users/roles/keys the attacker created for persistent access.
3) Check CloudTrail: search for the compromised access key to find the full blast radius — what else did they touch?
4) Rotate every credential the developer had, plus anything the compromised keys could read from Secrets Manager.
5) Remove the key from Git history using BFG Repo-Cleaner — deleting the file in a new commit is NOT enough, the old commit still has it.
6) Contact AWS Support — they often waive fraudulent charges for first incidents.
7) Prevent recurrence: ban long-lived access keys, use IAM Roles for everything, enable git-secrets pre-commit hooks, and enable GitHub secret scanning.

Q2: Scenario: Your company has 50 developers. Each has individually attached IAM policies. A new security rule says 'all developers must lose S3 delete permissions.' Changing 50 policies one by one will take hours. What should you have done and how do you fix it now?

What you should have done: use IAM Groups. Create a 'Developers' group, attach policies to the group, and add all 50 users to it. Changing one group policy instantly applies to all 50 users. Fix now:

1) Create the 'Developers' group with the corrected policy (S3 access without s3:DeleteObject/s3:DeleteBucket).
2) Add all 50 users to the group.
3) Remove the individually attached policies from each user. Going forward: never attach policies directly to users. Attach to groups. New developer joins → add to group. Developer leaves → remove from group.

Q3: Scenario: Your EC2 application needs to read from S3 and write to DynamoDB. A junior developer stored AWS access keys in the application's .env file baked into the AMI. Why is this wrong and what is the correct approach?

This is wrong for 3 reasons:

1) The keys are permanently baked into every instance launched from this AMI — including dev, staging, and any snapshots shared with other teams.
2) Anyone who gets the AMI or the instance can extract the keys.
3) The keys never rotate automatically. Correct approach: create an IAM Role with a policy allowing s3:GetObject and dynamodb:PutItem on the specific resources. Attach this role to the EC2 instance via an Instance Profile. The AWS SDK automatically discovers temporary credentials from the instance metadata service (169.254.169.254). No keys in code, no keys on disk, credentials rotate automatically every few hours.

Q4: Scenario: A policy allows s3:* on all resources. Another policy attached to the same user explicitly denies s3:DeleteBucket. Can the user delete a bucket? Explain the evaluation logic.

No, the user CANNOT delete a bucket. IAM evaluation logic:

1) Start with default implicit DENY on everything.
2) Evaluate all attached policies.
3) The s3:* Allow grants all S3 actions.
4) The explicit Deny on s3:DeleteBucket is found.

5) Explicit DENY ALWAYS overrides ALL Allows — no exceptions. Result: the user can do everything on S3 (GetObject, PutObject, ListBucket, etc.) EXCEPT delete buckets. This is how you grant broad permissions with surgical restrictions — a common and powerful pattern.

Q5: Scenario: Your company has 3 AWS accounts: Dev, Staging, and Prod. A senior engineer needs read-only access to Prod logs, but you must NOT create an IAM user in the Prod account. How?

Use cross-account role assumption. In the Prod account: create a role 'ProdReadOnlyLogs' with (a) permission policy = CloudWatchLogsReadOnlyAccess and (b) trust policy allowing the Dev account (arn:aws:iam::<DEV-ACCOUNT-ID>:root) with a Condition requiring aws:MultiFactorAuthPresent = true. In the Dev account: attach a policy to the engineer allowing sts:AssumeRole on the Prod role ARN. Now: the engineer logs into Dev with MFA, runs 'aws sts assume-role', gets temporary credentials for Prod, and can read logs only. No user created in Prod, temporary credentials expire automatically, every action logged in Prod's CloudTrail under the assumed-role identity.

Q6: Scenario: An intern with the AWS-managed 'PowerUserAccess' policy accidentally deleted a production S3 bucket with customer invoices. Management demands this never happens again. What layered controls do you implement?

Layer 1 — Least privilege: replace PowerUserAccess with a scoped custom policy. Interns do not need delete rights on production resources. Layer 2 — Explicit Deny: attach a policy with Effect: Deny on s3:DeleteBucket and s3:DeleteObject for the production bucket ARNs. Explicit deny overrides any allow. Layer 3 — SCP: in AWS Organizations, add a Service Control Policy on the Production OU that denies destructive S3 actions — this blocks even account admins. Layer 4 — S3 protections: enable Versioning (deletion only adds a delete marker, data recoverable), MFA Delete, and Object Lock in Compliance mode (nobody can delete, not even root). Layer 5 — Account separation: interns should not have credentials in the Prod account at all. Give them access only to the Dev account.

Q7: Scenario: Your CI/CD pipeline in Jenkins uses a long-lived access key stored as a Jenkins credential. The security team says this is a risk. Why, and what is the alternative?

Risk: the access key never expires, is stored in Jenkins' credential store (which can be dumped), and anyone with Jenkins admin access can see it. If Jenkins is compromised, the attacker has permanent AWS access. Alternative: use IAM Roles. If Jenkins runs on EC2 — attach an IAM Role to the EC2 instance. If Jenkins runs outside AWS — use OIDC federation: configure an IAM Identity Provider for Jenkins, create a role with a trust policy that trusts the OIDC provider, and Jenkins exchanges its identity token for temporary AWS credentials via sts:AssumeRoleWithWebIdentity. No long-lived keys anywhere, credentials expire in 1 hour, and each pipeline run gets its own scoped session.

Q8: Scenario: You wrote this IAM policy but it does not work — the user still cannot read from the S3 bucket. Find the bug: Effect: Allow, Action: s3:GetObject, Resource: arn:aws:s3:::my-bucket

The Resource ARN is wrong. arn:aws:s3:::my-bucket refers to the BUCKET itself (the container), not the OBJECTS inside it. s3:GetObject operates on objects, not buckets. Fix: change Resource to arn:aws:s3:::my-bucket/* — the /* means all objects inside the bucket. For listing the bucket contents (s3:ListBucket), you DO use the bucket ARN without /*. This bucket-vs-object ARN distinction is the single most common IAM policy bug for S3.

Q9: Scenario: You need to give a third-party analytics vendor temporary access to read specific S3 objects in your account. They are NOT an AWS customer and cannot assume roles. What do you use?

Use S3 Pre-Signed URLs. Your backend (which has the IAM permissions) generates a time-limited signed URL for the specific object — e.g., valid for 1 hour. You send this URL to the vendor. They access the object via a normal HTTPS GET without any AWS credentials. The URL carries your permissions and expires after the TTL. This is the correct pattern for giving non-AWS users temporary, scoped access to specific S3 objects without making the bucket public or sharing any credentials.

Q10: Scenario: A developer can launch EC2 instances in the Dev account but should NOT be able to launch anything larger than t3.medium. The current policy allows ec2:RunInstances on all resources. How do you restrict it?

Add a Condition block to the policy that restricts the instance type. Use the ec2:InstanceType condition key: Effect: Allow, Action: ec2:RunInstances, Resource: *, Condition: { StringEquals: { 'ec2:InstanceType': ['t3.micro', 't3.small', 't3.medium'] } }. Now the developer can launch instances, but any attempt to launch an m5.xlarge or p3.8xlarge is denied. This is how you prevent runaway costs in dev accounts. Combine with an SCP at the Organization level as a safety net: deny ec2:RunInstances where instance type is NOT in the approved list.

4. AWS VPC & Networking

VEERA

A VPC (Virtual Private Cloud) is your own private, isolated network inside AWS. You define IP address ranges, create subnets, control routing, and set up security rules. Every AWS resource (EC2, RDS, Lambda) lives inside a VPC.

4.1 CIDR — IP Address Ranges

CIDR (Classless Inter-Domain Routing) defines IP address ranges. You use CIDR everywhere in AWS — VPC ranges, subnet ranges, security group rules, route table entries.

- Format: IP/prefix. Example: 10.0.0.0/16 means IPs from 10.0.0.0 to 10.0.255.255 (65,536 total).
- Larger prefix = smaller network. /16 = 65,536 IPs. /24 = 256 IPs. /32 = 1 IP.

CIDR	Total IPs	AWS Usable (−5)	Typical Use
/16	65,536	65,531	Entire VPC (recommended default)
/24	256	251	Standard subnet (most common choice)
/28	16	11	Very small subnet (NAT GW, bastion only)
/32	1	1	Single IP in security group rules (your laptop)

Theory Explained
AWS reserves 5 IPs per subnet automatically: • x.x.x.0 — Network address • x.x.x.1 — VPC Router (default gateway) • x.x.x.2 — AWS DNS resolver • x.x.x.3 — Reserved for future use • x.x.x.255 — Broadcast address So /24 subnet = 256 - 5 = 251 usable IPs.

4.2 Subnets — Public, Private, and Isolated

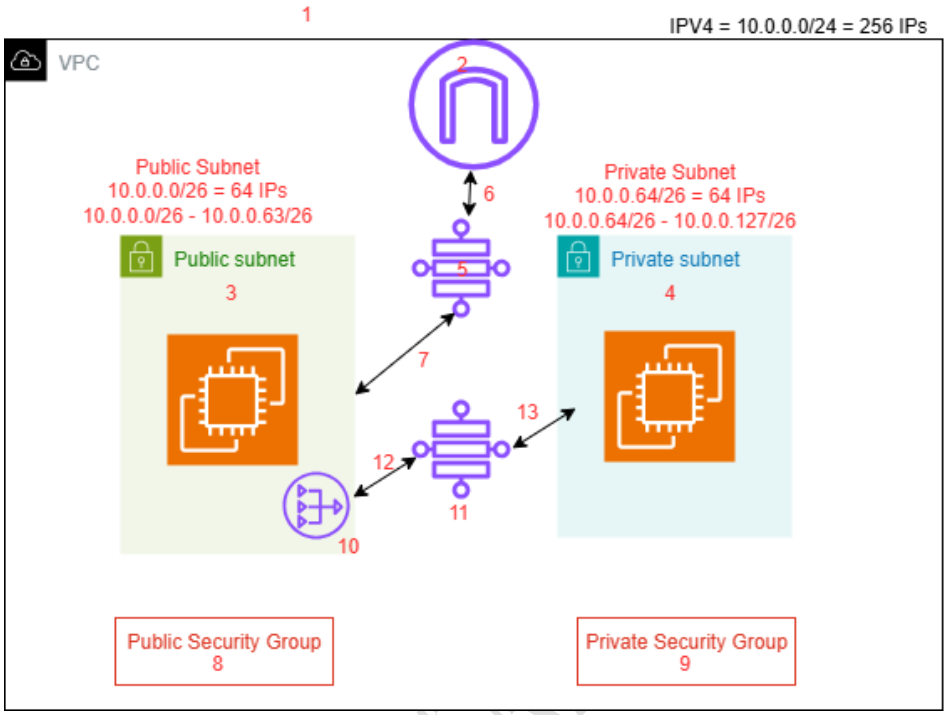
Type	Internet Access	Route Table	What Belongs Here
Public Subnet	Yes (via IGW)	0.0.0.0/0 → IGW	ALB, NAT Gateway, Bastion Host, web servers
Private Subnet	Outbound only (via NAT GW)	0.0.0.0/0 → NAT GW	App servers, ECS tasks, Lambda in VPC
Isolated Subnet	None — zero internet	Local routes only	RDS databases, ElastiCache, internal-only services

Real-World Example
Best-practice 3-tier VPC across 2 AZs: • 2 Public subnets (AZ-a + AZ-b): ALB + NAT Gateways • 2 Private subnets (AZ-a + AZ-b): EC2 app servers in ASG • 2 Isolated subnets (AZ-a + AZ-b): RDS Multi-AZ Total 6 subnets, full fault tolerance, database never exposed.

4.3 Internet Gateway (IGW)

- IGW connects your entire VPC to the public internet. Without IGW, nothing in your VPC can reach the internet.
- Highly available, horizontally scalable — no bandwidth bottleneck. Managed by AWS.
- One IGW per VPC only. Attach at VPC level, not subnet level.
- A subnet becomes 'public' only when its route table points 0.0.0.0/0 to the IGW.
- Instance also needs a public or Elastic IP to be reachable from the internet.

4.4 NAT Gateway — Private Subnet Internet Access



NAT Gateway: Private EC2 → NAT GW (public subnet) → Internet Gateway → Internet

Private subnet instances need internet access for OS updates, downloading packages, calling external APIs — but must NOT be reachable from the internet. NAT Gateway solves this perfectly.

- Deploy NAT Gateway in a PUBLIC subnet. Assign it an Elastic IP.
- Update PRIVATE subnet route table: 0.0.0.0/0 → NAT Gateway ID.
- Private EC2 sends request → NAT GW translates source IP to its Elastic IP → sends to internet.
- Return traffic arrives at NAT GW → translates back to private EC2 IP → delivered internally.
- The private EC2's IP is NEVER visible to the internet. Inbound connections from internet are impossible.

Feature	NAT Gateway (Managed)	NAT Instance (Legacy)
Management	Fully managed by AWS	You manage EC2 instance
High Availability	HA within AZ automatically	Manual HA setup required
Bandwidth	Up to 100 Gbps	Limited by instance type
Maintenance	AWS patches automatically	You patch the OS

Use	Always use this	Legacy only — avoid new projects

4.5 Route Tables

Every subnet is associated with one route table. Route tables contain rules (routes) that determine where traffic goes.

Destination	Target	Meaning
10.0.0.0/16	local	Traffic within the VPC stays internal
0.0.0.0/0	igw-xxxxx	All other traffic → Internet Gateway (public subnet)
0.0.0.0/0	nat-xxxxx	All other traffic → NAT Gateway (private subnet)
10.1.0.0/16	pcx-xxxxx	Traffic to peered VPC goes through peering connection
10.0.0.0/8	tgw-xxxxx	Corporate network traffic → Transit Gateway

4.6 Security Groups vs Network ACLs

Feature	Security Groups	Network ACLs (NACLs)
Level	Instance (per EC2, RDS, Lambda)	Subnet (applies to all resources in subnet)
Rules	Allow ONLY (no deny rules)	Allow AND Deny (explicit blocking)
State	Stateful — return traffic autoallowed	Stateless — must allow BOTH directions
Evaluation	All rules evaluated together	Rules evaluated in number order (lowest first)
Default (new custom)	Deny all inbound, allow all outbound	All rules ALLOW (default NACL)
Use case	Fine-grained per-resource firewall	Block specific IPs at subnet boundary

4.7 Bastion Host (Jump Server)

A Bastion Host is a hardened EC2 in the PUBLIC subnet used as a secure gateway to SSH into private instances. Private instances have no public IP — you SSH to bastion first, then hop to the private instance.

Architecture Flow
Your Laptop → SSH port 22 → Bastion Host (public subnet, has Elastic IP) → SSH port 22 → Private EC2 (private subnet, private IP only, no public IP, no internet exposure).

- Bastion security group: Allow SSH (22) inbound ONLY from YOUR specific IP address (/32). Not 0.0.0.0/0.
- Private EC2 security group: Allow SSH (22) ONLY from bastion's security group ID.
- Modern alternative: AWS Systems Manager Session Manager — SSH to private instances via browser. No bastion needed, no port 22 open, no key pair needed. More secure and auditable.

4.8 VPC Peering

VPC Peering creates a private network link between two VPCs. After peering, instances communicate using private IPs as if on the same network. Works across regions and across AWS accounts.

- CIDRs must NOT overlap between peered VPCs. Cannot peer 10.0.0.0/16 with 10.0.0.0/16.
- After creating peering connection, update route tables in BOTH VPCs.

⚠ Important
No Transitive Peering: VPC-A peers VPC-B. VPC-B peers VPC-C. VPC-A CANNOT reach VPCC through VPC-B. Each pair needs a direct peering connection. For many VPCs, use Transit Gateway instead.

4.9 VPC Endpoints

VPC Endpoints let EC2 in private subnets access AWS services privately — without internet, without NAT Gateway, without leaving AWS backbone network. More secure AND lower cost.

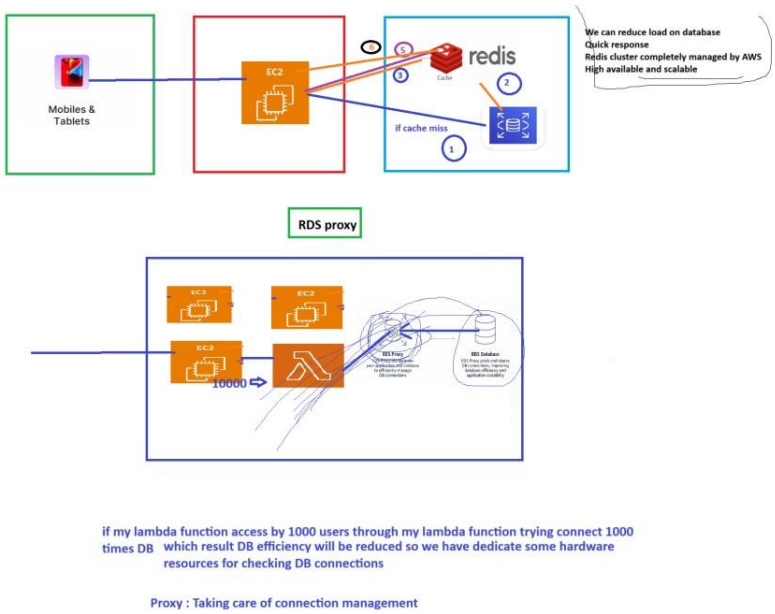
Endpoint Type	Services Supported	Implementation	Cost
Gateway Endpoint	S3 and DynamoDB ONLY	Route table entry	FREE

Interface Endpoint (PrivateLink)	Most other AWS services	ENI with private IP in subnet	~$0.01/hr + data transfer

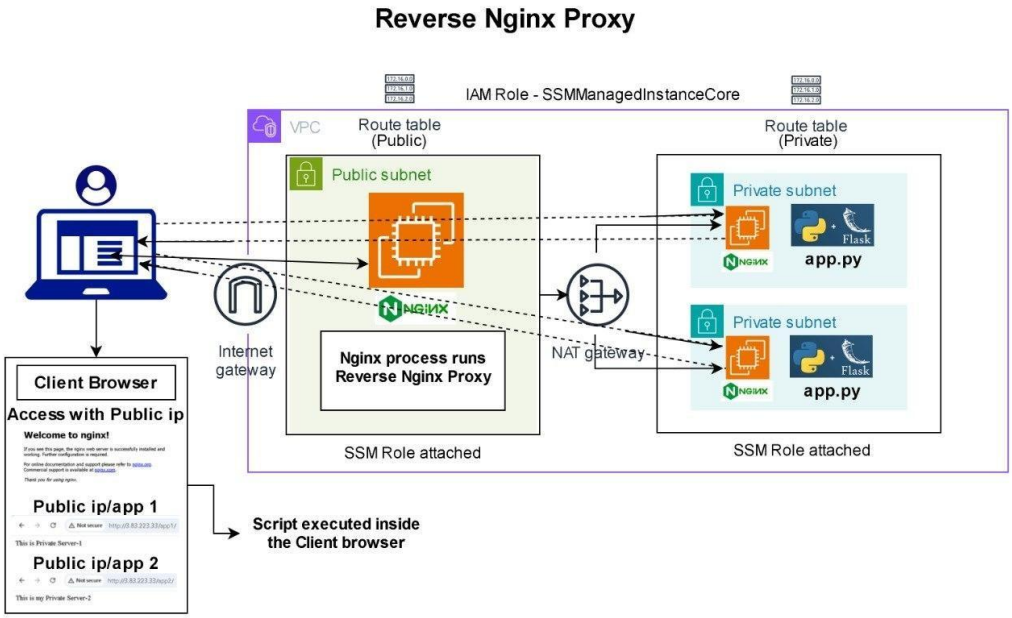
Key Tip

Use S3 Gateway Endpoint in every VPC. Without it, private EC2 accessing S3 goes through NAT Gateway — which costs $0.045/GB data processed. With the free Gateway Endpoint, traffic is free and faster.

4.10 Forward Proxy vs Reverse Proxy



Forward Proxy: Client → Proxy → Internet. Proxy acts on behalf of CLIENT



Reverse Proxy: Internet → Proxy → Backend Servers. Proxy acts on behalf of SERVER

Feature	Forward Proxy	Reverse Proxy
Acts on behalf of	CLIENT (outgoing requests)	SERVER (incoming requests)
Hides	Client's IP address	Backend server details
Direction	Client → Proxy → Internet	Internet → Proxy → Backend
AWS equivalent	Network Firewall, SquidProxy on EC2	ALB, CloudFront, Nginx, API Gateway
Use case	Corporate internet filtering, caching	Load balancing, SSL termination, security

Scenario-Based Interview Questions — AWS VPC & Networking

Q1: Scenario: Design a production VPC for a 3-tier app in Mumbai (ap-south-1) that survives an AZ failure. Give actual subnet CIDRs.

VPC CIDR: 10.0.0.0/16. Six subnets across 2 AZs: • Public: 10.0.1.0/24 (ap-south-1a), 10.0.2.0/24 (ap-south-1b) — for ALB and NAT Gateways. • Private App: 10.0.11.0/24 (1a), 10.0.12.0/24 (1b) — for EC2/ECS app servers. • Isolated DB: 10.0.21.0/24 (1a), 10.0.22.0/24 (1b) — for RDS Multi-AZ. Two NAT Gateways — one per AZ — so AZ-a failure does not kill AZ-b's outbound internet. IGW attached to VPC. Route tables: public subnets → 0.0.0.0/0 → IGW; private subnets → 0.0.0.0/0 → NAT-GW in same AZ; DB subnets → local only. S3 Gateway Endpoint on private route tables to avoid NAT cost for S3 traffic.

Q2: Scenario: Your private EC2 instance cannot run 'yum update' or reach the internet. It has the correct IAM role. Debug step by step.

IAM roles have nothing to do with internet access — that is a red herring. Check in order:

1) Route table of the private subnet — is there 0.0.0.0/0 → NAT Gateway? (Most common missing piece.)
2) Is the NAT Gateway in a PUBLIC subnet? A NAT GW placed in a private subnet is the classic mistake — it needs the IGW route itself.
3) Does the NAT GW have an Elastic IP and is its status 'Available'?
4) Does the public subnet's route table have 0.0.0.0/0 → IGW?
5) Security Group outbound: is port 443/80 allowed outbound? (Default allows all outbound, but someone may have restricted it.)
6) NACL: outbound 443 allowed AND inbound ephemeral ports 1024-65535 allowed for return traffic? NACLs are STATELESS — you must allow both directions.
7) DNS: is enableDnsSupport and enableDnsHostnames ON for the VPC?

Q3: Scenario: A malicious IP (198.51.100.99) is sending thousands of requests to your application. Your Security Groups only support Allow rules. How do you block this specific IP?

Use a Network ACL (NACL) on the public subnet. Add a DENY rule with a LOW number (e.g., Rule 1

0) for source 198.51.100.99/32, port All, Action DENY. Since NACLs evaluate rules in number order and stop at the first match, Rule 10 (DENY) fires before Rule 100 (ALLOW all). This is the exact use case for NACLs: Security Groups cannot deny, but NACLs can. For ongoing protection: add AWS WAF with a rate-based rule or an IP block-list rule on your ALB/CloudFront to handle this at Layer 7 as well.

Q4: Scenario: Two teams each built a VPC using 10.0.0.0/16. Now they must communicate. VPC Peering fails. What is the problem and what are the solutions?

Problem: VPC Peering (and Transit Gateway) require NON-OVERLAPPING CIDRs. Both VPCs use the identical range 10.0.0.0/16, so the router cannot determine which side owns an IP address. Solutions:

1) Re-IP one VPC — build a new VPC as 10.1.0.0/16 and migrate workloads (the correct permanent fix).
2) AWS PrivateLink — expose only the specific service behind an NLB as an endpoint. PrivateLink works across overlapping CIDRs because it uses ENIs, not routing.
3) The lesson: establish a company-wide CIDR allocation plan (use AWS VPC IPAM) BEFORE anyone creates a VPC. This is the #1 networking mistake companies make.

Q5: Scenario: Your private subnet EC2 instances write 10 TB/month to S3. The NAT Gateway data processing bill alone is Rs 4 lakh/month. How do you eliminate this cost?

Create an S3 Gateway Endpoint and attach it to the private subnet's route table. Gateway Endpoints for S3 are completely FREE. S3 traffic from private instances goes over the AWS private backbone instead of through the NAT Gateway. The entire Rs 4 lakh NAT processing charge for S3 traffic drops to Rs 0 instantly. Also create a DynamoDB Gateway Endpoint (also free) if your app uses DynamoDB. For other AWS services (Secrets Manager, ECR, CloudWatch), use Interface Endpoints (PrivateLink) — they cost ~$0.01/hr but are still far cheaper than NAT for high-volume traffic.

Q6: Scenario: You need to SSH into a private EC2 instance for debugging. It has no public IP. What are your two options and which is better?

Option 1 — Bastion Host: deploy a small EC2 in a public subnet. SSH from your laptop to the bastion (allow port 22 only from YOUR IP /32), then SSH from the bastion to the private instance (private SG allows SSH only from the bastion's SG ID). Option 2 — AWS Systems Manager Session Manager: no bastion needed, no port 22 open anywhere, no key pair needed. The private instance runs the SSM agent, and you connect via the AWS console or CLI. Every keystroke is logged to CloudTrail and S3. Session Manager is better: no bastion to patch, no port 22 attack surface, full audit trail, and works even without internet if you set up SSM VPC endpoints.

Q7: Scenario: Your Security Groups reference each other in a chain: ALB-SG allows 443 from 0.0.0.0/0, App-SG allows 8080 from ALB-SG, DB-SG allows 3306 from App-SG only. Explain why this is superior to using IP-based rules.

With SG-to-SG referencing:

1) As Auto Scaling adds/removes instances, the rule automatically applies because it follows the group membership, not a hardcoded IP. No manual rule updates needed.
2) The database is mathematically unreachable from the internet — there is no path from 0.0.0.0/0 through to DB-SG, even if someone misconfigures a route table.
3) It is self-documenting: 'only app servers can reach the DB' is explicit in the rule. With IP-based rules: you would need to update the CIDR list every time an instance changes IP, which fails with auto-scaling, and a single wrong CIDR could expose the database.

Q8: Scenario: You set up VPC Peering between VPC-A, VPC-B, and VPC-B, VPC-C. VPC-A cannot communicate with VPC-C. Why?

VPC Peering is NOT transitive. VPC-A peered with VPC-B and VPC-B peered with VPC-C does NOT mean VPC-A can reach VPC-C through VPC-B. Each pair requires its own direct peering connection. For 3 VPCs, you need 3 peering connections (A↔B, B↔C, A↔C). For 10 VPCs, you would need 45 connections. At that scale, use AWS Transit Gateway instead — a hub-and-spoke model where each VPC connects once to the TGW and can reach all others. TGW also supports route tables for granular control of which VPCs can communicate.

Q9: Scenario: A new developer says 'I put my RDS database in a public subnet with Publicly Accessible = Yes so I can connect from my laptop. It works fine.' Explain why this must be changed immediately.

This is a critical security violation. That database is now directly reachable from the entire internet on port 3306/5432. Bots scan for open database ports continuously — brute-force attacks start within minutes. Even with a strong password, you are one exploit away from a full data breach. Fix:

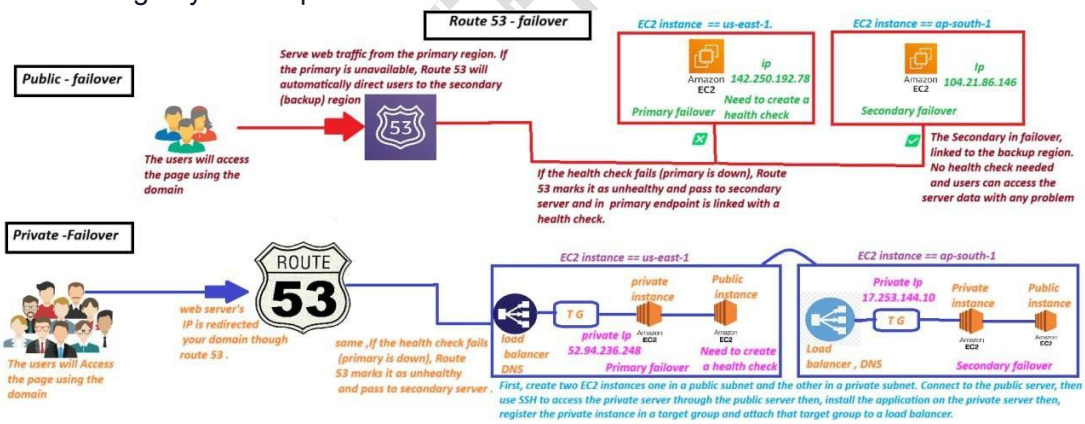
1) Move RDS to an isolated/private subnet with NO internet route.
2) Set Publicly Accessible = No.
3) Connect via a bastion host or Session Manager from your laptop.
4) The DB Security Group should allow port 3306 ONLY from the App-SG, not from any IP. The developer can use SSH tunneling through the bastion to run queries locally. Production databases must never be internet-accessible.

Q10: Scenario: Your company has 15 VPCs across 4 AWS accounts. They all need to communicate, and some need on-prem connectivity via a VPN. VPC Peering is becoming unmanageable. What is the solution?

Use AWS Transit Gateway (TGW). Attach all 15 VPCs and the VPN connection to a single TGW. This replaces the mesh of peering connections (which would require 105 connections for full connectivity) with 15 attachments to one hub. TGW supports route tables, so you can control which VPCs can talk to which — for example, isolate the production VPCs from dev, or allow dev VPCs to reach on-prem but not production databases. Cross-account attachments are supported via Resource Access Manager (RAM). For multi-region, use TGW Peering between regional TGWs. This is the standard architecture for enterprise-scale AWS networking.

5. Amazon Route 53 — DNS & Domain Service

Route 53 is AWS's scalable, highly available DNS service. The name '53' refers to DNS port 53. It provides three functions: (1) domain registration, (2) DNS routing of traffic to your resources, and (3) health checking of your endpoints.



Route 53: Domain → Hosted Zone → Records → Routing Policy → Health Check → Target

5.1 How DNS Works — Full Step-by-Step

Theory Explained

1. User types 'app.mycompany.com' in Chrome browser. 2. Browser checks local DNS cache (TTL not expired?). Not found. 3. OS sends query to configured DNS resolver (usually ISP resolver or

8.8.8.8). 4. Resolver asks root DNS servers → finds .com name servers. 5. .com name servers say 'mycompany.com is handled by Route 53 NS: ns-123.awsdns.com'. 6. Resolver asks Route 53 name servers → Route 53 applies routing policy → returns IP address. 7. Resolver caches result for TTL duration. Browser connects to that IP. Total time < 100ms.

5.2 Hosted Zones

- **Public Hosted Zone:** Accessible from the internet. Anyone can query. Used for public websites and APIs. Example: myapp.com public zone.
- **Private Hosted Zone:** Accessible ONLY from inside specified VPCs. For internal service discovery. Example: database.internal → 10.0.2.50. Only EC2 in the VPC can resolve this — internet cannot.
- One domain = one hosted zone. Hosted zone cost: ~$0.50/month.
- Each hosted zone has 4 name servers (NS records) — always use ALL 4 for redundancy.

5.3 DNS Record Types

Record	Purpose	Example
A	Maps hostname to IPv4 address	api.myapp.com → 52.10.20.30
AAAA	Maps hostname to IPv6 address	api.myapp.com → 2001:db8::1
CNAME	Maps hostname to another hostname	www.myapp.com → myapp.com
Alias	AWS-specific: maps to AWS resource	myapp.com → my-alb.us-east-1.elb.amazonaws.com
MX	Mail server records	Email routing for domain
TXT	Text — domain verification	SPF, DKIM records for email; Google verify
NS	Name server records	Which DNS servers are authoritative for domain
SOA	Start of Authority	Primary info about the hosted zone

5.4 Alias vs CNAME — Critical Difference

Feature	Alias Record	CNAME Record
Root domain	YES — works for example.com	NO — subdomains only (www.example.com)
Points to	AWS resources ONLY (ALB, CF, S3, API GW)	Any hostname anywhere
Cost	FREE — no DNS query charges	Charged per DNS query
Auto-update	Yes — auto-follows AWS resource IP changes	No — static, manual update
Recommendation	Always use for AWS resources	Non-AWS destinations only

> ⚠ **Important**
> Rule: Pointing to an AWS service? Always use ALIAS. Alias works at root domain (example.com), CNAME does NOT. Alias is FREE, CNAME has query costs. ALB/CloudFront IPs change — Alias auto-follows, CNAME breaks.

5.5 Routing Policies — All Types

1. Simple Routing

- Default DNS. One domain name → one or multiple IPs. No health checks supported.
- Multiple IPs: Route 53 returns ALL of them, client randomly picks one.
- Use when: single resource, simple static site, no failover needed.

2. Weighted Routing

- Assign a weight (0–255) to each record. Traffic distributed proportionally.
- Example: Weight 70 + Weight 30 → approximately 70% to first resource, 30% to second.
- Weight 0 = no traffic sent (useful to disable without deleting the record).
- Use case: A/B testing, canary deployments, gradual rollout of new version.

3. Latency-Based Routing

- Route 53 measures latency between user and each AWS region where you have resources.
- Routes user to the region with LOWEST network latency at that moment.
- Not geographic — a user in Mumbai might get routed to Singapore if Singapore is faster.
- Use case: Global apps requiring fastest possible response time for users worldwide.

4. Failover Routing

- Primary resource: your main production server. Secondary resource: your DR/backup server.
- Route 53 continuously health-checks primary. If it FAILS → all traffic automatically moves to secondary.
- When primary recovers health check → traffic automatically returns to primary.
- Use case: Active-passive disaster recovery. Example: us-east-1 primary, us-west-2 failover.

5. Geolocation Routing

- Routes based on geographic location of the user (determined by user's IP address).
- Configure: Users from IN (India) → India server. Users from DE (Germany) → Germany server. Default → global server.
- Use case: Data compliance (EU data must stay in EU), language-specific content, regional pricing.

6. Multi-Value Answer

- Returns up to 8 healthy IP addresses per DNS query. Client randomly picks one.
- Supports health checks — unhealthy resources automatically excluded from responses.
- Not a load balancer, but provides basic client-side distribution.

VEERA

5.6 Route 53 Health Checks

- Route 53 periodically sends HTTP/HTTPS/TCP requests to your endpoints from multiple global locations.
- Configuration: check interval (10s or 30s), failure threshold, string matching in response body.
- Used with: Failover, Weighted, Latency, Geolocation routing to remove unhealthy resources automatically.
- Can monitor CloudWatch alarms — for checking internal/private resources Route 53 can't directly reach.

Scenario-Based Interview Questions — Amazon Route 53 — DNS & Domain Service

Q1: Scenario: You need to migrate your website to a new server IP with zero user impact. The current DNS TTL is 86400 seconds (24 hours). What is your step-by-step plan?

1) One week ahead: lower the TTL from 86400 to 60 seconds.
2) WAIT a full 24 hours — every DNS resolver worldwide must first expire the cached 86400 entry before they even learn about the new 60s TTL. This waiting step is what everyone forgets.
3) Build and fully test the new server.
4) Change the A record to the new IP. Because TTL is now 60s, the world switches over in ~1 minute.
5) Keep the OLD server running for several hours — some resolvers ignore TTLs. Monitor traffic on both IPs.
6) Once old server traffic is zero, decommission it and raise TTL back to 3600-86400 to reduce query cost and DNS load.

Q2: Scenario: Your app runs in Mumbai and Frankfurt. Indian users should go to Mumbai, European users should go to Frankfurt. If Mumbai goes down, ALL users should automatically go to Frankfurt. Design the Route 53 configuration.

Use Latency-Based Routing with Health Checks. Create two A-Alias records for app.company.com: Record 1 → Mumbai ALB, routing policy = Latency, region = ap-south-1, health check on Mumbai ALB enabled. Record 2 → Frankfurt ALB, routing policy = Latency, region = eu-central-1, health check on Frankfurt ALB enabled. Normally: Indian users get routed to Mumbai (lower latency), Europeans to Frankfurt. If Mumbai's health check fails: Route 53 removes the Mumbai record from the answer set entirely. Now Indian users receive the Frankfurt IP automatically. Failover comes built-in with latency routing when health checks are enabled — you do not need a separate failover policy.

Q3: Scenario: You bought your domain on GoDaddy but want to use Route 53 for DNS. After switching nameservers, your company email stopped working for a day. What happened?

You switched the nameservers to Route 53 BEFORE recreating the MX records in the Route 53 hosted zone. When the NS switch propagated, DNS queries for your domain went to Route 53, which had no MX record, so email delivery failed silently — mail servers could not find where to deliver. Fix: ALWAYS recreate ALL existing DNS records (A, CNAME, MX, TXT, SPF, DKIM) in Route 53 FIRST, verify them with dig, and ONLY THEN change the nameservers at GoDaddy. The MX record is the one people forget most often because they do not test email during migration — they only test the website.

Q4: Scenario: A developer asks: 'I want to point my naked domain (example.com — without www) to our ALB. I tried a CNAME but Route 53 says it is not allowed at the zone apex. What do I use?'

Use an Alias record instead of CNAME. The DNS RFC forbids CNAME at the zone apex (the naked domain) because it conflicts with the required SOA and NS records. Route 53's Alias record is a proprietary extension that solves this: it resolves the domain directly to the AWS resource's IP addresses inside Route 53, returning the IPs in one lookup (no extra DNS hop). Three bonus advantages:

1) Alias queries to AWS resources are FREE (CNAME queries are billed).
2) Alias is faster — one lookup instead of two.
3) Alias health-checks the target automatically. Rule: whenever you point at an AWS resource (ALB, CloudFront, S3, API Gateway), always use Alias.

Q5: Scenario: You are doing a canary deployment — 95% of traffic should go to the stable server, 5% to the new version. Which Route 53 routing policy do you use and how do you configure it?

Use Weighted Routing. Create two A records for the same domain: Record 1 → stable ALB, weight = 95. Record 2 → new version ALB, weight = 5. Route 53 returns each record in proportion to its weight. ~5% of DNS resolutions return the new server's IP. Combine with health checks: if the new version starts failing, Route 53 automatically stops returning it and sends 100% to the stable server. After confirming the canary is healthy, gradually increase the weight: 5 → 20 → 50 → 100. This gives you real-user validation with a tiny blast radius.

Q6: Scenario: Your Route 53 health check is configured to monitor your ALB's /health endpoint. The health check shows 'Healthy' but your website is actually down — users see errors. How is this possible?

The /health endpoint is lying. It returns HTTP 200 without actually checking the dependencies the real application needs. Likely: the /health handler is a simple static 200 response that does not verify the database connection, Redis availability, or downstream API status. Fix: make /health a REAL readiness check that tests the actual critical dependencies — connect to the DB, ping Redis, check the payment API — and return 200 ONLY if all of them pass. Also: Route 53 health checks run from specific AWS edge locations. If your security group or WAF blocks those IPs, the health check fails even when the app is fine. Route 53 publishes its health-checker IP ranges — whitelist them.

Q7: Scenario: Your website serves users from India, the US, and Germany. You need to serve different language content to each country. Which routing policy and how?

Use Geolocation Routing. Create three records: Record 1 → India origin, geolocation = India (or Asia). Record 2 → US origin, geolocation = United States (or North America). Record 3 → Germany origin, geolocation = Germany (or Europe). Each record points to the server hosting that country's content. IMPORTANT: always create a Default record as a fallback — without it, users from countries you did not list (e.g., Brazil) get no answer and see an error. Geolocation is about the user's LOCATION, not latency — a user in India always goes to the India record even if the US server is somehow faster.

Q8: Scenario: After creating a Route 53 hosted zone, you notice it has 4 NS records. You delete the hosted zone and recreate it. The NS records are different now. Why does this matter?

Every time you create a new hosted zone, Route 53 assigns a DIFFERENT set of 4 nameservers. If you already updated your domain registrar (GoDaddy, etc.) with the OLD NS records, and then you deleted and recreated the zone, the registrar still points to the old NS values — which no longer exist. Result: your entire domain becomes unresolvable. All DNS queries fail. Fix: go back to the registrar and update the NS records to the NEW values. Lesson: never delete and recreate a hosted zone casually. If you must, update the registrar NS records immediately and accept up to 48 hours of potential propagation issues.

Q9: Scenario: Your app has a primary server and a disaster recovery server. Normal traffic should go to the primary. Only if the primary is down should traffic go to DR. Which policy?

Use Failover Routing. Create two records: Record 1 → primary ALB, failover type = Primary, with an active health check monitoring the primary's /health endpoint. Record 2 → DR server (or an S3 static website with a 'we're experiencing issues' page), failover type = Secondary. When the primary health check passes, 100% of traffic goes to it. When the health check fails, Route 53

automatically returns the secondary record. When the primary recovers, traffic automatically returns to it. The secondary can be as simple as a static S3 website — cheap and always available — or a full warm-standby in another region.

Q10: Scenario: You have an API at api.company.com behind an ALB. After a deploy, some users get the new version and some get the old version for 5 minutes. Nothing is wrong with the deploy. What is happening?

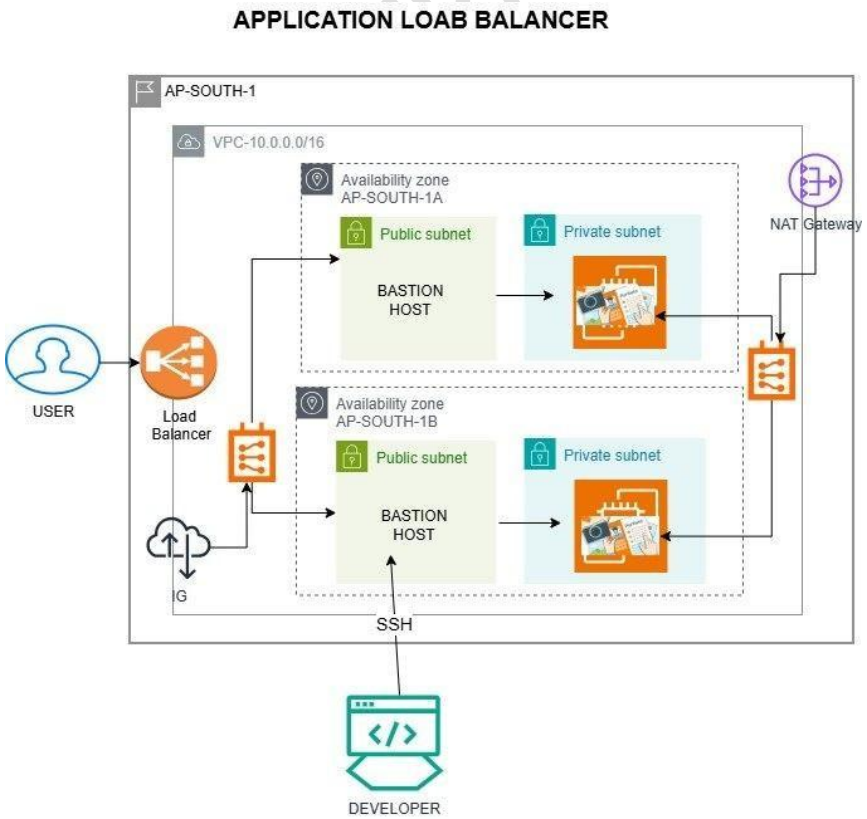
DNS caching. When you updated the A record, Route 53 returned the new IP immediately, but DNS resolvers around the world cached the OLD IP for the remaining TTL duration. Users whose resolvers have not expired their cache still resolve to the old IP. This is not a bug — it is how DNS works. Fix:

1) Keep the TTL low on frequently changing records (60-300s).

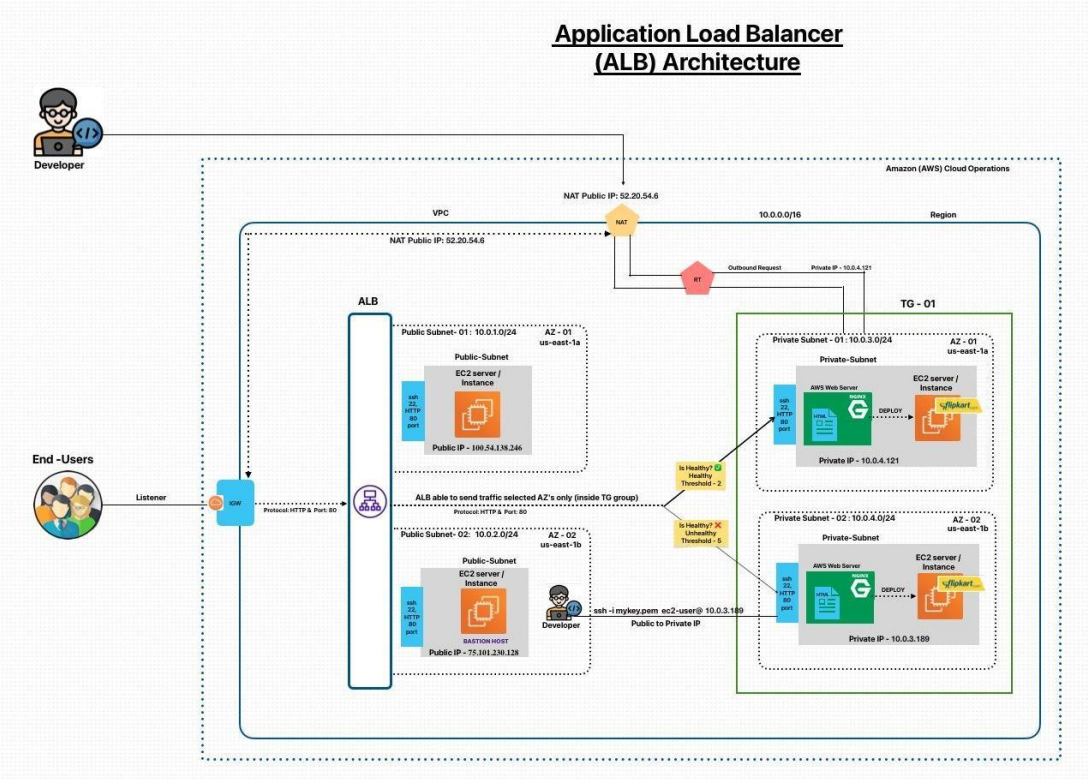
2) For zero-downtime deploys, do NOT change DNS at all — deploy behind the SAME ALB using target groups, blue/green with CodeDeploy, or weighted target groups. DNS changes are the wrong tool for application deployments.

6. AWS Elastic Load Balancer (ELB)

A load balancer distributes incoming traffic across multiple targets (EC2, containers, Lambda) ensuring high availability, fault tolerance, and scalability. It is the single entry point to your application — users talk to the load balancer, never directly to EC2 instances.



ALB Overview: Internet → ALB Listener → Rules → Target Groups → EC2 Instances



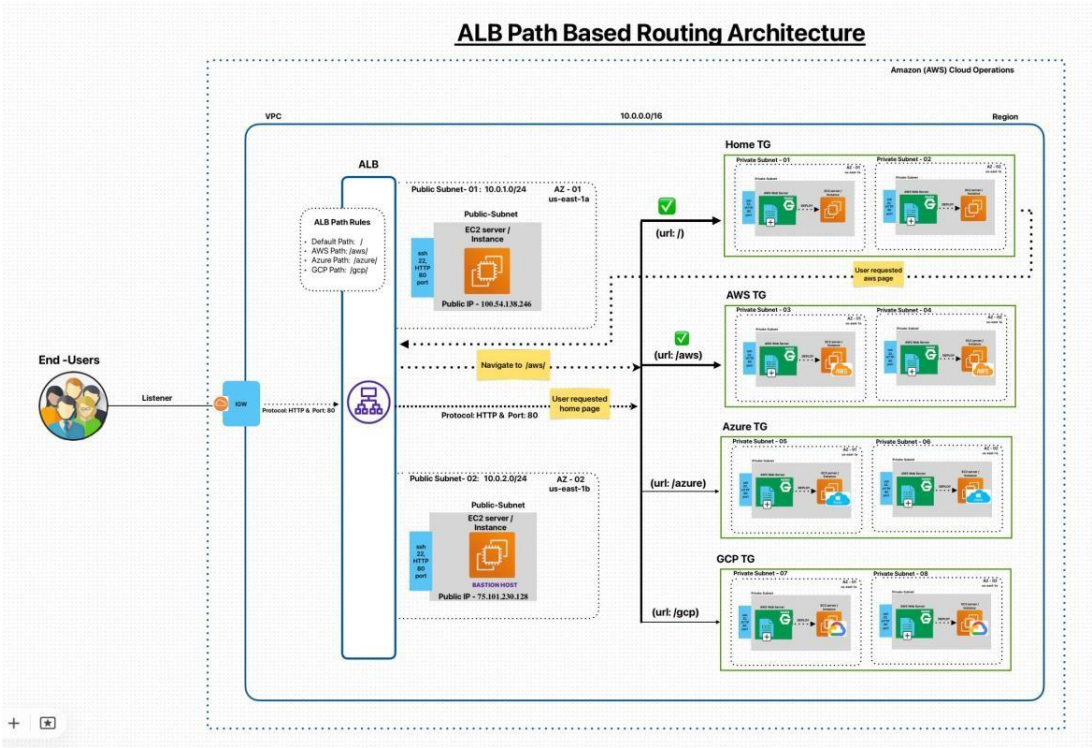
ALB Full Architecture: SSL Termination, Listener Rules, Target Groups, Health Checks

6.1 Types of Load Balancers

Type	Layer	Protocol	Key Feature	Use Case
ALB — Application	Layer 7	HTTP/HTTPS/WebSocket	Path+host+header routing, WAF, SSL	Web apps, microservices, APIs
NLB — Network	Layer 4	TCP/UDP/TLS	Ultra-fast, static IP, millions req/sec	Gaming, IoT, realtime, non-HTTP
GWLB — Gateway	Layer 3	IP packets	Routes through virtual security appliances	3rd-party firewalls, IDS/IPS
CLB — Classic	Layer 4/7	HTTP/TCP	Legacy only — no new features	Old EC2-Classic apps — DO NOT USE

6.2 ALB — Application Load Balancer (Layer 7)

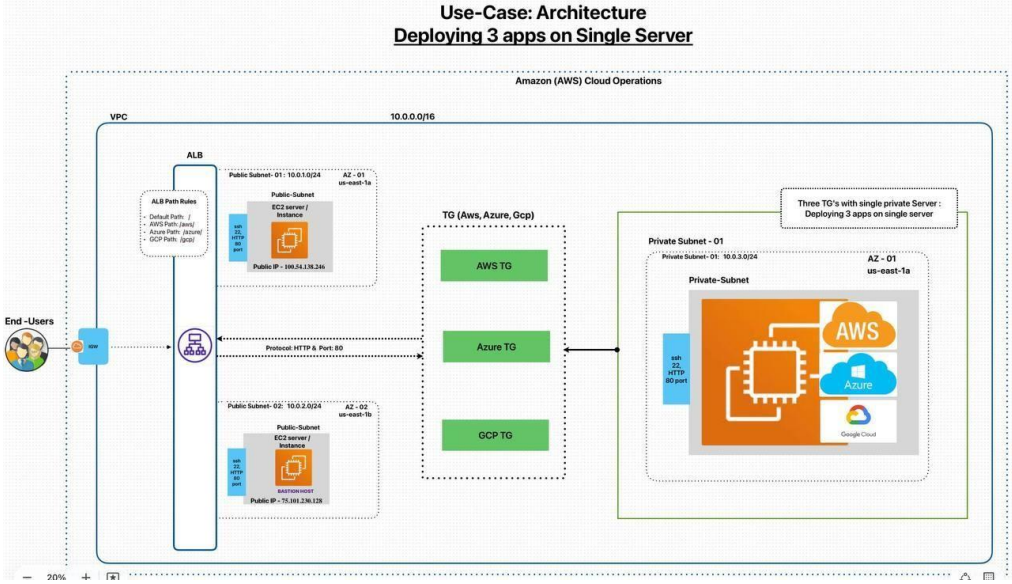
ALB operates at Layer 7 (HTTP/HTTPS). It reads and understands HTTP request content — URL path, hostname, headers, query strings. This enables intelligent content-based routing that Layer 4 cannot do.



ALB Path-Based Routing: /api/ → API service, /images/* → Image service, /* → Frontend*

ALB Core Components

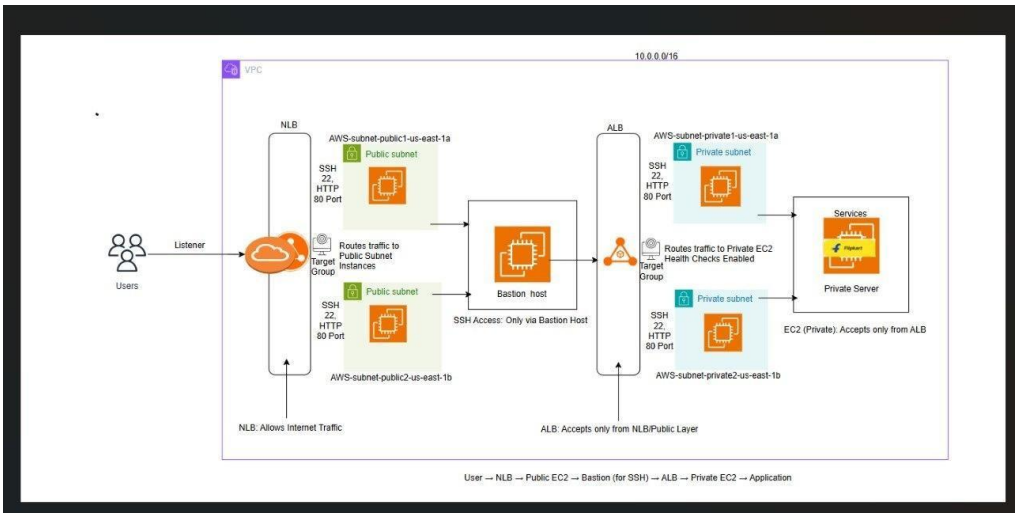
- **Listener:** Checks for connections on a configured port. Port 80 for HTTP, port 443 for HTTPS. One ALB can have multiple listeners.
- **Rules:** Ordered conditions. Each rule: IF (path=/api/* AND host=app.com) THEN (forward to TargetGroup-API). Evaluated in priority order. First match wins.
- **Default Rule:** If no rule matches → default rule applies. Usually forwards to a catch-all target group.
- **Target Group:** Group of registered targets with shared health check config. Target types: instance (EC2), ip (any IP), lambda.
- **Health Check:** ALB probes a configured URL path every N seconds. Unhealthy targets removed from rotation automatically.
- **Stickiness:** Optional: bind user session to a specific target using a cookie. For stateful legacy apps.



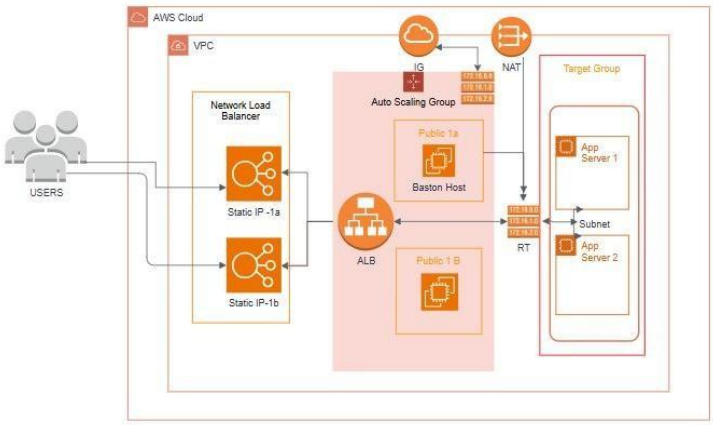
Three apps path-based routing: /app1 /app2 /app3 route to separate target groups on one ALB

ALB Routing Type	Condition	Example
Path-Based	/api/* matches	Route API traffic to microservices, web traffic to frontend
Host-Based	app1.example.com hostname	Multi-tenant: each customer subdomain → different target
HeaderBased	X-Version: v2 header	Route beta users to new version
Query String	?platform=mobile	Different backend for mobile vs desktop
Source IP	10.0.0.0/8 CIDR	Internal employees → different experience

6.3 NLB — Network Load Balancer (Layer 4)



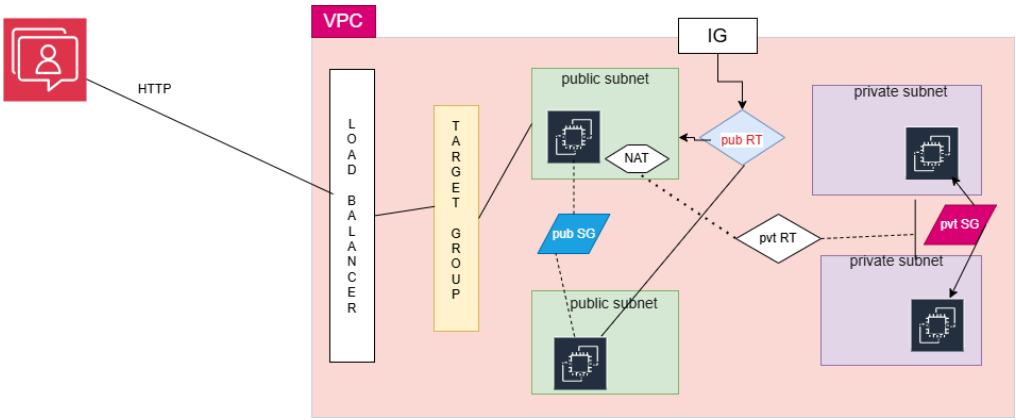
NLB Architecture: Ultra-high performance TCP/UDP load balancing with static IPs



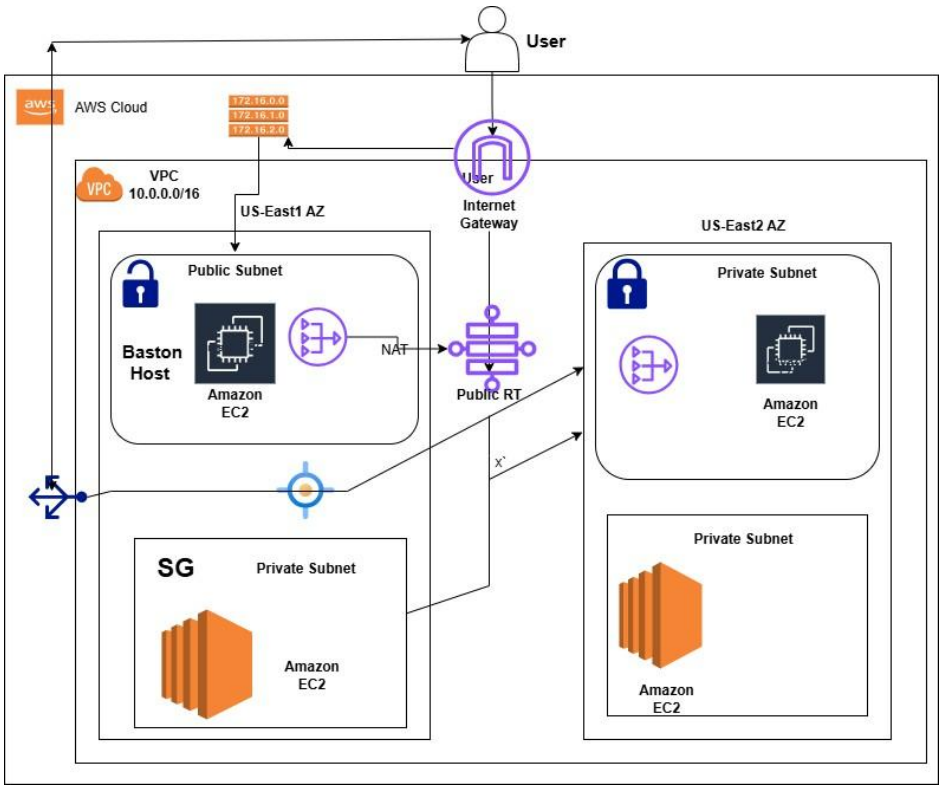
NLB vs ALB Comparison Chart

- Layer 4 only: routes based on IP address + port. Does NOT understand HTTP content.
- Handles millions of requests per second with ultra-low latency (microseconds, not milliseconds).
- Static Elastic IP per AZ: critical for firewall whitelisting (clients whitelist fixed IPs).
- Preserves original CLIENT IP: backend sees real user IP without X-Forwarded-For header tricks.
- Supports TLS termination at Layer 4.
- No path-based or host-based routing — NLB cannot inspect HTTP content.

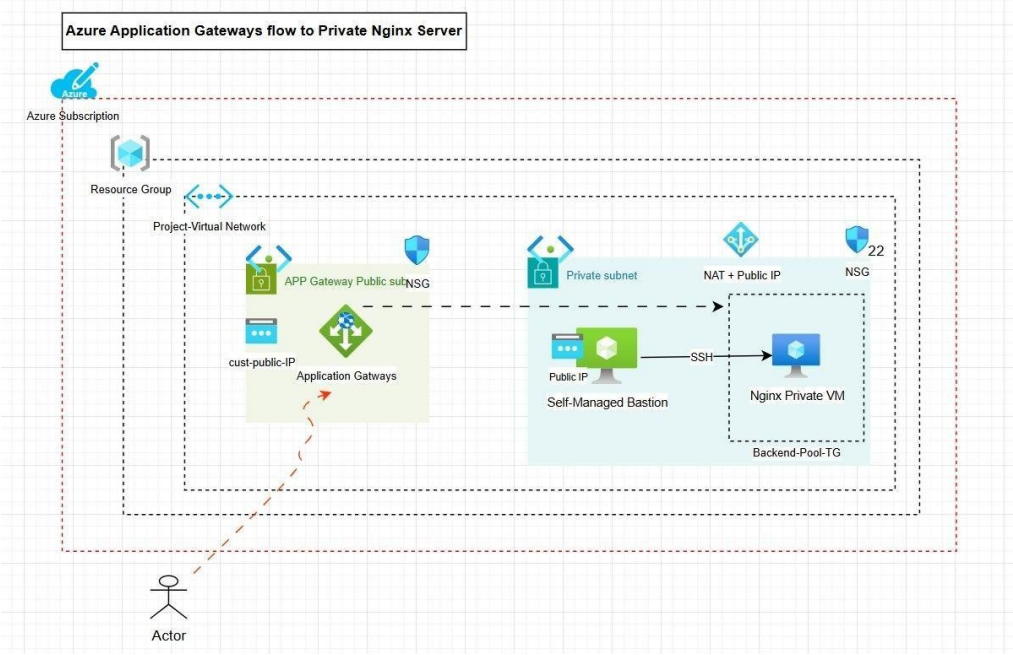
6.4 Load Balancer Architecture



LB Architecture Diagram — internet → LB → target groups → EC2 instances



Complete LB Architecture with multiple target groups



ALB + WAF: WAF inspects HTTP requests → blocks attacks → ALB routes to healthy targets

6.5 Target Groups & Health Checks Deep Dive

Setting	Description	Recommended Value
Health Check Protocol	HTTP, HTTPS, or TCP	HTTP or HTTPS matching your app
Health Check Path	URL ALB calls to verify target	Use /health or /ping endpoint
Interval	Seconds between health checks	10s for critical apps, 30s default
Healthy Threshold	Consecutive successes to mark healthy	2–3 checks
Unhealthy Threshold	Consecutive failures to mark unhealthy	2 checks (fast removal)
Timeout	Seconds to wait for response	5 seconds
Success Codes	HTTP codes considered healthy	200 or 200-299

Scenario-Based Interview Questions — AWS Elastic Load Balancer (ELB)

Q1: Scenario: Users report intermittent 502 Bad Gateway errors. The ALB target group shows all 4 targets as healthy. What is causing this and how do you fix it?

502 means the ALB reached the target but got an invalid or broken response. Most common cause: Keep-alive mismatch. The ALB's idle timeout is 60 seconds (default), but your app's keep-alive timeout (Nginx: keepalive_timeout, Node: server.keepAliveTimeout) is LOWER — say 30 seconds. The app closes a connection the ALB thinks is still open. Fix: set the app's keep-alive timeout to 65+ seconds (higher than the ALB's 60). Other causes: app crashing on specific requests and recovering before the next health check; response headers exceeding ALB's limit; or deregistration delay too short during deploys. Turn on ALB access logs — the target_status_code vs elb_status_code columns prove exactly where it breaks.

Q2: Scenario: A client says 'our firewall can only whitelist IP addresses, not DNS names. Your ALB's IP keeps changing.' But you also need path-based routing (/api vs /web). How do you solve this?

The conflict: ALB gives path-based routing but has changing IPs. NLB gives static IPs (one EIP per AZ) but cannot read URL paths. Solution: put an NLB in front of an ALB. The NLB gets an Elastic IP per AZ — give those IPs to the client for their firewall. The NLB's target group type is 'ALB', forwarding to your ALB, which then does the path-based routing to /api and /web target groups. This is an AWS-supported pattern. Alternative: AWS Global Accelerator — gives you two static anycast IPs pointing at your ALB, plus it optimises global routing via the AWS backbone.

Q3: Scenario: After a rolling deployment, 30% of users see errors for 5 minutes, then everything recovers. All health checks are green during this period. What is wrong?

The new instances pass the health check before the application is actually READY to serve real traffic. The health check sees HTTP 200, but the app is still warming up — JVM loading classes, connection pool filling, cache empty. So the ALB routes full traffic to a cold instance that cannot handle it. Fixes:

1) Make /health a DEEP readiness check that returns 200 only once all dependencies are truly ready (DB connected, cache primed).

2) Enable ALB Slow Start — ramp traffic to a new target gradually over 30-120 seconds instead of sending full load immediately.

3) Increase the health check grace period in the ASG so new instances have time before being judged.

4) Set the deregistration delay high enough that OLD instances finish in-flight requests during the rollover.

Q4: Scenario: You have an ALB with 2 instances in AZ-a and 8 instances in AZ-b. With cross-zone load balancing OFF, AZ-a instances are at 90% CPU while AZ-b instances are at 15% CPU. Explain why and fix it.

With cross-zone OFF, each ALB node only sends traffic to targets in its own AZ. The ALB splits traffic 50/50 between AZ-a and AZ-b. So AZ-a's 50% of traffic is handled by only 2 instances →

25% each → overloaded. AZ-b's 50% is handled by 8 instances → 6.25% each → idle. Fix: enable cross-zone load balancing. Now every ALB node distributes traffic across ALL 10 instances regardless of AZ. Each instance gets ~10% of traffic. Cross-zone is always ON and free for ALB (this is the default). For NLB it is OFF by default and enabling it incurs inter-AZ data charges — which is why AWS leaves the choice to you for NLB.

Q5: Scenario: Your ALB has path-based routing: /api/* goes to the API target group, /images/* goes to the images target group, and everything else goes to the default. A request for /API/users returns 404. Why?

ALB path-based routing rules are CASE-SENSITIVE. /API/users does not match the rule /api/*. Lowercase /api/users would match. Fix: either add a second rule for /API/* pointing to the same target group, or (better) add a Listener Rule action of type 'redirect' that lowercases the path. Or configure your client/app to always use lowercase paths. This case-sensitivity catches teams more often than you would expect, especially with mobile apps where different OS versions may capitalise URLs differently.

Q6: Scenario: During a deployment, in-flight user checkout requests are being cut off and users see payment errors. What ALB setting is misconfigured?

The deregistration delay (connection draining) is too low or set to 0. When a target is being removed during a deploy, the ALB should stop sending NEW requests but let IN-FLIGHT requests finish. The default is 300 seconds. If set to 0, the ALB immediately kills all connections to the target being removed — including that user's checkout POST that was mid-flight. Fix: set deregistration delay to a value that exceeds your longest expected request (e.g., 120 seconds for checkout flows). For very long operations (file uploads, report generation), use 300+ seconds. Never set to 0 in production.

Q7: Scenario: Your app stores user sessions in local server memory. You enabled sticky sessions (session affinity) on the ALB to fix the 'user gets logged out' problem. Your manager says this is wrong. Why?

Sticky sessions tie a user to one specific server via a cookie (AWSALB). Problems:

1) Load becomes uneven — one server may get all the heavy users while others idle.
2) If the sticky server dies, the user is logged out and loses their cart — the exact problem returns.
3) Auto Scaling becomes ineffective — a new instance gets no sticky users, and a server tagged for scale-in cannot be removed without disrupting all its sticky users. The correct fix: externalize sessions to ElastiCache Redis. All servers read/write sessions from Redis. Now any server can handle any user, the ALB distributes load evenly, servers can die/scale freely, and nobody is ever logged out unexpectedly.

Q8: Scenario: Your ALB currently forwards HTTP traffic. You need to add HTTPS support with automatic HTTP-to-HTTPS redirection. Walk through the exact steps.

1) Request a free SSL certificate from ACM (AWS Certificate Manager) for your domain — use DNS validation (one click if the domain is in Route 53).
2) Add an HTTPS:443 listener to your ALB and attach the ACM certificate. Set the default action to forward to your target group.
3) Modify the existing HTTP:80 listener — change its action from 'forward' to 'redirect to HTTPS:443' with status code 301 (permanent).
4) Ensure the Security Group allows inbound port 443.

5) Update the Route 53 record if needed. Now: all HTTP requests are automatically redirected to HTTPS, the ALB terminates TLS (your backend servers handle plain HTTP on port 80/8080), and the certificate auto-renews via ACM.

Q9: Scenario: You have one ALB serving both the customer website (www.shop.com) and the admin panel (admin.shop.com). You want different target groups for each. How?

Use Host-Based Routing (host header condition) on the ALB listener rules. In the HTTPS:443 listener: Rule 1 — Condition: Host header = admin.shop.com → Forward to admin-target-group. Rule 2 — Condition: Host header = www.shop.com → Forward to web-target-group. Default rule → Fixed response 404 (catch-all). Both domains point to the same ALB (via Route 53 Alias records), but the ALB routes to different backend fleets based on the hostname in the request. This is how a single ALB can serve multiple services/microservices — much cheaper than running one ALB per service.

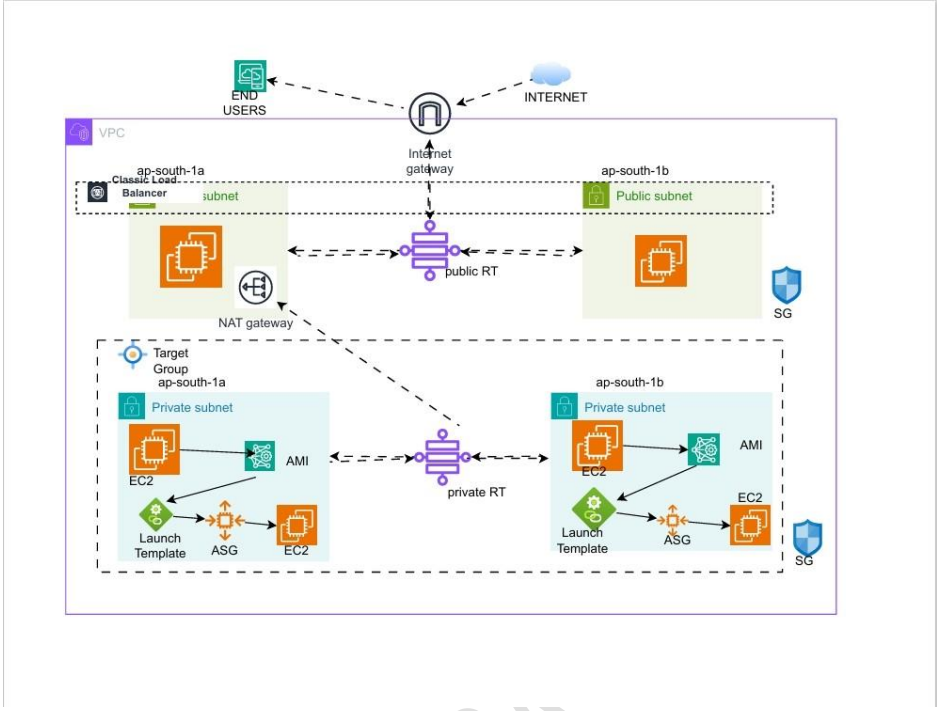
Q10: Scenario: Your NLB health checks show targets as unhealthy, but the application is running fine. curl from within the VPC to the target's port works. What is happening?

NLB health checks come from the NLB nodes' IPs within the target's subnet — these IPs are NOT the NLB's own IPs but rather private IPs within the AZ. The most common cause: the target's Security Group does not allow health check traffic from the NLB. Unlike ALB, NLB passes through the client's source IP (or the NLB node IP for health checks), so you need to allow the VPC CIDR or the subnet CIDR range in the Security Group, not just a specific IP. Check:

1) SG allows the health check port from the VPC CIDR.
2) The health check port matches what the app actually listens on.
3) The health check path (for HTTP checks) returns 200.
4) The instance's OS firewall (iptables) is not blocking the NLB's IPs.

7. AWS Auto Scaling Group (ASG)

An Auto Scaling Group manages a fleet of EC2 instances. It automatically launches instances when demand rises (scale out) and terminates them when demand drops (scale in). Goal: alwaysright capacity — never too few instances (poor performance) and never too many (wasted cost).



ASG: Min/Desired/Max counts — CloudWatch metrics drive all scaling decisions

7.1 ASG Key Concepts

- MINIMUM: The floor. ASG never goes below this count. Ensures baseline availability even at zero traffic.
- DESIRED: Target count. ASG constantly tries to maintain this. Start here, then let scaling policies adjust.
- MAXIMUM: The ceiling. ASG never exceeds this. Prevents runaway costs during unexpected traffic spikes.
- Cooldown Period: After a scaling action, ASG waits N seconds before the next action. Prevents rapid flapping. Default 300 seconds.

7.2 AMI + Launch Template — The Foundation

Theory Explained

The connection between AMI, Launch Template, and ASG: 1. You build and configure an EC2 instance with your application. 2. Create a custom AMI from that instance — captures everything. 3. Create a Launch Template referencing the AMI + instance type + SG + key pair + IAM role + user data. 4. Create ASG using the Launch Template. Result: Every instance ASG launches is IDENTICAL — same app version, same config, same dependencies. No manual setup, no configuration drift.

- **Launch Template:** Defines all instance launch parameters: AMI ID, instance type, key pair, security groups, IAM role, EBS volumes, user data script, network settings. Versioned — can have v1, v2, v3.
- **Launch Configuration:** Legacy predecessor to Launch Templates. Missing many features. Do NOT use for new ASGs.

7.3 Scaling Policies

Policy Type	How It Works	Example Configuration	Best For
Target Tracking (recommended)	AWS auto-adjusts to maintain metric at target	Keep CPUUtilization at 50%	Most workloads — simple and effective
Step Scaling	Scale in configured steps based on alarm thresholds	CPU 70-90% → add 2; CPU 90%+ → add 5	Fine-grained multi-step scaling
Scheduled	Scale at specific times on a schedule	Every Mon-Fri 8AM: desired=10	Predictable daily/weekly traffic patterns
Predictive	ML forecasts future traffic, scales proactively	Enable on ASG, review forecast	Apps with consistent daily/weekly patterns

VEERA

Real-World Example
E-commerce flash sale scenario: • Normal: desired=3 instances. • Flash sale starts 2PM: CPU hits 85%. • CloudWatch alarm fires → Step scaling: CPU 70-90% add 2, CPU 90%+ add 5. • ASG scales to 8 instances within 5 minutes. • Sale ends: CPU drops → scale-in policy removes instances after 10 minutes. • Back to 3 instances. Cost optimized. No manual intervention.

7.4 ASG + ALB Integration Flow

- ASG launches new instance → instance registers itself with the ALB target group automatically.
- ALB starts running health checks on the new instance.
- Only after N consecutive healthy checks → ALB routes production traffic to the new instance.
- ASG terminates instance → ALB connection draining: waits for in-flight requests to complete (default 300s) then terminates.

- ALB health check marks instance unhealthy → ASG automatically terminates and replaces it.

7.5 Instance Refresh — Rolling Deployments

When you update AMI version or launch template, Instance Refresh rolls out the change across all ASG instances with zero downtime.

- Configure 'minimum healthy percentage' — e.g. 80% means at least 80% of instances always healthy during refresh.
- ASG terminates a batch of old instances → launches new instances → waits for health → next batch.
- Checkpoints: Pause refresh at a percentage (e.g. at 50%) to verify new instances before completing.
- If new instances fail health checks: refresh pauses or rolls back automatically.

Scenario-Based Interview Questions — AWS Auto Scaling Group (ASG)

Q1: Scenario: Your ASG keeps launching and terminating instances in a loop every 3 minutes. Traffic is normal. What is causing this flapping?

The instance fails the ELB health check before it finishes booting. The app takes 4 minutes to start but the health check grace period is only 60 seconds. ASG marks it unhealthy at 60s, terminates it, launches another — infinite loop. Fix: increase the health check grace period above the real boot time. Other causes: user data script failing, or scale-out/scale-in alarms overlapping with no dead band.

Q2: Scenario: Every Monday at 9 AM your app is slow for 10 minutes. Target Tracking is configured at 50% CPU. Why and how do you fix it?

Target Tracking is REACTIVE — it detects CPU spike after it happens, then new instances need 2-4 minutes to boot. The 10-minute lag is detection + launch + warm-up. Fix: use Scheduled Scaling — set desired capacity to Monday peak at 8:45 AM. Layer Predictive Scaling on top for ML-based pre-warming. Use Warm Pools for pre-initialized stopped instances that resume in seconds.

Q3: Scenario: During scale-in, users report file uploads failing mid-transfer. How do you prevent this?

The ASG terminates an instance still processing an upload. Fixes:

1) Set ALB deregistration delay to exceed your longest upload (300s).
2) Add a Terminating:Wait lifecycle hook — instance is held while your script confirms uploads are done.

3) Use Instance Scale-In Protection on instances handling long jobs.
4) Architectural fix: use S3 pre-signed URLs so browsers upload directly to S3 — no server in the data path.

Q4: Scenario: You updated the Launch Template with a new AMI. How do you roll it out to 20 running instances with zero downtime?

Use Instance Refresh with MinHealthyPercentage=90% and a warm-up period matching boot time. ASG replaces instances in batches — launching new ones from the updated AMI, waiting for health checks, then terminating old ones. Enable auto-rollback so a failing deploy reverts to the previous Launch Template version automatically.

Q5: Scenario: Your ASG has min=2, desired=2, max=10. It scaled to 10 during a spike but won't scale back down. Why?

Check:
1) Scale-in alarm threshold never met — e.g., scale-out at CPU>50% but scale-in at CPU<10%, and idle instances sit at 15%. Fix: create a dead band (out>70%, in<40%).
2) Instance Scale-In Protection enabled.
3) Suspended ScaleIn process.
4) Conflicting Simple/Step policy overriding Target Tracking's scale-in.
5) Cooldown period too long.

Q6: Scenario: Your Launch Template uses a blank AMI. User data installs everything in 6 minutes. Scale-out is too slow. How do you fix it?

Build a Golden AMI: launch one instance, install everything, verify, create a custom AMI. Update the Launch Template to use it. New instances boot in ~40 seconds with the app pre-installed. For even faster: use Warm Pools with pre-initialized stopped instances that resume in seconds.

Q7: Scenario: You set ASG health check type to EC2 (default). Your app crashed and returns 500 errors but the instance is never replaced. Why?

EC2 health checks only check the hypervisor — the OS is up, so it passes even with a dead application. Fix: change health check type to ELB. Now the ALB's /health endpoint determines health. When the app returns 500, ALB marks it unhealthy, ASG terminates and replaces it. Always use ELB health checks in production.

Q8: Scenario: Your ASG runs in 2 AZs. AZ-a has 5 instances, AZ-b has 1. A scale-in triggers. Which instance is terminated?

ASG terminates from the AZ with the MOST instances (AZ-a) to maintain balance. Within AZ-a, it uses the termination policy (default: OldestLaunchTemplate then OldestInstance then ClosestToNextBillingHour). After termination: AZ-a=4, AZ-b=1. AZ rebalancing is automatic.

Q9: Scenario: Your ASG uses a deprecated Launch Configuration. You need mixed instance types for Spot pricing. What do you do?

Launch Configurations are immutable and deprecated — they cannot do mixed instances or Spot. Migrate to a Launch Template with the same settings, then configure a Mixed Instances Policy on the ASG with multiple types (m5.large, m5a.large, m6g.large) and capacity-optimized allocation for Spot. Set On-Demand base capacity for your minimum, rest on Spot.

Q10: Scenario: You need to run a custom script to upload logs to S3 before an instance is terminated by the ASG. How?

Use a Lifecycle Hook on the Terminating transition. Configure a Terminating:Wait hook with a heartbeat timeout (300s). When ASG terminates: instance enters Terminating:Wait, hook triggers EventBridge/SNS. A Lambda or SSM Run Command runs your script to upload logs. Script calls complete-lifecycle-action when done. Without this hook, the instance is destroyed instantly and logs are lost forever.

1. ACM & CloudFront — SSL/TLS and CDN

VEERA

1.1 AWS Certificate Manager (ACM)

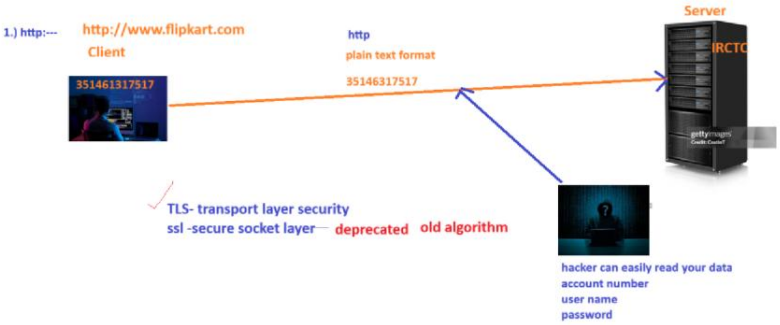
ACM provisions, manages, and auto-renews SSL/TLS certificates for your domains. HTTPS encrypts all traffic between users and your application — required for security, SEO ranking, and browser trust.

AWS Certificate manager (ACM)

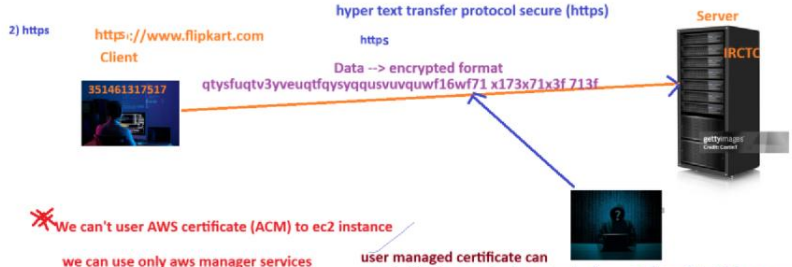
Http ---> Protocol

Https --? TLS/SSL certificate to access the server with https protocol

How this certificate helps to enable https protocol to enable encryption for the data travel in between client and server



hyper text transfer protocol secure (https)



We can't user AWS certificate (ACM) to ec2 instance

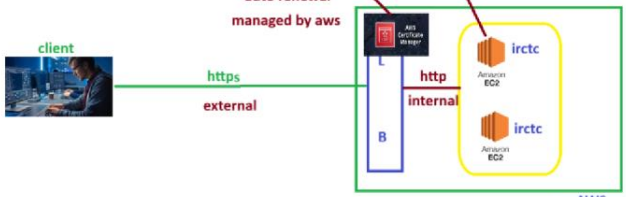
we can use only aws manager services
ex: load balancer
s3 static website
cloudfront
API gateway

user managed certificate can attach (third party sites)

hacker not possible to read data as encryption enabled

hacker can decrypt this encrypted data ? NO
key required to do this

AWS managed certificate
auto renewal
managed by aws



hardware secure model

AWS HSM

private key

pvt key unable to download server

client send request to server

1

https://irctc.com

aws maged TLS

Server

IRCTC

server send certificate to browser

2

client

browser validate the certificate 3

Browser can send session key to server (if certificate validate only)

server decrypt key with private key

encryption process begins

hacker can able to decrypt or not ?

don't have private key

Step	What Happens
1. Client → Hello	Says "I want a secure connection"
2. Server → Certificate	Sends its public certificate
3. Key Exchange	Client + Server create shared key
4. Encrypted Channel	Both sides use same key
5. HTTPS Data Flow	Secure communication happens
6. Close / Reuse Session	Handshake done

certificate ----> veeraonline.in ---> if I create my record veeronline.in will it work or not https ?
only accepts including sub domains

ACM Flow: Domain validation → Certificate issued → Attach to ALB/CloudFront → HTTPS enabled

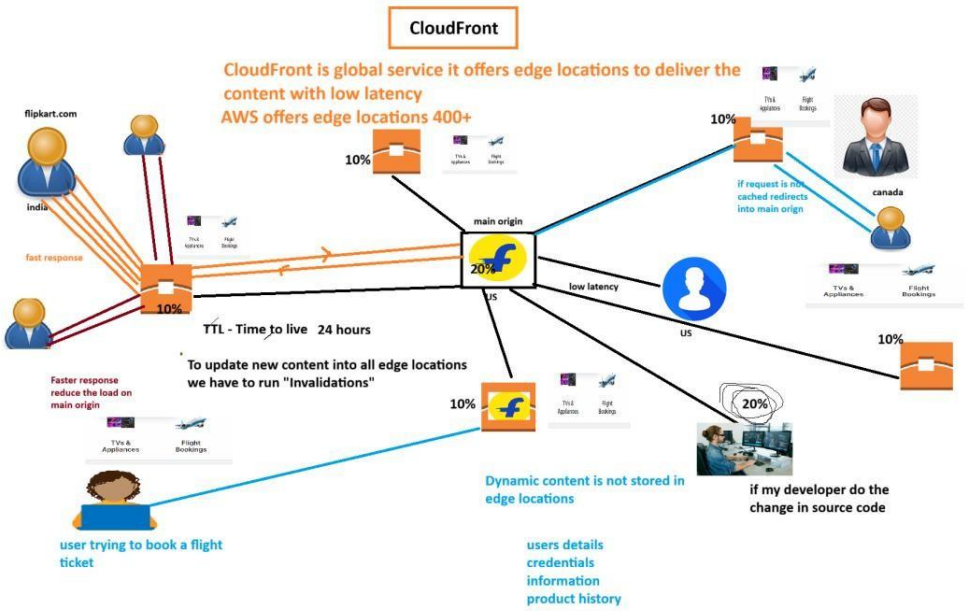
- FREE for use with ALB, CloudFront, API Gateway, Elastic Beanstalk — you never pay for the certificate.
- Auto-renewal: ACM renews 60 days before expiry. You never have to worry about expired certificates again.
- Domain validation — two methods: (1) DNS validation: add a CNAME record to Route 53 (one click automatic). (2) Email validation: respond to email sent to admin@yourdomain.com.
- Wildcard certs: Request *.myapp.com to cover ALL subdomains — api.myapp.com, www.myapp.com, staging.myapp.com.

```
# Steps to enable HTTPS on ALB: 1. Go to ACM → Request Certificate → Enter domain (*.myapp.com) 2. Choose DNS validation → Add CNAME to Route 53 (or autoadd) 3. Certificate status changes to 'Issued' within minutes 4. Go to ALB → Listeners → Add listener → HTTPS:443 → Select ACM cert 5. Add redirect rule: HTTP:80 → HTTPS:443 (301 redirect) 6. Done — all traffic now encrypted
```

8.2 Amazon CloudFront — CDN

CloudFront is AWS's global Content Delivery Network with 400+ edge locations (Points of Presence) worldwide. It caches your content at edges geographically close to users — they get files from a nearby edge instead of your distant origin server.

VEERA



CloudFront Architecture: User → Nearest Edge (CACHE HIT: instant | MISS: fetch origin then cache)

How CloudFront Works — Step by Step

- 1. Create a CloudFront Distribution and specify your origin (S3 bucket, ALB, EC2, or API Gateway).
- 2. CloudFront assigns you a domain: d1234abcd.cloudfront.net. Point your domain to it via Alias record.
- 3. User requests myapp.com → DNS resolves to CloudFront → request routes to nearest edge location.
- 4. CACHE HIT: Edge has the file cached. Serves immediately. Latency <10ms from edge.
- 5. CACHE MISS: Edge doesn't have it → fetches from your origin → caches the response → serves to user.
- 6. All subsequent requests for same content served from edge until TTL expires.

CloudFront Key Settings

- **Distribution:** Main CF config object. Has origins, cache behaviors, price class, SSL cert, WAF.
- **Origin:** Source: S3 bucket, ALB, EC2, API Gateway, or any HTTP server. Can have multiple origins per distribution.
- **Cache Behavior:** URL pattern → cache settings. /images/* cache 7 days. /api/* don't cache. /* cache 1 hour.
- **TTL:** Time-To-Live. Min/Default/Max TTL settings. Set via CF cache policy or origin CacheControl headers.
- **Invalidation:** Force-clear cached objects before TTL. Syntax: /index.html or /images/* (wildcard). First 1000 paths/month free.
- **Price Class:** Choose which edges to use: All (global), 100 (North America + Europe), 200 (+ Asia). Balances cost vs performance.
- **Origin Shield:** Extra caching layer between all edges and your origin. Dramatically reduces origin request volume.

CloudFront Security Controls

- HTTPS only: Set viewer protocol policy to 'Redirect HTTP to HTTPS' — all users get encrypted connection.
- OAC (Origin Access Control): Locks S3 bucket to CloudFront only. Direct S3 URL access returns 403 Forbidden. Users MUST go through CloudFront.
- Signed URLs: Generate a time-limited, cryptographically signed URL for a specific private file. Example: 1-hour download link for a paid ebook.
- Signed Cookies: Same protection but covers multiple files. Better for premium video streaming (many segments).
- Geo Restriction: Block or whitelist users by country from the CloudFront distribution itself.
- AWS WAF: Attach WAF Web ACL to CloudFront → inspect and filter HTTP requests at the edge — before they even reach your ALB or application.
- AWS Shield Standard: Built-in DDoS protection at CloudFront edge. Included free with every distribution.

Key Tip

CloudFront Free Tier (permanent every month): 1 TB data transfer out, 10 million HTTP/HTTPS requests, 2 million CloudFront Function invocations. Small applications may pay nothing for CloudFront.

VEERA

Scenario-Based Interview Questions — ACM & CloudFront — SSL/TLS and CDN

Q1: Scenario: You deployed a fix to index.html but users still see the old broken page after an hour. CloudFront is in front. Fix it now and forever.

Now: create a CloudFront invalidation for /index.html. Forever: two-tier caching — index.html gets Cache-Control: no-cache (always revalidated). Asset files get fingerprinted names (app.a3f9b2.js) with max-age=31536000, immutable. New deploys change filenames — instant, free cache busting without invalidations.

Q2: Scenario: You created an ACM certificate in Mumbai for your ALB. It does not appear in CloudFront's dropdown. Why?

CloudFront certificates MUST be in us-east-1 (N. Virginia). CloudFront is a global service with its control plane in us-east-1. You need TWO certificates: one in ap-south-1 for the ALB, one in us-east-1 for CloudFront. Both are free with ACM and both auto-renew.

Q3: Scenario: Your S3-hosted React SPA works on the homepage but refreshing on /dashboard gives 403. Fix it.

S3 looks for a literal object at key 'dashboard' which does not exist. React Router needs index.html for all paths. Fix in CloudFront: create Custom Error Responses mapping 403->>/index.html with response code 200, and 404->/index.html with response code 200. Alternative: CloudFront Function on viewer-request rewriting extensionless paths to /index.html.

Q4: Scenario: People are hotlinking your images from other websites. Your bandwidth bill doubled. Stop it.

1) AWS WAF Referer header rule — allow only your domains (speed bump, spoofable).
2) Signed URLs or Signed Cookies — require authentication (the real fix).
3) Origin Access Control (OAC) so S3 is only accessible via CloudFront. For paid content, Signed Cookies give subscribers access to the whole library.

Q5: Scenario: CloudFront CacheHitRate is only 35%. Most requests hit the origin. How do you improve it?

65% cache misses means the cache key is too broad. Fixes:
1) Strip unnecessary query strings, headers, and cookies from the cache key — each unique combination creates a separate cached object.
2) Enable Origin Shield — collapses duplicate origin requests across all edges into one.
3) Set appropriate TTLs — very short TTLs force constant revalidation.
4) Enable compression.

Q6: Scenario: Your ACM certificate expired despite auto-renewal being enabled. You used DNS validation. What happened?

Someone deleted the _acme-challenge CNAME validation record from your DNS zone. ACM cannot prove domain ownership without it and renewal fails silently. Fix: re-add the CNAME validation record from the ACM console. Prevention: never delete _acme-challenge records. Set a CloudWatch alarm on DaysToExpiry metric.

Q7: Scenario: You need to restrict paid PDF reports so only subscribers can download them. Links should expire after 2 hours.

Use CloudFront Signed URLs. Backend verifies subscription, generates a signed URL for the specific PDF with 2-hour expiry. Subscriber downloads via the URL. After expiry, URL returns 403. S3 bucket stays private via OAC. Use Signed URL for single files; Signed Cookies for access to many files (e.g., streaming video library).

Q8: Scenario: Your origin server crashes during traffic spikes even with CloudFront in front. What is wrong?

Most requests are cache misses hitting the origin. Fixes:

1) Enable Origin Shield — collapses all edge misses into one origin request.
2) Increase TTL for content tolerating slight staleness.
3) Review Cache Policy — forwarding ALL cookies/headers/query strings creates unique cache keys.
4) For dynamic content, reduce origin load with origin request collapse.

Q9: Scenario: CloudFront shows a certificate mismatch warning: 'certificate is for *.cloudfront.net, not cdn.company.com.' Fix it.

You added the Alternate Domain Name (CNAME) but did not attach a matching ACM certificate. Fix: create an ACM certificate in us-east-1 covering cdn.company.com, validate via DNS, then attach it under Custom SSL Certificate in the CloudFront distribution. Ensure Route 53 Alias points cdn.company.com to the distribution.

Q10: Scenario: 95% of users are in India but your Price Class is 'All Edge Locations.' You are paying for unused edges in South America and Africa.

Change Price Class to PriceClass_200 (includes India, Europe, US, Asia). This excludes expensive low-traffic regions. The 5% from excluded regions are served from the nearest included edge — slightly higher latency but significantly lower cost. Check CloudFront access logs to confirm which edges actually serve traffic.

9. Amazon S3 — Simple Storage Service

S3 is AWS's flagship object storage. Infinitely scalable, 11 nines durable (99.999999999%), and globally accessible. Every application uses S3 for something: images, videos, backups, logs, data lakes, ML training data, or static websites.

9.1 Core S3 Concepts

- **Bucket:** Container for objects. Globally unique name across ALL of AWS (no two accounts can have same name). Created in a specific region. Data stays there unless replicated.
- **Object:** A file + metadata. Any file type. Size: 0 bytes to 5 TB per object. Stored by key.
- **Key:** The unique identifier/path for an object in a bucket. Example: images/users/2024/avatar-user123.jpg
- **Metadata:** System metadata (size, content-type, last-modified) and custom user metadata (key-value pairs).
- All objects are PRIVATE by default — you must explicitly grant access via bucket policy or ACL.

- Strong read-after-write consistency for all operations (PUT, DELETE, LIST) since December 2020.

9.2 S3 Storage Classes — Full Comparison

Storage Class	Retrieve Time	Durability	Min Storage	Cost	Best Use Case
S3 Standard	Milliseconds	11 nines	None	Highest	Frequently accessed: website assets, mobile apps
Standard-IA	Milliseconds	11 nines	30 days	Lower	Infrequent but rapid: backups, DR files
IntelligentTiering	Milliseconds	11 nines	30 days	Auto	Unknown access pattern: data lakes
One Zone-IA	Milliseconds	11 nines (1 AZ)	30 days	20% less than IA	Non-critical secondary copies
Glacier Instant	Milliseconds	11 nines	90 days	Low	Archive needing instant access: medical records
Glacier Flexible	Min–12 hrs	11 nines	90 days	Very Low	Compliance archives, longterm backup
Glacier Deep Archive	12–48 hrs	11 nines	180 days	Lowest	7-10 year regulatory retention

VEERA

9.3 Versioning

Versioning maintains EVERY version of every object in a bucket. Overwrite a file? Both versions stored. Delete a file? A delete marker is added — original still there. Full history preserved.

- Protect against: accidental overwrites, accidental deletes, ransomware (cannot encrypt history).
- Enabling: once enabled, cannot be fully disabled — only suspended (stops versioning new objects).
- Each version stored and billed separately. Use lifecycle rules to expire old versions.
- Delete marker: 'deleting' a versioned object adds a marker. Remove the marker to restore.

> ⚠ **Important**
> Versioning + DELETE does NOT free storage. The 'deleted' file is still there under all previous versions. To actually remove and free storage: delete each individual version including delete markers. Use lifecycle rules to auto-expire non-current versions after N days.

9.4 Lifecycle Policies

Lifecycle policies automate cost optimization — automatically move objects to cheaper classes or expire them based on age. Set once, runs forever.

```
Example lifecycle rule: • After 30 days:  transition to Standard-IA  (55%
cheaper) • After 90 days:  transition to Glacier Flexible Retrieval  (80%
cheaper) • After 365 days: permanently DELETE the object
```

> 💡 **Key Tip**
> S3 lifecycle policies are one of the easiest and highest-impact AWS cost optimizations. For log files and old data, properly configured lifecycle rules can cut S3 costs by 70-90%.

9.5 Replication

Type	Replicates To	Setup Requirement	Use Case
CRR (CrossRegion)	Different AWS region	Versioning on both buckets	DR, compliance, low latency for global users
SRR (Same-Region)	Different bucket, same region	Versioning on both buckets	Dev/test copies, compliance, log aggregation

- Replication is asynchronous — slight delay between write and replica.
- Only replicates NEW objects after replication is enabled. Existing objects: use S3 Batch Replication.

9.6 S3 Security

- **Bucket Policy:** JSON policy on the bucket. Controls which AWS accounts/users/roles/services can access and what actions.
- **Block Public Access:** Account or bucket-level setting. OVERRIDES all policies. Enable for all non-website buckets.
- **Pre-Signed URL:** Time-limited cryptographic URL for temporary access to a private object. No permission changes needed.
- **SSE-S3 Encryption:** AWS-managed keys. Auto-encrypt all objects. No extra cost. Enabled by default for new buckets.

- **SSE-KMS:** Customer-managed KMS keys. Full audit trail in CloudTrail. Small KMS API charge.

9.7 Static Website Hosting

- Host a complete static website (HTML + CSS + JavaScript + images) directly from S3 — no EC2, no web server.
- Setup: Bucket Properties → Static website hosting → Set index document (index.html) + error document.
- Make objects public via bucket policy: Allow s3:GetObject for Principal: *.
- For HTTPS + custom domain: add CloudFront distribution in front → use ACM certificate.

9.8 Event Notifications & Multipart Upload

Event Notifications

- S3 triggers events on: object created (PUT/POST/COPY/multipart), object deleted, restore started/completed.
- Send to: Lambda (process the file), SNS (notify multiple services), SQS (queue for async processing).

Real-World Example

Image pipeline: Upload photo → S3 triggers Lambda → Lambda creates 3 sizes (thumbnail, medium, large) → stores them back to S3 → updates DynamoDB metadata. Fully serverless, no EC2 needed.

Multipart Upload

- Required for objects > 5 GB. Recommended for objects > 100 MB.
- File split into parts → parts uploaded in parallel → S3 assembles them.
- AWS CLI and SDKs handle this automatically. Benefit: 10x faster for large files, resume interrupted uploads.

Scenario-Based Interview Questions — Amazon S3 — Simple Storage Service

Q1: Scenario: A junior dev made your S3 bucket public and customer data was indexed by Google. What do you do now and how do you prevent it forever?

Now:

1) Enable Block Public Access at bucket and account level.

2) Check CloudTrail/access logs for what was downloaded.

3) Request URL removal from Google Search Console. Forever: account-level Block Public Access enforced by SCP, serve files via CloudFront+OAC with pre-signed URLs, enable S3 Access Analyzer, add Config rule with auto-remediation.

Q2: Scenario: Users upload 5 GB videos through your EC2 app. Uploads are slow, timeout, and fill the disk. Redesign it.

Use S3 pre-signed URLs. Browser requests a signed URL from backend, then uploads DIRECTLY to S3 using multipart upload. File never touches EC2. Add S3 Transfer Acceleration for distant users. S3 Event triggers Lambda/MediaConvert for transcoding. Serve via CloudFront with signed cookies. Add lifecycle rule to abort incomplete multipart uploads after 7 days.

Q3: Scenario: Your S3 bill jumped from Rs 20K to Rs 3 lakh with no new data. Find the cause.

Check S3 Storage Lens and Cost Explorer. Usual culprits:

1) Versioning ON with no lifecycle rule — every overwrite doubles storage.
2) Incomplete multipart uploads — invisible orphaned parts.
3) Retrieval fees from Glacier/IA data.
4) Millions of tiny requests from a polling loop.
5) Access logging writing to the SAME bucket causing recursive growth.

Q4: Scenario: You lifecycled 50 million 5 KB log files from Standard to Glacier. The bill went UP. Explain.

Three traps:

1) Glacier bills minimum 128 KB per object — your 5 KB file is billed as 128 KB (25x inflation).
2) Per-object metadata overhead at Standard rates.
3) 50M transition requests cost ~$2,500. Fix: AGGREGATE logs into large compressed files (100 MB+) FIRST, then lifecycle the large files to Glacier. Add ObjectSizeGreaterThan filter to prevent small file transitions.

Q5: Scenario: A developer deleted the production invoices folder. Versioning is enabled. Walk through recovery.

With versioning, deletion only adds DELETE MARKERS — data is still there. Recovery: enable 'Show versions' in console, delete the delete markers to restore objects. For bulk: use CLI to list-object-versions and delete each marker's VersionId. Add MFA Delete for critical buckets. If versioning was NOT enabled, data is gone forever.

Q6: Scenario: Your app needs to serve private profile photos. Each visible only to the owning user, URL expires in 15 minutes.

Use S3 Pre-Signed URLs. Backend verifies identity, generates a signed GET URL for the user's specific photo with 15-minute expiry. Frontend uses it in the img tag. S3 serves the photo directly — backend never downloads/re-serves it. Bucket stays private. Scales to millions of users with zero server bandwidth.

Q7: Scenario: You enabled S3 CRR from Mumbai to Singapore for DR. Existing objects are not replicated. Why?

S3 Replication is NOT retroactive — only objects created AFTER the rule is enabled are replicated. Fix: use S3 Batch Replication for existing objects. Other gotchas: versioning required on BOTH buckets, deletes are NOT replicated by default (a safety feature for DR), replication is asynchronous.

Q8: Scenario: You want ALL objects uploaded to a bucket encrypted with a specific KMS key. Reject unencrypted uploads.

Bucket policy: Deny s3:PutObject where s3:x-amz-server-side-encryption-aws-kms-key-id does not equal your KMS key ARN. Also enable Default Encryption with your KMS key as fallback. Policy enforces (rejects non-compliant uploads); default encryption auto-encrypts if uploader forgets.

Q9: Scenario: 50 TB bucket: 20% accessed daily, 30% monthly, 50% rarely. You pay full Standard price. Optimize.

Use S3 Intelligent-Tiering. It monitors each object and moves automatically: Frequent (30 days) then Infrequent (40% cheaper) then Archive Instant (90 days). No retrieval fees between automatic tiers. The 80% infrequently accessed data saves significantly. Small per-object monitoring fee is far outweighed by savings on 50 TB.

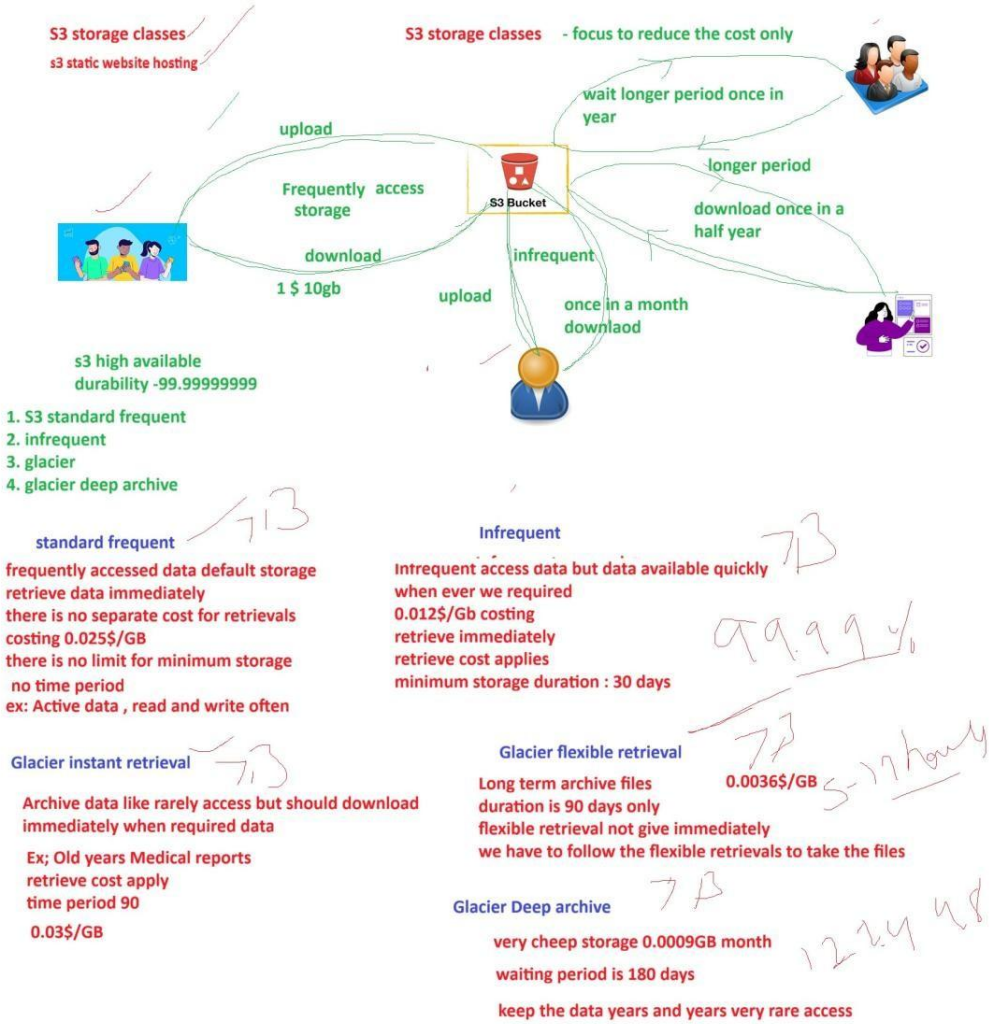
Q10: Scenario: Design S3 so a ransomware attacker with full admin credentials CANNOT destroy your backups.

1) S3 Object Lock in COMPLIANCE mode — nobody including root can delete before retention expires.
2) Versioning + MFA Delete.
3) SEPARATE AWS account for backups with SCP denying Delete operations.
4) Production role can PUT but has no Get/Delete/List — write-only drop box.
5) CRR to second region/account — deletes not replicated by default.
6) GuardDuty + alarms on Delete* API calls.

S3 Deep Dive — Storage Classes & Replication

VEERA

9.9 S3 Storage Classes — Visual Mind Map



S3 Storage Classes Mind Map: Standard → Infrequent → Glacier Instant → Flexible → Deep Archive + pricing

S3 storage classes exist for ONE purpose: cost optimization. Less frequent access = cheaper storage class, but longer minimum duration and higher retrieval cost.

Standard Frequent

- Frequently accessed data — the default storage class.
- Retrieve data immediately with no separate retrieval cost.
- No minimum storage duration — delete anytime.
- Cost: $0.025/GB. Use for: active data, websites, apps, read and write often.

Standard-IA (Infrequent Access)

- Data accessed less frequently but available quickly whenever required.
- Cost: $0.012/GB — retrieval cost applies per GB when you download.
- Minimum storage duration: 30 days — wait 30 days to get pricing benefits.
- Availability: 99.9% (slightly lower than Standard). Use for: backups, DR files.

Glacier Instant Retrieval

- Archive data like rarely accessed medical records that must download immediately when needed.
- Cost: $0.03/GB. Retrieve cost applies. Minimum 90-day storage duration.
- Examples: Old medical reports, legal documents, yearly financial reports.

Glacier Flexible Retrieval

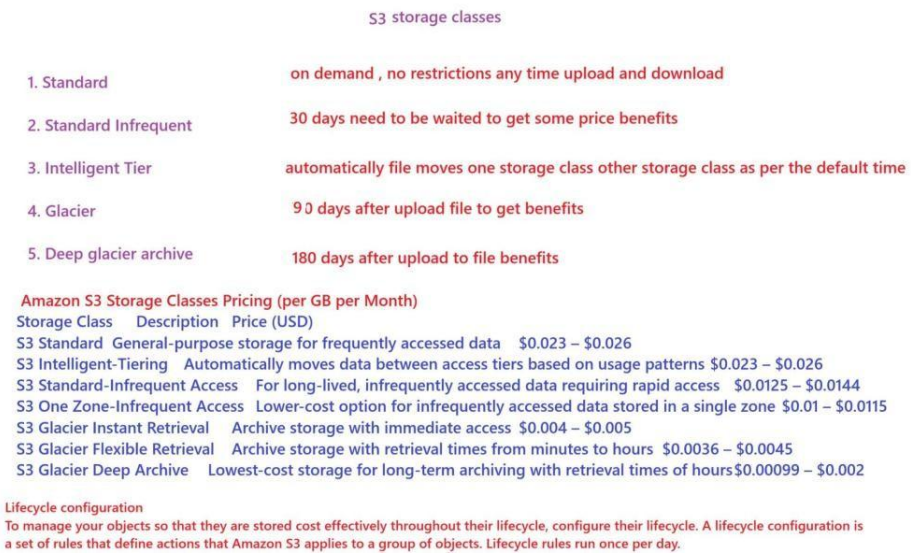
- Long-term archive files. Retrieval NOT immediate — must follow flexible retrieval (5–17 hours).
- Cost: $0.0036/GB. Minimum storage duration: 90 days.
- Use for: compliance archives, long-term backups, data you access once a half year.

Glacier Deep Archive

- Cheapest possible S3 storage — $0.0009/GB per month.
- Waiting period: 180 days minimum storage. Retrieval: 12–48 hours.
- Keep data for years with very rare access: 7–10 year regulatory retention requirements.

VEERA

9.10 S3 Storage Classes Pricing Reference



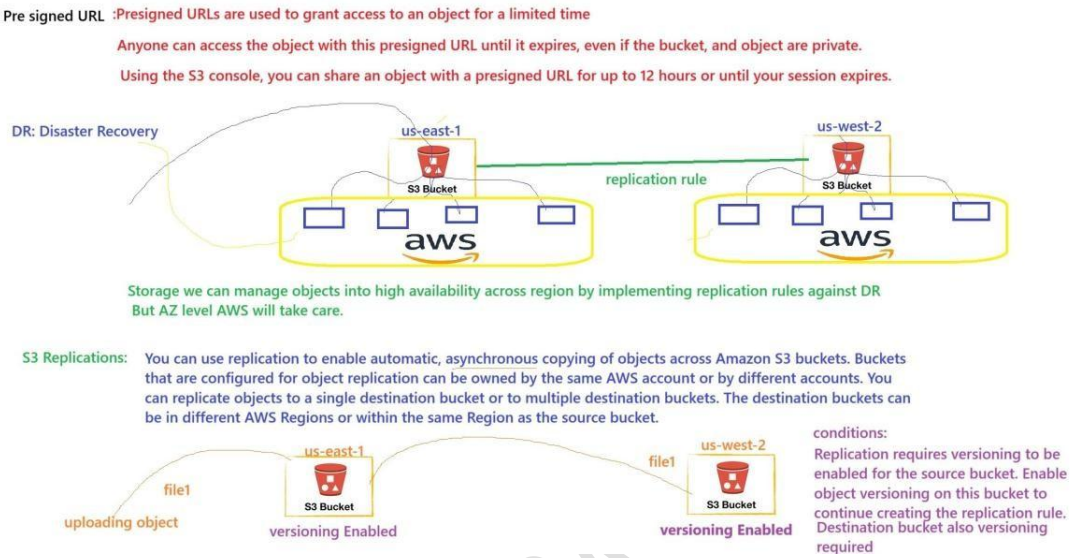
S3 Storage Classes Pricing Table — per GB per month for all classes

Storage Class	Price/GB/Month	Retrieval Cost	Min Days	Access Pattern
S3 Standard	$0.023–$0.026	FREE	None	Any time — on demand
Standard-IA	$0.0125–$0.0144	Per GB	30 days	Once a month
Intelligent-Tiering	$0.023–$0.026	None (auto)	30 days	Unknown — automoves
One Zone-IA	$0.01–$0.0115	Per GB	30 days	Infrequent, single AZ
Glacier Instant	$0.004–$0.005	Per GB	90 days	Rare but instant needed
Glacier Flexible	$0.0036–$0.0045	Per GB	90 days	5–17 hour wait OK
Glacier Deep Archive	$0.00099–$0.002	Per GB	180 days	12+ hours wait OK

Theory Explained

Lifecycle Configuration: Configure lifecycle rules so objects AUTOMATICALLY move between storage classes based on age: • Day 0: Upload → S3 Standard ($0.023/GB) • Day 30: Autotransition → Standard-IA ($0.012/GB) • Day 90: Auto-transition → Glacier Flexible ($0.0036/GB) • Day 365: Auto-expire → DELETE Lifecycle rules run ONCE PER DAY automatically. Set them up once — save money forever.

9.11 S3 Pre-Signed URLs & Cross-Region Replication



S3 Pre-Signed URL + Cross-Region Replication: us-east-1 → us-west-2 with versioning required

Pre-Signed URLs

- Pre-Signed URLs grant TEMPORARY access to a private S3 object without changing bucket permissions.
- Anyone with the URL can access the object until it expires — even without AWS credentials.
- Console limit: up to 12 hours. SDK/CLI: up to 7 days.
- Use case: Share a private invoice PDF with a customer for 1 hour. Share a private video for 24 hours.

```
# Generate pre-signed URL via AWS CLI (1 hour = 3600 seconds) aws s3 presign s3://my-private-bucket/invoice-user123.pdf --expires-in 3600  # Output: full signed URL that anyone can use for 1 hour
```

S3 Cross-Region Replication (CRR)

- Automatically copies objects from source bucket (us-east-1) to destination bucket (us-west-2) using replication rules.
- CONDITION: Versioning must be ENABLED on BOTH source AND destination buckets.
- Objects replicate to destination bucket — same object, different region.
- Use for: Disaster Recovery (if us-east-1 goes down, us-west-2 has full copy), compliance (data must exist in multiple regions), global low-latency access.

- AZ-level HA is handled by AWS automatically inside S3. CRR is for cross-REGION protection.

Scenario-Based Interview Questions — S3 Deep Dive — Storage Classes & Replication

Q1: Scenario: How do you choose an S3 storage class? Give the decision tree.

Ask: how often is it read, and how fast must you get it back? Accessed daily -> Standard. Pattern unknown -> Intelligent-Tiering (auto-moves, no retrieval fees). Monthly, need instantly -> Standard-IA. Same but re-creatable -> One Zone-IA (20% cheaper, single AZ). Few times a year, need milliseconds -> Glacier Instant Retrieval. Archive, minutes-to-hours OK -> Glacier Flexible. Compliance archive, 12+ hours OK -> Glacier Deep Archive (cheapest storage on earth).

Q2: Scenario: What are the hidden costs of cheaper storage classes that catch teams?

1) Minimum duration: IA charges for 30 days even if deleted day 1; Glacier Flexible/Instant for 90 days; Deep Archive for 180 days.
2) Retrieval fees per GB — one analytics job on archived data can cost more than a year of storage.
3) Minimum object size: IA/Glacier bill minimum 128 KB — archiving millions of 2 KB files inflates cost 64x.
4) Transition request costs — moving a million tiny objects costs more in requests than you save. Rule: only lifecycle LARGE, OLD, COLD objects.

Q3: Scenario: Explain S3 Intelligent-Tiering and when it is the right default.

IT monitors each object and moves automatically: Frequent -> Infrequent (30 days no access) -> Archive Instant (90 days) -> optional Archive/Deep Archive. KEY advantage: NO retrieval fees and NO minimum duration penalties between automatic tiers. You pay a small per-object monitoring fee. Right default when access patterns are genuinely unknown. Wrong for: millions of tiny objects (objects <128 KB are never moved, monitoring fee is wasted).

Q4: Scenario: What are the exact requirements and limitations of S3 Replication?

Requirements: versioning on BOTH buckets, IAM role for S3. Limitations:
1) NOT retroactive — existing objects need Batch Replication.
2) Deletes NOT replicated by default (safety feature for DR).
3) Asynchronous — use RTC for 15-minute SLA.
4) SSE-C objects not replicated.
5) KMS objects need explicit key permissions in both regions.
6) Does not chain — A->B->C needs explicit rules.

Q5: Scenario: What is S3 Object Lock and when do you use Governance vs Compliance mode?

Object Lock = WORM (Write Once Read Many). Objects cannot be deleted/overwritten for a retention period. Governance mode: users with s3:BypassGovernanceRetention CAN override — internal accident protection. Compliance mode: NOBODY can delete, not even root, not even AWS — regulatory/legal requirement (SEC 17a-4). Best ransomware defense: attacker with full admin still cannot destroy data. Must be enabled at bucket creation. Requires versioning.

Q6: Scenario: What is the difference between S3 Transfer Acceleration and CloudFront?

CloudFront = CDN for DOWNLOADS. Caches content at edges for many users. Transfer Acceleration = for UPLOADS of large files over long distances. User connects to nearest edge, data travels on AWS private backbone to the bucket. Use when a Mumbai user uploads 10 GB to a Virginia bucket. AWS provides a speed comparison tool. You only pay if it is actually faster than regular upload.

Q7: Scenario: How do you make S3 fast? Explain the performance model.

S3 scales to 3,500 PUT and 5,500 GET per second PER PREFIX automatically. Parallelizing across prefixes gives unlimited throughput. Other levers: multipart upload for anything >100 MB (mandatory >5 GB), byte-range GETs for parallel downloads, CloudFront for repeated reads, Transfer Acceleration for distant uploads. Random hash prefixes are no longer required (as of 2018), but spreading across prefixes still helps scale faster.

Q8: Scenario: Your data lake on S3 has millions of small Parquet files. Athena queries are extremely slow. What is wrong?

The small-file problem. Each file requires a separate S3 GET request and Athena task. Millions of small files create massive overhead. Fix: compact files into 128 MB-1 GB sizes using a Glue ETL job or Athena CTAS (CREATE TABLE AS SELECT). Partition by commonly filtered columns (year/month/day). Target: fewer, larger files per partition. The general rule: in S3, the NUMBER of objects matters as much as the NUMBER of bytes.

Q9: Scenario: You need to serve S3 content to users globally with minimum latency. The content changes frequently. What is your architecture?

S3 as origin + CloudFront with a SHORT TTL (30-60 seconds) and Origin Shield. CloudFront edge locations serve content from cache for the TTL duration. Origin Shield adds an extra layer between edges and S3 — when TTL expires, only Shield requests from S3 (not 600 edges). This reduces S3 costs and load. For real-time content (< 1 second freshness): use Lambda@Edge or CloudFront Functions for dynamic content generation at the edge, or set TTL to 0 with stale-while-revalidate.

Q10: Scenario: Your team wants a shared data lake in S3 for analytics. What design decisions matter most?

1) Partitioning — determines Athena cost forever. Partition by columns you filter on (year=2026/month=07/day=17).
2) File format — Parquet (columnar, compressed). Cuts scan cost by 90%+ vs JSON/CSV.
3) File size — target 128 MB-1 GB. Compact small files regularly.
4) Zones — raw (immutable), curated (validated Parquet), consumption (aggregated).
5) Catalog — Glue Data Catalog as single metadata store.
6) Governance — Lake Formation for column/row-level access.
7) Table format — consider Iceberg for ACID updates and GDPR delete compliance.

10 Amazon RDS — Relational Database Service

RDS is a managed relational database service. AWS handles patching, backups, hardware provisioning, replication setup, failover, and monitoring. You get production-grade database capabilities without managing database servers.

10.1 Supported Engines

- Amazon Aurora (MySQL + PostgreSQL compatible), MySQL, PostgreSQL, MariaDB, Oracle, Microsoft SQL Server.
- Choose engine based on your existing application. For new projects: Aurora is the best choice.

Scenario-Based Interview Questions — Amazon RDS — Relational Database Service

Q1: Scenario: A developer ran DELETE FROM orders without WHERE at 3 PM. It is 3:05 PM. Recover the data.

Use Point-In-Time Recovery. Restore to a NEW instance at 2:59 PM (PITR always creates a new instance). Verify orders table on restored instance. Export orders table and import into production — preserving legitimate transactions in other tables after 3 PM. PITR only works if automated backups are enabled. Check your backup configuration TODAY.

Q2: Scenario: RDS MySQL primary is at 95% CPU. App is 90% SELECTs. What do you do in order of impact?

1) Check slow query log — 90% of the time a missing index causes a full table scan. One index can drop CPU from 95% to 15% free.
2) Add Read Replicas for the read-heavy load.
3) Add ElastiCache Redis for hot repeated queries.
4) Scale up instance class as short-term fix. The mistake: scaling up first and paying forever for a missing index.

Q3: Scenario: Multi-AZ failover completed in 90 seconds per RDS, but app was broken for 15 minutes. Why?

Your APPLICATION failed to reconnect. The endpoint DNS was repointed but app cached the old IP. Causes: JVM DNS caching (infinite default TTL), connection pool holding stale connections, no retry logic. Fixes: set JVM networkaddress.cache.ttl to 5s, enable pool validation, add retry with backoff. Best fix: RDS Proxy handles failover transparently.

Q4: Scenario: Migrate 500 GB on-prem MySQL to AWS RDS with under 10 minutes downtime. How?

AWS DMS with CDC. Full Load copies 500 GB while source stays LIVE (hours, zero downtime). CDC replicates ongoing changes continuously. Monitor until lag near zero. Cutover: stop source

writes, wait lag=0 (seconds), repoint app to RDS, restart. The 500 GB copy happens with the app fully online — only the final drain needs downtime.

Q5: Scenario: RDS free storage dropping 5 GB/day, 20 GB left, Friday evening. What do you do?

Immediate: enable Storage Autoscaling with sensible max. Set CloudWatch alarm at 20% free. Find the cause: query information_schema for biggest tables. Usual culprits: unbounded log/audit table, binary logs retained too long, temp tablespace from stuck query. Trap: InnoDB DELETE does not return space to OS — need OPTIMIZE TABLE or dump-and-restore. RDS storage only scales UP, never down.

Q6: Scenario: RDS PostgreSQL has Publicly Accessible=Yes in a public subnet. Security team flags it. Dev team needs laptop access. Fix it properly.

Move to private subnet. Set Publicly Accessible=No. DB SG allows port 5432 only from App-SG. For laptop access: SSH tunnel through bastion, or SSM Session Manager port forwarding (no bastion needed), or AWS Client VPN. Production databases must NEVER be internet-accessible.

Q7: Scenario: Choosing between RDS MySQL and Aurora MySQL for production needing 15 read replicas and fast failover.

Aurora. RDS supports 5 replicas with seconds of lag; Aurora supports 15 with millisecond lag. Aurora failover under 30 seconds vs 60-120 for RDS. Aurora storage auto-grows to 128 TB with 6-way replication. Aurora reader endpoint auto-load-balances; RDS replicas each have separate endpoints. Trade-off: Aurora costs more per hour.

Q8: Scenario: Your encrypted RDS snapshot cannot be shared cross-account. You used the AWS-managed KMS key. Why?

AWS-managed keys (aws/rds) cannot have their key policy modified for cross-account access. Fix: copy the snapshot within your account, re-encrypting with a Customer Managed Key. Modify the CMK policy to grant the other account access. Share the re-encrypted snapshot. Lesson: use CMKs from the start for shareable databases.

Q9: Scenario: You deleted an RDS instance without a final snapshot. Can you recover?

Only if retained automated backups exist (recently added feature). Check RDS console > Retained Automated Backups. If they exist, restore from them. If not, data is gone forever. Prevention: DeletionProtection=ON always. Take manual snapshots before destructive operations. In CloudFormation, set DeletionPolicy: Snapshot.

Q10: Scenario: You have a primary and a read replica. Primary fails. Does the replica auto-promote?

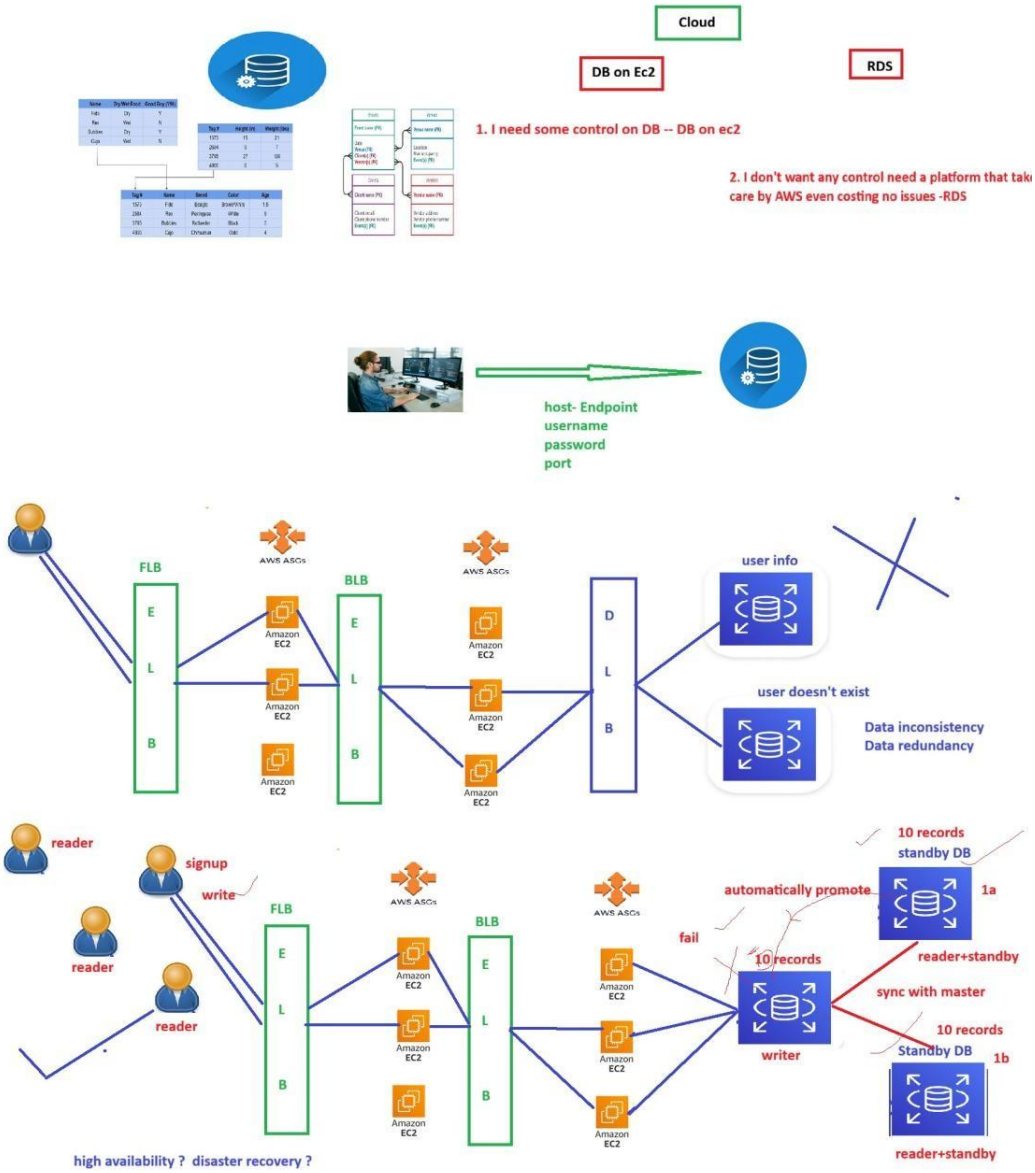
No. Read replica promotion is MANUAL. Multi-AZ failover IS automatic (synchronous standby takes over in 60-120s). Read replicas are asynchronous and separate. To use: promote manually, wait for replication lag to apply, update app connection string. For automatic failover: use Multi-AZ or Aurora with automatic replica failover. Read replicas are for PERFORMANCE, not automatic HA.

10.2. RDS Deep Dive — Architecture with Real Diagrams

This section covers RDS architecture in full detail with diagrams showing how frontend, backend, load balancers, and databases connect in a real production system.

10.3 DB on EC2 vs RDS — Key Decision

VEERA



RDS Architecture: DB on EC2 vs RDS — connect via Endpoint/username/password + Multi-AZ HA

Theory Explained

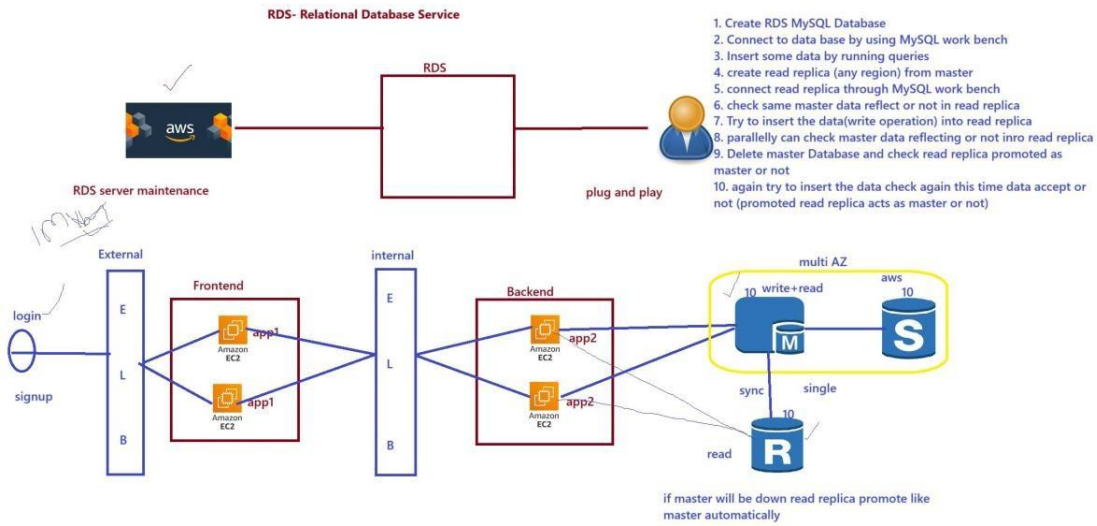
DB on EC2: Full OS access. You manage patching, backups, replication, hardware. Choose when you need a specific DB version, custom plugins, or are migrating a legacy app. RDS Managed: AWS handles patching, backups, failover, hardware. You manage only the data. Use RDS for 99% of production workloads — it saves enormous operational effort. Connection: RDS has no IP address — it has an ENDPOINT (DNS name). Always connect via endpoint.

How to Connect to RDS

- RDS endpoint format: mydb.abc123xyz.ap-south-1.rds.amazonaws.com
- Connection requires: host (endpoint), username (master user), password, port (3306 MySQL / 5432 PostgreSQL).
- Multi-AZ failover: RDS updates DNS to point to standby automatically. App reconnects to the SAME endpoint — no code change.

```
# Connect to RDS MySQL from EC2 (bastion) or local machine mysql -h
mydb.abc123xyz.ap-south-1.rds.amazonaws.com -u admin -p -P 3306
```

10.4 Multi-AZ: High Availability & Disaster Recovery

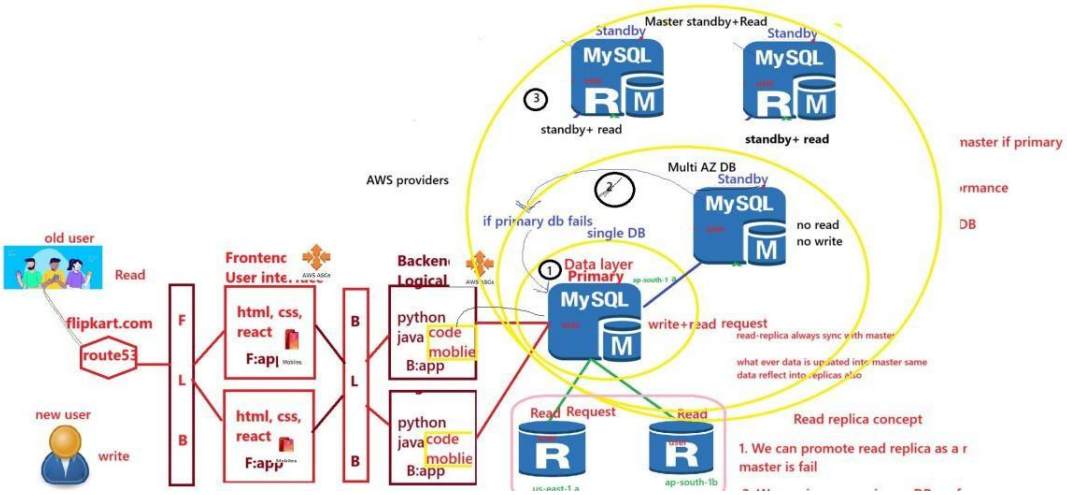


RDS Full Architecture: ELB → Frontend EC2 → Backend EC2 → Multi-AZ Primary + Standby + Read Replicas

- Primary (Master): Handles ALL write operations (INSERT/UPDATE/DELETE) and reads by default.
- Standby: Synchronous copy of Primary in a DIFFERENT AZ. Not readable. Exists purely for automatic failover.
- Failover: Primary AZ fails → RDS auto-promotes Standby in ~60 seconds. App reconnects to same endpoint.
- Read Replica: Asynchronous copy. App explicitly directs SELECT queries here to offload Primary.
- Promote Read Replica: If Primary fails, manually promote a Read Replica to become the new Primary.

Database Role	Readable	Writable	Failover Behavior
Primary (Master)	Yes	Yes	Source of truth — all writes go here
Multi-AZ Standby	NO — passive only	NO	Auto-promoted in ~60s on primary failure
Read Replica	YES — for SELECT only	NO	Manually promote to master if needed

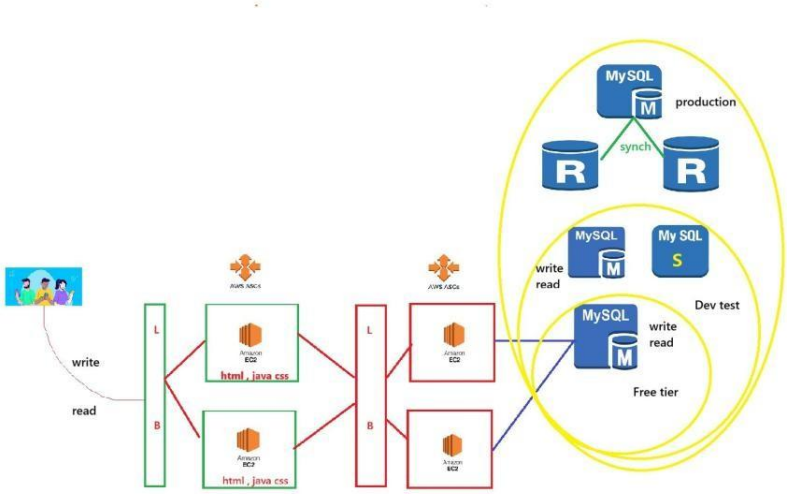
10.5 RDS + Read Replicas — Production Architecture (Flipkart Style)



Production RDS: Primary (ap-south-1) → Multi-AZ Standby → Read Replicas across regions

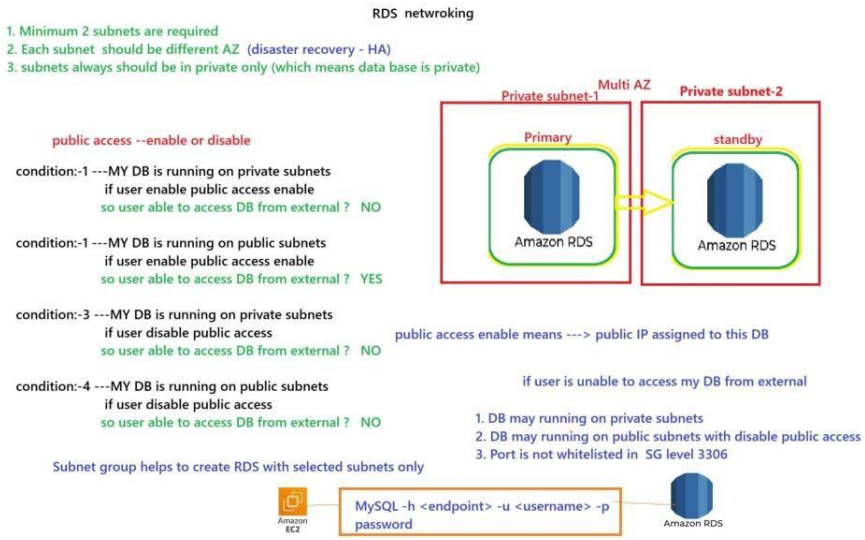
Architecture Flow

Flipkart-style Production DB Architecture: • Route53 → Frontend LB → EC2 (html/css/react) → Backend LB → EC2 (python/java) • Backend writes → RDS Primary (ap-south-1a) • Backend reads → Read Replicas (ap-south-1b + us-east-1) for SELECT queries • Primary (ap-south-1a) syncs SYNCHRONOUSLY to Multi-AZ Standby (ap-south-1b) • Primary also replicates ASYNCHRONOUSLY to Read Replicas • read-replica always syncs with master — whatever data updated in master reflects in replicas



RDS Lab: Production + Dev/Test + Free tier environments with MySQL

10.6 RDS Networking — Subnets and Access Control



RDS Networking: Private subnets Multi-AZ, Public Access conditions, Subnet Groups, connection command

RDS Networking Requirements

- Minimum 2 subnets required — each in a DIFFERENT AZ (for Multi-AZ support and disaster recovery).

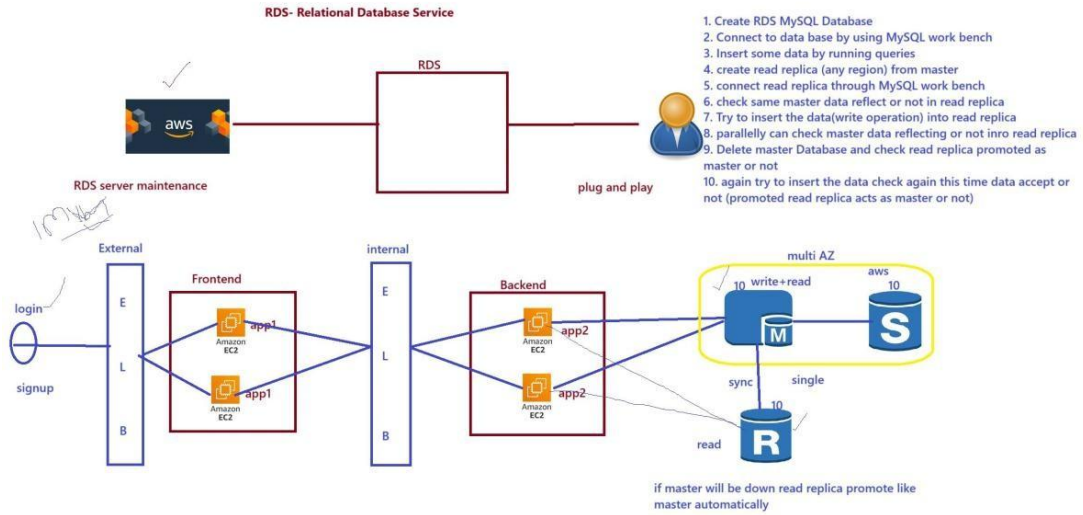
- Subnets MUST be PRIVATE — databases must never be directly internet-accessible in production.
- Subnet Group: A named collection of subnets you specify. RDS uses this to place Primary and Standby in different AZs.

Public Access Setting — All 4 Conditions

Condition	Subnet	Public Access	External Access?	Reason
Condition 1	Private	Enabled	NO	Private subnet = no public IP possible even if enabled
Condition 2	Public	Enabled	YES	Gets public IP + SG allows → external access works
Condition 3	Private	Disabled	NO	No public IP assigned
Condition 4	Public	Disabled	NO	No public IP assigned even in public subnet

⚠ Important
Cannot access RDS from outside? Debug checklist: 1. DB running on PRIVATE subnet? → Cannot be publicly accessed even with public access enabled 2. DB on public subnet but public access = disabled? → No public IP assigned 3. Port 3306 (MySQL) NOT whitelisted in Security Group? → Connection refused Fix for dev/test only: Public subnet + Enable public access + Open port 3306 from your IP in SG

10.7 RDS Hands-On Lab — Step by Step

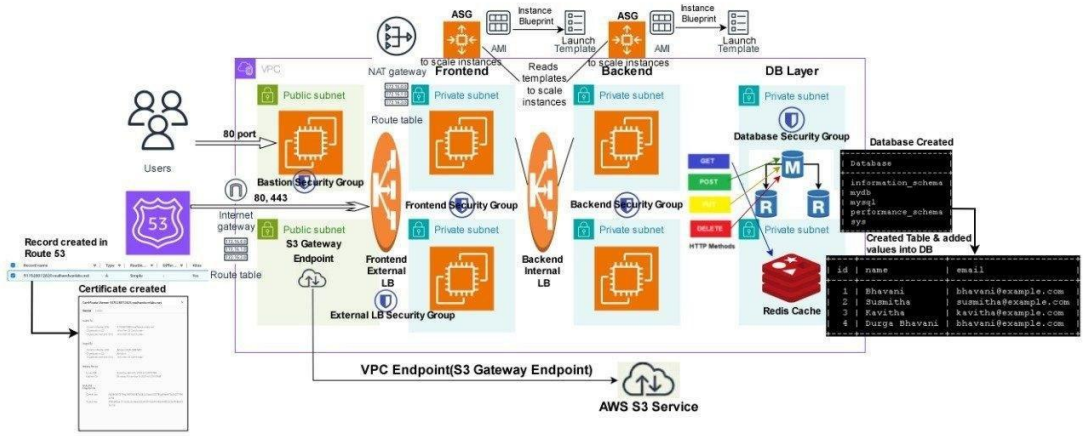


RDS Hands-On: Create MySQL → Connect → Insert data → Create Read Replica → Verify sync → Promote

- Step 1: Create RDS MySQL instance (Multi-AZ for production, Single-AZ for free tier testing).
- Step 2: Connect via MySQL Workbench using endpoint, username (admin), password, port 3306.
- Step 3: Create database and insert test records: INSERT INTO users VALUES(1, 'Veera', 'admin@test.com').
- Step 4: Create Read Replica from master (can be in any region — e.g., us-east-1).
- Step 5: Connect to Read Replica using its SEPARATE endpoint — verify test data is replicated.
- Step 6: Try to INSERT into Read Replica → will FAIL with 'The MySQL server is running with the --read-only option'.
- Step 7: Verify same master data reflects in Read Replica (read-replica always syncs with master).
- Step 8: Delete master → verify Read Replica is promoted as master automatically.
- Step 9: After promotion, INSERT into promoted replica → it now ACCEPTS writes as new master.

```
# Step 5: Connect to Read Replica mysql -h replica.def456.us-east-
1.rds.amazonaws.com -u admin -p  # Step 6: This FAILS on replica (read-only)
INSERT INTO users VALUES (99, 'test', 'test@test.com'); -- ERROR 1290 (HY000):
The MySQL server is running with --read-only  # After promotion (Step 9): same
INSERT now SUCCEEDS
```

10.8 Three-Tier Architecture with RDS



3-Tier Architecture: Web (public) → App (private) → RDS Database (isolated private subnet)

10.9 Multi-AZ vs Read Replicas

Feature	Multi-AZ Deployment	Read Replicas
Purpose	High Availability + Disaster Recovery	Read performance scaling
Replication	Synchronous — zero data loss	Asynchronous — small lag
Standby accessible?	NO — passive hot-standby only	YES — send SELECT queries here
Failover	Automatic, ~60s, DNS-based	Manual promotion to standalone DB
Cross-Region	Same region only	Yes — cross-region replicas supported
Max replicas	1 standby	5 (MySQL/PG/MariaDB), 15 (Aurora)
Use case	Production — never lose data	Reporting, analytics, read-heavy apps

> **Theory Explained**
> Multi-AZ is for HIGH AVAILABILITY, not for performance. The standby accepts ZERO read or write queries from your app. It exists purely for automatic failover. Read Replicas are for

PERFORMANCE. Your app explicitly directs SELECT queries to replica endpoints. Writers still go to primary. Up to 15 replicas for Aurora.

10.10 Backup & Restore

- Automated Backups: Daily full snapshot + continuous transaction log streaming. Retention: 1–35 days (set to 35 for production).
- Point-in-Time Recovery (PITR): Restore to ANY second within the retention window. Data corrupted at 2:34:17 PM? Restore to 2:34:00 PM.
- Manual Snapshots: On-demand. Kept until explicitly deleted (not subject to retention policy). Good for pre-change backups.
- Cross-Region Snapshots: Copy to another region for geographic disaster recovery.

⚠ **Important**
Setting automated backup retention period to 0 DISABLES automated backups entirely and deletes all existing automated backups. NEVER set to 0 for production databases.

VEERA

10.11 Backup Window & Maintenance Window

- **Backup Window:** Daily 30-minute window when automated backups occur. Multi-AZ: backup from standby = zero performance impact. Single-AZ: brief I/O suspension. Set to low-traffic time (e.g., 3:00–3:30 AM).
- **Maintenance Window:** Weekly 30-minute window for AWS patching, minor version upgrades, hardware replacement. Single-AZ: brief downtime. Multi-AZ: automated failover, ~30-60 second reconnection. Set for Sunday 2:00–2:30 AM.

10.12 Publicly Accessible Setting

Setting	What Happens	Security Level	When to Use
Public = YES	RDS gets public IP, reachable from internet	Low — must add strict SG rules	Dev/test only, external DB tool access
Public = NO	No public IP, VPCinternal only	High — recommended for ALL production	Connect via EC2 in same VPC or bastion host

10.13 RDS Proxy

RDS Proxy is a fully managed database proxy between your app and RDS. It pools and reuses database connections — solving the connection explosion problem common in serverless and microservice architectures.

> **Theory Explained**
> The Problem without RDS Proxy: Lambda function receives 500 concurrent requests → tries to open 500 simultaneous new database connections → MySQL max connections = 200 → database crashes with 'Too many connections' error. App returns 500 errors. With RDS Proxy: 500 Lambda requests connect to Proxy → Proxy maintains pool of 20 actual DB connections → multiplexes all 500 through 20 connections. Database sees only 20 connections. Never overwhelmed. App works perfectly.

- Connection pooling and sharing: dramatically reduces connection overhead.
- High availability: deployed across multiple AZs automatically.
- Faster failover: during RDS Multi-AZ failover, Proxy reconnects within seconds instead of minutes.
- Integrates with Secrets Manager: automatic credential rotation without proxy restart.

10.14 Amazon Aurora

Aurora is AWS's cloud-native relational database. MySQL and PostgreSQL compatible. AWS engineered it from scratch for cloud — separating compute and storage layers.

- Storage: 6 copies distributed across 3 AZs (2 per AZ). Self-healing — continuously scans and repairs disk errors.
- Fault tolerance: can lose 2 copies and still WRITE. Can lose 3 copies and still READ. Zero data loss.
- Up to 15 Read Replicas with < 100ms replication lag (vs 5 for standard RDS).
- Storage auto-scales 10 GB to 128 TB in 10 GB increments. Zero manual resizing.

Aurora Feature	What It Does	When to Use
Aurora Serverless v2	Scales compute up/down in fine increments automatically	Variable traffic, dev/test, SaaS, unpredictable load
Aurora Global Database	1 primary + up to 5 read-only regions, <1 second replication	Multi-region apps, global disaster recovery
Aurora MultiMaster	Multiple write nodes across AZs (MySQL compatible)	Zero write downtime requirements

Backtrack	Rewind DB to any point in time in seconds (no restore needed)	Quickly undo accidental deletes/bad migrations

10.15 Self-Managed Database on EC2

You can install a database directly on EC2 instead of using RDS. More control, more responsibility.

Aspect	EC2 Self-Managed	RDS Managed
OS access	Full OS access	No OS access
Patching	You patch OS + DB	AWS handles automatically
Backups	Manual (cron + dump + S3)	Automated with PITR
High Availability	Manual replication setup	Multi-AZ checkbox
Best use case	Specific DB version, custom plugins	99% of production workloads

VEERA

10.15 AWS Secrets Manager — Credential Management

Secrets Manager stores and auto-rotates database passwords, API keys, and other sensitive credentials. Eliminates hardcoded credentials from application code entirely.

- Store DB credentials once → app retrieves at runtime → never hardcoded in source code or config files.
- Automatic rotation: Secrets Manager rotates the RDS/Aurora password every N days. Updates DB and the secret atomically. App always gets the current password transparently.
- Encryption via KMS. All access logged in CloudTrail for audit compliance.

```
# Python — retrieve secret dynamically at runtime import boto3, json client =
boto3.client('secretsmanager', region_name='ap-south-1') secret = json.loads(
client.get_secret_value(SecretId='prod/myapp/db')['SecretString'] ) db_host
= secret['host'] db_password = secret['password']  # always current, autorotated
```

Scenario-Based Interview Questions — RDS Deep Dive — Architecture & Diagrams

Q1: Scenario: Explain why your app connects to an RDS ENDPOINT (DNS name) and never to an IP address.

The endpoint is a DNS CNAME that AWS controls. During a Multi-AZ failover, AWS repoints the CNAME to the standby's IP — your app's connection string never changes. If you hardcoded an IP, failover would break your app because the old IP now points to a dead host. The endpoint indirection is the whole trick: it allows invisible failover, maintenance, and infrastructure changes without touching your application configuration.

Q2: Scenario: Walk through exactly what happens during a Multi-AZ failover. Where do teams usually fail?

1) Primary in AZ-a fails.
2) AWS detects within seconds.
3) The standby in AZ-b (receiving SYNCHRONOUS writes) is promoted — zero data loss.
4) AWS updates the endpoint DNS CNAME to the new primary.
5) Total: 60-120 seconds. Where teams fail: the APP caches the DNS resolution. Java's JVM caches DNS infinitely by default. The app keeps dialing the dead IP even though the CNAME points elsewhere. Fix: set JVM networkaddress.cache.ttl=5, configure connection pool validation (testOnBorrow), add retry logic. Test with a failover game day and measure APP recovery, not RDS recovery.

Q3: Scenario: You have a production RDS and a read replica for dashboards. The dashboard queries are heavy and slow down production reads that go to the same replica. How do you isolate them?

With standard RDS: create a SECOND read replica dedicated to dashboards. Point the dashboard app at this replica's endpoint and production reads at the first replica's endpoint. Each replica has its own endpoint. With Aurora: use Custom Endpoints — create a custom endpoint pointing at specific replicas for analytics/dashboards, keeping the default reader endpoint for production reads. Aurora's custom endpoints are the clean solution for workload isolation.

Q4: Scenario: Your RDS instance encrypted with the default AWS-managed key needs to be migrated to a Customer Managed Key. How?

You cannot change encryption keys on a running RDS instance. Process:
1) Take a snapshot of the instance.
2) Copy the snapshot with the new encryption key — specify your Customer Managed Key during the copy.
3) Restore the copied snapshot to a new RDS instance.
4) Test the new instance thoroughly.
5) Update the application endpoint to point at the new instance.
6) Delete the old instance after confirming. This requires a maintenance window because the restore creates a NEW instance with a new endpoint.

Q5: Scenario: Your RDS storage autoscaling triggered and grew from 100 GB to 500 GB due to a bug. The bug is fixed and actual data is only 80 GB. Can you shrink the storage back?

No. RDS storage can ONLY scale UP, never down. Once grown to 500 GB, you pay for 500 GB forever on this instance. The only way back: dump the data (mysqldump/pg_dump), create a new RDS instance with the correct storage size, restore the data, and switch over. This requires

downtime. Prevention: always set a max storage threshold on autoscaling. Monitor FreeStorageSpace and investigate sudden growth before it autoscales. Fix the root cause (runaway log table, abandoned temp tables) before autoscaling kicks in.

Q6: Scenario: Design the RDS setup for a fintech app: zero data loss, minimal downtime, 7-year audit retention, read-heavy dashboards.

Zero data loss: Multi-AZ (synchronous standby). Use Aurora for 6-way replication across 3 AZs and sub-30-second failover. Minimal downtime: Aurora + RDS Proxy for transparent failover. Dashboards: Aurora read replicas with a Custom Endpoint for analytics isolation. 7-year retention: automated backups max at 35 days — use AWS Backup with lifecycle to cold storage plus Backup Vault Lock in compliance mode (tamper-proof). Export snapshots to S3 Parquet for Athena queries. Security: private subnets, CMK encryption, TLS enforced, IAM auth or Secrets Manager with rotation, Database Activity Streams for SQL-level audit.

Q7: Scenario: Your app uses RDS Proxy. A developer asks: what does RDS Proxy actually do differently from a regular connection?

RDS Proxy sits between your app and the database, providing three key benefits:

1) Connection POOLING — 1000 Lambda invocations share a small pool of real DB connections instead of each opening its own. This prevents connection exhaustion.
2) Faster FAILOVER — the proxy holds client connections and silently reroutes them during a Multi-AZ failover, reducing app-visible downtime by ~66%.
3) IAM authentication — the proxy fetches credentials from Secrets Manager and handles auth, so your app never touches database passwords. The app connects to the proxy endpoint instead of the RDS endpoint. Everything else is transparent.

Q8: Scenario: You need to create a read replica of your production RDS in another region for disaster recovery. What are the important considerations?

1) Cross-region read replicas use ASYNCHRONOUS replication — there WILL be lag. You may lose the most recent transactions in a disaster (RPO is not zero). For zero RPO, you need Aurora Global Database with write forwarding.
2) Network: data replication crosses regions — you pay cross-region data transfer.
3) Encryption: if the source is encrypted, the replica must be encrypted. You need a KMS key in the destination region.
4) Promotion: promoting a cross-region replica to a standalone DB is MANUAL and takes minutes. It is a one-way operation — it becomes independent and can never rejoin as a replica.
5) After promotion: update your app's connection string and DNS to point at the promoted instance.

Q9: Scenario: Your RDS automated backup retention is 7 days. An incident happened 10 days ago and you need to restore data from before the incident. Can you?

No. PITR is only available within the retention window (1-35 days). At 7 days, anything older than 7 days is gone from automated backups. If you took a manual snapshot before the incident, you can restore from that. Manual snapshots persist until you delete them — they are not affected by the retention period. Prevention:

1) Set retention to 35 days for critical databases.
2) Take weekly manual snapshots as an additional safety net.
3) Use AWS Backup with longer retention policies and Vault Lock for compliance.

Q10: Scenario: Your team uses CloudFormation to manage RDS. A developer changed the DBInstanceIdentifier in the template. The change set shows Replace: True. What happens if this is executed?

CloudFormation will DELETE the current RDS instance (with all data) and CREATE a new one with the new identifier. DBInstanceIdentifier is an immutable property — it cannot be changed in place. This is a production-destroying change. The developer probably thinks they are just renaming it. NEVER execute this change set. Protection: set DeletionPolicy: Retain and UpdateReplacePolicy: Retain so the old instance survives even if someone executes it. Apply a Stack Policy denying Update:Replace on the DB logical ID. Put stateful resources in a separate stack.

11. Amazon DynamoDB — NoSQL Database

DynamoDB is a serverless, fully managed NoSQL database. No servers, no clusters, no schema migrations. Single-digit millisecond latency at ANY scale. Amazon.com uses it for their shopping cart — handling 10+ trillion requests per day.

11.1 Why NoSQL? When to Use DynamoDB

Theory Explained

SQL (RDS): Fixed schema, strong relationships (JOINs), ACID transactions. Best for: complex relational data, financial systems, ERP. NoSQL (DynamoDB): Flexible schema, horizontal scale, key-based access. Best for: high-volume simple data access, user profiles, sessions, carts, gaming, IoT. Choose DynamoDB when: you need massive scale, your access patterns are known and simple, you need single-digit ms latency guaranteed, and you want zero database administration.

11.2 DynamoDB Data Model

- Table: Container for items. Tables are region-specific. No fixed schema — each item can have different attributes.
- Item: One record (like a JSON object). Must contain the primary key. All other attributes are optional and flexible.
- Partition Key (Hash Key): REQUIRED. Must be unique per item (if no sort key). DynamoDB hashes this to determine which physical partition stores the data. Choose a high-cardinality key (user ID, order ID) for even distribution.
- Sort Key (Range Key): OPTIONAL. Combined with Partition Key = composite primary key. Enables range queries (get all orders for user X between dates Y and Z).
- **String:** Text. Example: userId = 'user-123456' • **Number:** Integer or decimal. Example: price = 29.99
- **Boolean:** true or false.

- **List:** Ordered collection. Example: tags = ['aws', 'cloud', 'devops']
- **Map:** Nested JSON object. Example: address = {street: '..', city: '..', zip: '..'}

Real-World Example
E-commerce orders table design: • Partition Key: userId (e.g., 'user-1000001') • Sort Key: orderDate (e.g., '2024-01-15T10:30:00') • Attributes: items[], total, status, shippingAddress{} Query: 'Get all orders for user-1000001 in Jan 2024' → DynamoDB jumps directly to that user's partition, scans by date range. Millisecond response even with billions of orders.

11.3 Indexes — Query Pattern Support

- **GSI (Global Secondary Index):** Completely different partition+sort keys from main table. Own separate storage. Supports queries that the main table's key structure cannot. Up to 20 GSIs per table.
- **LSI (Local Secondary Index):** Same partition key, different sort key. MUST be defined at table creation. Cannot add later. Up to 5 LSIs per table.

Key Tip
DynamoDB design rule: Define all your query patterns BEFORE creating the table. For each query pattern you need, either the main table key supports it OR you create a GSI. There are no ad-hoc JOINs or arbitrary queries in DynamoDB.

11.4 Capacity Modes

Mode	How Scaling Works	Pricing	Best For
On-Demand	Instantly scales to any traffic, no planning	Higher per-request cost	New apps, unpredictable spikes, < 1M requests/day
Provisioned + Auto Scaling	You set RCU + WCU. Auto Scaling adjusts within limits.	Lower per-unit cost	Predictable steady traffic, cost optimization

- **RCU:** Read Capacity Unit: one strongly consistent read/sec for item ≤ 4KB.
- **WCU:** Write Capacity Unit: one write/sec for item ≤ 1KB.

11.5 DynamoDB Accelerator (DAX)

DAX is a fully managed, clustered in-memory cache SPECIFICALLY for DynamoDB. Reduces read latency from milliseconds to MICROSECONDS. Zero application code changes required.

- Without DAX: DynamoDB read = ~1–10ms. With DAX: DynamoDB read = ~microseconds (100–1000x faster).
- Write-through: writes go to DAX and DynamoDB simultaneously. Cache always consistent.
- Transparent: change DynamoDB client endpoint to DAX cluster endpoint. Same API, same code.
- Multi-AZ cluster: primary + replicas across AZs for HA.
- NOT for: strongly consistent reads (DAX returns cached value), write-heavy workloads.

11.6 DynamoDB Streams & Transactions

- **Streams:** Ordered log of ALL item-level changes (INSERT, MODIFY, DELETE). Retained 24 hours. Trigger Lambda to react to every DB change in near real-time.
- **Transactions:** ACID transactions across multiple items or tables in a single request. All succeed or all fail. Up to 25 items, 4MB per transaction.

Real-World Example

DynamoDB Streams use case: User updates profile (name + email). DynamoDB Streams captures the change → triggers Lambda → Lambda updates Elasticsearch search index + sends SNS notification + invalidates CloudFront cache. All automatic, triggered by the DB write.

Scenario-Based Interview Questions — Amazon DynamoDB — NoSQL Database

Q1: Scenario: Your table uses Date as partition key. Throttles daily at peak despite 30% average capacity. Explain.

Average is a lie — capacity is per PARTITION. Date as PK means 100% of today's writes hit ONE partition at 100% while yesterday's partitions sit idle. Fix: use high-cardinality key (UserID, OrderID). If you must use date, write-shard: DATE#2024-07-17#<0-9> to spread across 10 partitions. Use Contributor Insights to identify hot keys.

Q2: Scenario: Monthly revenue report on DynamoDB takes 40 minutes and costs a fortune. What is wrong?

Wrong tool. DynamoDB is for operational lookups, not analytics. That report is a full Scan — reads and bills every item. Fixes:

1) Export to S3 (zero-RCU native export) and query with Athena.

2) Stream to S3/Redshift via Kinesis Firehose.

3) Pre-aggregate with Streams+Lambda. Rule: OLTP in DynamoDB, OLAP in Athena/Redshift.

Q3: Scenario: Design a table for e-commerce: get order by ID, all orders for a user by date, all PENDING orders.

Base table: PK=ORDER#orderId, SK=ORDER#orderId (GetItem by ID). GSI1: PK=USER#userId, SK=createdAt (Query user orders by date). GSI2: PK=STATUS#status, SK=createdAt (Query PENDING orders). Use sparse index for GSI2 — only write the attribute while pending, remove when shipped. The GSI only contains open orders.

Q4: Scenario: Lambda writes to DynamoDB. Under load you get ProvisionedThroughputExceededException despite auto-scaling. Why?

Auto-scaling is REACTIVE (1-2 minute lag). Sudden spikes exceed capacity before it reacts. Fixes:

1) Switch to On-Demand mode (instant scaling, higher per-request cost).

2) Raise auto-scaling minimum to absorb initial burst.

3) Use DAX cache for reads.

4) SDK automatically retries with exponential backoff for throttled requests.

Q5: Scenario: Developer uses Scan to find users in a city. 10 million items, 30 seconds, expensive. Redesign.

Scan reads ALL items then filters — billed for everything read. Fix: GSI with PK=CITY#cityName, SK=userId. 'All users in Mumbai' becomes a Query — reads only Mumbai users, fast and cheap. If you need Scan, your data model or indexes are wrong.

Q6: Scenario: User sessions with 24-hour expiry pile up. Storage growing. How do you auto-cleanup?

Enable DynamoDB TTL on an expiresAt attribute set to Unix epoch timestamp. DynamoDB auto-deletes expired items within 48 hours at ZERO cost and zero capacity. Do NOT use scheduled Lambda scans — TTL is purpose-built for this and is free.

Q7: Scenario: You need the table available in Mumbai AND Frankfurt for low-latency global reads AND writes.

DynamoDB Global Tables. Add a replica in Frankfurt. Active-active multi-region: users read/write locally, changes replicate near real-time. Last-writer-wins conflict resolution. Requires On-Demand or Provisioned with auto-scaling, and Streams enabled.

Q8: Scenario: App writes to DynamoDB then immediately reads it back. Occasionally gets OLD data. Why?

Default reads are Eventually Consistent — may hit a replica without the latest write. Fix: use ConsistentRead: true in GetItem/Query. Reads from the primary partition leader, guaranteeing latest data. Costs 2x RCUs. GSIs are always eventually consistent — cannot use strong consistency on GSIs.

Q9: Scenario: Critical order accidentally updated with wrong data. No Point-In-Time Recovery enabled. How to prevent this?

Enable PITR on every production table TODAY — it is off by default. With PITR: restore to any second within 35 days (creates new table). Also enable Streams for audit trail. AWS Backup also

supports DynamoDB. Prevention: PITR + Streams + Backup. Without any of these, accidental updates are unrecoverable.

Q10: Scenario: Flash sale: reads spike from 1K to 50K RCU in seconds. Auto-scaling cannot keep up. Best mode?

Switch to On-Demand. It scales instantly to any level — adapts in seconds without provisioning or warm-up. Costs ~5-7x more per request but avoids throttling and lost orders. After the sale, if traffic becomes predictable, switch back to Provisioned + auto-scaling (once per 24 hours).

12. Amazon API Gateway

API Gateway is a fully managed service to create, publish, secure, and monitor APIs at any scale. It is the front door for serverless architectures — connecting HTTP requests from browsers/apps to Lambda, EC2, or any HTTP backend.

12.1 API Types

API Type	Protocol	Latency	Cost	Best For
REST API	HTTP/HTTPS	Standard	Higher	Full-featured APIs: caching, transforms, validation, API keys
HTTP API	HTTP/HTTPS	30% faster	70% cheaper	Simple Lambda/HTTP proxy, JWT auth, low cost
WebSocket API	WebSocket	Real-time	Per message	Chat apps, live dashboards, real-time games, stock tickers

VERA

12.2 Core Concepts

- **Resource:** A URL path. Example: /users, /users/{userId}, /users/{userId}/orders.
- **Method:** HTTP verb on a resource. GET /users returns all users. POST /users creates one.
- **Integration:** The backend: Lambda function, HTTP endpoint, AWS service (S3/DynamoDB directly), or Mock.
- **Stage:** A deployment environment: dev, staging, prod. Each has its own URL and can have different settings.
- **Usage Plan + API Key:** Rate limit and quota specific API clients. Prevent one client from consuming all capacity.

12.3 Endpoint Types

Type	Where Deployed	Access	Use Case
Edge-Optimized	Global via CloudFront edges	From internet	Global API — users worldwide, lowest global latency
Regional	Single AWS region	From internet	Same-region clients, or behind your own CDN
Private	VPC only via Interface Endpoint	VPC internal only	Internal microservices, private APIs

12.4 Security Methods

- IAM Permissions: Caller signs request with AWS credentials (SigV4). For AWS service-toservice calls.
- Lambda Authorizer: Custom auth in a Lambda function. Validate JWT, OAuth tokens, SAML. Returns IAM policy.
- Cognito User Pools: API GW validates Cognito-issued JWT automatically. For user-facing apps.
- API Keys + Usage Plans: Simple key-based client identification. Good for rate limiting, NOT for auth.
- Resource Policies: Restrict API access by IP CIDR, VPC endpoint, or AWS account.
- WAF Integration: Protect against OWASP Top 10, bots, and DDoS at API level.

VEERA

Scenario-Based Interview Questions — Amazon API Gateway

Q1: Scenario: API Gateway + Lambda returns 502. Lambda logs show successful execution. What is wrong?

The response SHAPE is invalid. Lambda Proxy requires: { statusCode: 200, headers: {...}, body: '<STRING>' }. Common causes: body is an object not JSON.stringify(), missing statusCode, missing await so function returns undefined. Enable execution logging and check Endpoint response body.

Q2: Scenario: A partner hammers your API at 10K req/sec. Other customers suffer. Isolate them without blocking.

API Keys + Usage Plans. Issue each partner an API key with a Usage Plan: rate=100/sec, burst=200, quota=2M/month. The noisy partner gets 429 when exceeding THEIR limit. Others unaffected. Add WAF rate-based rules as second layer. Return Retry-After headers.

Q3: Scenario: React app gets CORS errors calling your API Gateway. Fix it.

Two fixes needed:

1) Enable CORS on API Gateway (creates OPTIONS method with CORS headers).

2) Your Lambda must ALSO return Access-Control-Allow-Origin in its response — enabling CORS in console only handles OPTIONS. For cookies: cannot use * origin, echo specific origin with Allow-Credentials:true. Configure Gateway Responses for 4XX/5XX with CORS headers too.

Q4: Scenario: You have REST API and HTTP API options. Lambda backend, no caching/WAF/usage plans needed. Which?

HTTP API. ~70% cheaper, lower latency. Supports Lambda Proxy and JWT authorizers. You lose caching, usage plans, WAF, request transformation. For a straightforward Lambda backend, HTTP API is correct.

Q5: Scenario: After deploy, API returns stale data for 5 minutes. API Gateway caching is enabled with 300s TTL.

Cache serves pre-deploy responses until TTL expires. Fix: invalidate cache via Cache-Control:max-age=0 header. Design fix: never cache user-specific or rapidly changing data. Ensure the Authorization header is part of the cache key — otherwise user A sees user B's cached data.

Q6: Scenario: API receives 15 MB file upload. Returns 413 Payload Too Large. How do you handle it?

API Gateway has a HARD 10 MB payload limit. Cannot be increased. Solution: bypass API Gateway for uploads. Client requests S3 pre-signed PUT URL from your API, then uploads directly to S3. S3 Event triggers Lambda to process. API only handles metadata, never the file. Works for any file size.

Q7: Scenario: Cognito Authorizer adds significant latency under load. How do you reduce it?

Enable Authorizer Result Caching with TTL matching JWT expiry (e.g., 3600s). Authorizer runs once per unique token — subsequent requests skip validation. Or switch to JWT Authorizer on HTTP API (validates natively, no Lambda invocation).

Q8: Scenario: You need dev, staging, prod with different backends but one API definition. How?

Use Stages and Stage Variables. Deploy to three stages: dev, staging, prod. Define stage variable lambdaAlias=dev/staging/prod. Reference ${stageVariables.lambdaAlias} in Lambda integration. Each stage gets its own URL, throttling, caching, logging.

Q9: Scenario: Report generation endpoint takes 45 seconds. API Gateway times out. What is the limit and fix?

Hard timeout of 29 seconds, cannot be increased. Fix: async pattern. POST /reports starts the job (writes to SQS/Step Functions), returns {jobId, status:processing} in <1 second. Backend worker processes and stores result in S3. Client polls GET /reports/{id} until complete, then downloads via pre-signed URL.

Q10: Scenario: Lambda authorizer runs on every request (50K/min), adds 100ms latency. Optimize it.

Enable Authorizer Result Caching (TTL matching token expiry). 50K requests from 100 users means authorizer runs 100 times instead of 50K. Or switch to native JWT Authorizer on HTTP API — validates tokens without Lambda, zero invocation cost.

13. AWS Lambda — Serverless Compute

Lambda is the serverless compute service. Write a function, define its trigger, and Lambda handles everything: servers, OS, scaling, patching, availability. Pay only per millisecond of execution. Zero idle cost. Zero server management.

13.1 Serverless vs Traditional

Aspect	Traditional EC2	AWS Lambda
Server management	Provision, patch, manage EC2	AWS manages everything
Scaling	Manual or ASG config needed	Instant auto-scale, 0 to 10,000 concurrent
Availability	Configure Multi-AZ	Built-in, multi-AZ automatically
Cost model	Pay 24/7 whether idle or busy	Pay only per ms of actual execution
Max runtime	Unlimited	15 minutes per invocation
Cold start	Instant (already running)	100ms–2s first invocation

VEERA

13.2 Lambda Triggers

Trigger	Type	Example Use Case
API Gateway	Synchronous	HTTP GET /users → Lambda returns user list
ALB	Synchronous	HTTP request to ALB target → Lambda processes
S3	Asynchronous	File uploaded → Lambda resizes image, extracts metadata
DynamoDB Streams	Polling	Record changed → Lambda reacts in real-time
SQS Queue	Polling	Message arrives → Lambda batch-processes messages

SNS Topic	Asynchronous	Alert published → Lambda sends Slack notification
CloudWatch Events	Asynchronous	Every 5 minutes → Lambda runs scheduled cleanup job
EventBridge	Asynchronous	order.placed event → Lambda processes the order
Kinesis	Polling	Streaming data → Lambda performs real-time analytics

13.3 Lambda Execution Model

- **Cold Start:** First invocation: Lambda provisions environment, loads code, runs init. 100ms–2s. Users see slight delay.
- **Warm Start:** Container reused from previous invocation. Near-instant. Lambda keeps containers warm ~15 minutes.
- **Concurrency:** Lambda scales by running multiple simultaneous copies. Default limit 1000 concurrent executions.
- **Memory:** 128 MB to 10,240 MB. CPU proportional to memory. 1,769 MB = 1 full vCPU.
- **Timeout:** Default 3 seconds. Maximum 15 minutes. Set appropriate timeout — don't leave default for long operations.
- **Provisioned Concurrency:** Pre-warm N Lambda instances. Always warm, zero cold starts. Costs extra. Use for latency-critical paths.

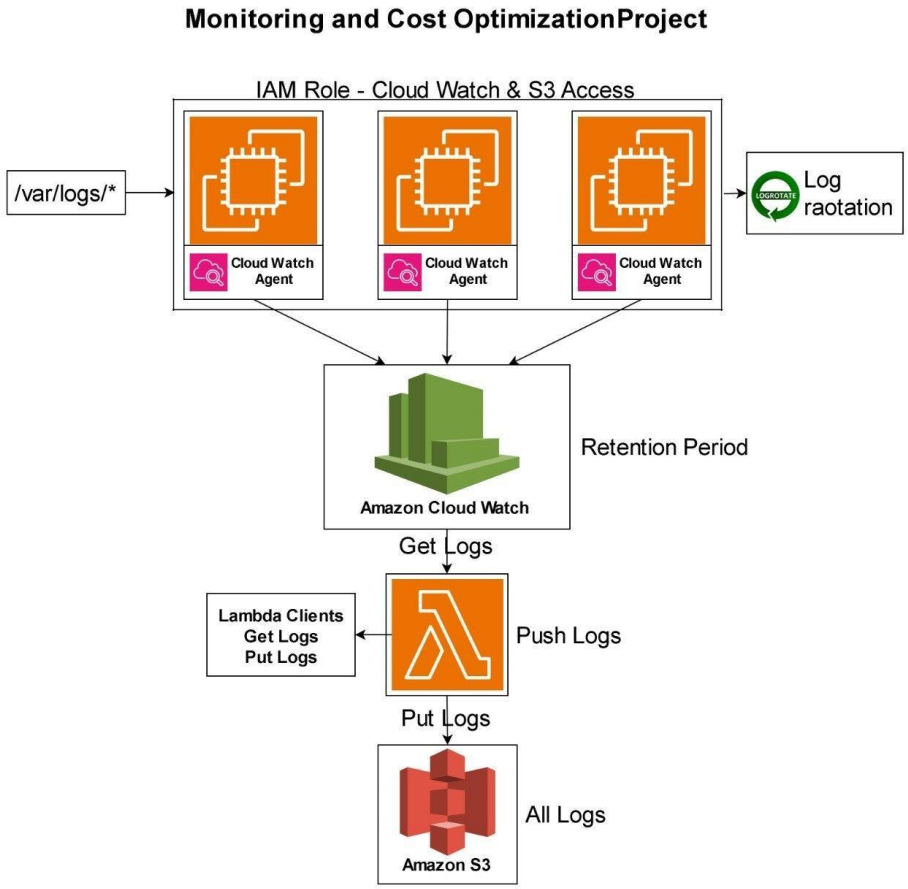
Theory Explained

Cold Start Optimization Tips: 1. Use Python or Node.js (fastest cold starts). Java is slowest. 2. Keep deployment package small — Lambda loads code package at cold start. 3. Use Lambda Layers for large shared dependencies. 4. Put only frequently changing code in function, stable libs in layers. 5. Use Provisioned Concurrency for customer-facing APIs. 6. Java: use SnapStart for faster cold starts.

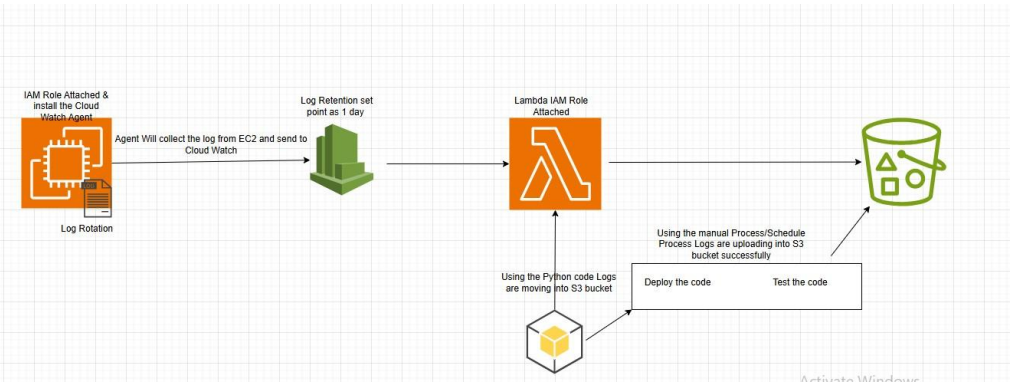
13.4 Lambda Layers

- A Layer is a ZIP archive of libraries, dependencies, or custom runtimes shared across multiple functions.
- Example: numpy + pandas + scikit-learn = 150MB. Include in EVERY function → each deployment is 150MB+ slow. Put in ONE Layer → all functions reference it. Deployments are tiny and fast.
- Up to 5 Layers per function. Combined unzipped size limit: 250 MB.
- Layers are versioned (v1, v2, v3) and immutable. Old functions keep old layer versions.
- AWS provides public layers for common runtimes. Share layers across accounts in your org.

13.5 Lambda Monitoring



CloudWatch + Lambda: Metric breach triggers Lambda for auto-remediation



Lambda Log Pipeline: CloudWatch Logs → Lambda → Forward to S3/SIEM

- All Lambda stdout/stderr automatically goes to CloudWatch Logs group /aws/lambda/function-name.

- Key metrics: Invocations, Duration, Errors, Throttles, ConcurrentExecutions, IteratorAge (for Kinesis/DynamoDB).
- Lambda can subscribe to any CloudWatch Log Group to process logs in real-time.

Scenario-Based Interview Questions — AWS Lambda — Serverless Compute

Q1: Scenario: Your Lambda connects to RDS. Under load, RDS runs out of connections and everything dies. Explain and fix.

Lambda scales to 1000 concurrent executions, each opening a DB connection, but db.t3.medium allows ~90. Fixes:
1) RDS Proxy — pools connections so 1000 Lambdas share a few real DB connections.
2) Reserved Concurrency — cap Lambda at 50 to protect the DB.
3) Open connection OUTSIDE the handler so warm invocations reuse it.
4) Consider DynamoDB which has no connection limit.

Q2: Scenario: An S3-triggered Lambda writes back to the SAME bucket. Your bill explodes overnight. What happened?

Infinite recursive loop. Lambda's write fires a new S3 event which invokes Lambda again — forever, scaling to max concurrency. Stop it NOW: set reserved concurrency to 0 (instant kill switch). Fix: write output to a DIFFERENT bucket. Or scope the event notification with prefix filters so trigger prefix and output prefix never overlap. Set a CloudWatch billing alarm.

Q3: Scenario: Lambda processes SQS messages. One bad message keeps failing and blocks everything. Fix it.

Poison-pill problem. Message fails, returns to queue after visibility timeout, gets re-delivered, fails again forever. Fix:
1) Dead Letter Queue with maxReceiveCount=3 — after 3 failures, message moves to DLQ.
2) ReportBatchItemFailures for partial batch response.
3) Visibility timeout >= 6x function timeout.
4) Alarm on DLQ.
5) Make handler idempotent — SQS standard guarantees at-least-once.

Q4: Scenario: Your Lambda has cold starts of 3 seconds. Users notice the delay on the first API call. Eliminate it.

1) Provisioned Concurrency — keeps N environments pre-initialized and warm. Zero cold starts. Costs money even when idle.
2) Keep deployment package small.
3) Move heavy init (DB connections, SDK clients) OUTSIDE the handler.
4) Increase memory (more CPU, faster init).

5) Use Node.js/Python (faster cold start than Java/.NET).

6) For Java: SnapStart reduces cold starts significantly.

Q5: Scenario: Your Lambda at 128 MB takes 10 seconds and costs $0.02 per invocation. At 1024 MB it takes 0.8 seconds. Which is cheaper?

Calculate: 128 MB × 10s = 1280 MB-seconds × $0.0000166667 = $0.021. 1024 MB × 0.8s = 819 MB-seconds × $0.0000166667 = $0.014. The higher-memory Lambda is 33% CHEAPER because it finishes so much faster. Counter-intuitive: more memory often LOWERS cost. Use AWS Lambda Power Tuning to find the sweet spot empirically.

Q6: Scenario: Your Lambda needs to access RDS in a private subnet. After putting Lambda in the VPC, it cannot reach S3 or the internet. Why?

Lambda in a VPC loses default internet access. It needs a NAT Gateway (costly) or VPC Endpoints to reach AWS services. Fix: create an S3 Gateway Endpoint (free), Interface Endpoints for Secrets Manager/ECR/CloudWatch. Only put Lambda in a VPC if it MUST reach private resources. Modern Hyperplane ENIs removed the old cold-start penalty but internet access still requires NAT or endpoints.

Q7: Scenario: Your Lambda function timeout is 15 minutes but your report generation takes 25 minutes. What do you do?

15 minutes is Lambda's HARD maximum timeout. Lambda is the wrong tool. Options:

1) AWS Step Functions — orchestrate the work across multiple Lambda invocations, each under 15 minutes.

2) ECS Fargate task — no timeout limit.

3) AWS Batch — for heavy batch processing.

4) Break the report into parallel chunks with Step Functions Map state, each chunk processed by a Lambda, results merged. Step Functions is the most common solution.

Q8: Scenario: Your Lambda is invoked asynchronously by S3. Some events are silently lost. How do you ensure none are lost?

Asynchronous invocation retries 2 times (3 attempts total), then discards the event. Configure a Destination on failure — send failed events to an SQS queue or SNS topic. Or configure a Dead Letter Queue (DLQ) on the function. Always set MaximumRetryAttempts and add an on-failure destination. Monitor the Errors and Throttles metrics. Without a failure destination, silently lost events are guaranteed.

Q9: Scenario: You deploy a new Lambda version and route 100% traffic immediately. The new version has a bug. 10,000 users are affected before you notice. How do you prevent this?

Use Lambda Aliases with weighted traffic shifting. Create a 'prod' alias pointing to the stable version. Deploy the new version and update the alias to route 5% of traffic to the new version (canary). Monitor error rates and latency for 10 minutes. If healthy, shift to 100%. If not, roll back the alias to 100% old version instantly. Use CodeDeploy with automatic rollback on CloudWatch alarm for full automation.

Q10: Scenario: Your Lambda handles 10,000 concurrent requests but your account limit is 1,000. What happens to the excess?

The excess requests are THROTTLED — they receive a 429 TooManyRequestsException. For synchronous invocations (API Gateway), the caller gets an error. For asynchronous invocations (S3, SNS), events go to an internal queue and retry for up to 6 hours. Fixes:

1) Request a concurrency limit increase from AWS Support.

2) Use Reserved Concurrency on critical functions to guarantee capacity.
3) Use SQS as a buffer between the event source and Lambda to smooth the load.
4) Use Provisioned Concurrency to pre-warm a portion.

2. Amazon ElastiCache — In-Memory Caching

ElastiCache is a fully managed in-memory data store. It sits in front of your database and serves frequently accessed data from RAM at microsecond speed. Dramatically reduces database load and improves application response time.

14.1 The Caching Pattern

Theory Explained
Without Cache: Every request → Application → Database → Disk read → Return data Latency: 5-50ms. DB CPU: 100%. DB connections: maxed out. With ElastiCache: Request → Check cache → CACHE HIT (90% of requests) → Return from RAM in < 1ms. DB never touched. CACHE MISS (10%) → Query DB → Store result in cache → Return. Result: 10x faster response. 90% less DB load. Database can handle 10x more users without scaling.

Feature	Redis	Memcached
Data persistence	Yes: AOF + RDB snapshots (survives restart)	No: all data lost on restart
Data types	Rich: strings, hashes, lists, sets, sorted sets, JSON	Strings only
Replication	Primary + Read Replicas + MultiAZ failover	No replication, no failover
Pub/Sub	Built-in publish/subscribe messaging	Not supported
Sorted Sets (leaderboards)	Native, O(log N) operations	Not supported
Multi-threading	Single-threaded (but very fast)	Multi-threaded

2.2 Redis vs Memcached

Best for	Production, persistence, rich data structures	Simple caching, max throughput with multi-threading

14.3 ElastiCache Use Cases

- Session Store: Store user sessions in Redis. User can hit ANY EC2 instance and stay logged in. Horizontal scaling becomes seamless.
- DB Query Cache: Cache results of expensive SQL JOINs and aggregations. Example: product catalog rarely changes — cache for 10 minutes, save 1000 DB queries/minute.
- Rate Limiting: Use Redis INCR to count API calls per user per minute. Increment on each request. If > 100, reject. O(1) operation.
- Leaderboards: Redis Sorted Sets are purpose-built for real-time rankings. Update score: ZADD, Get top 10: ZREVRANGE. Instant results even with millions of entries.
- Pub/Sub: Lambda publishes events to Redis channel. Other services subscribe and receive instantly. Simple real-time messaging.
- Distributed Locks: Use Redis SET with NX (not exists) + EX (expiry) for distributed locking across multiple EC2 instances.

VEERA

Scenario-Based Interview Questions — Amazon ElastiCache — In-Memory Caching

Q1: Scenario: Your app added Redis caching. Now users occasionally see ANOTHER user's data. What went wrong?

Cache key collision. The key does not include the user identity — e.g., cached under 'profile' instead of 'user:12345:profile'. Whoever's data lands first is served to everyone. Fix: include every input affecting the response in the key — user ID, tenant ID, locale. FLUSH the cache immediately to stop active leakage. Treat as a potential data breach. Same bug can happen at CloudFront/API Gateway caching layers.

Q2: Scenario: Redis is at 95% memory. Eviction rate climbing. Hit rate falling. What is happening and what are your options?

Redis is full and evicting keys to make room. Check maxmemory-policy: if noeviction, writes will FAIL with OOM. If volatile-lru, only keys WITH TTL are evicted — keys without TTL accumulate forever (common trap). Fixes:

1) Set allkeys-lru policy.

2) Scale up to larger node.

3) Add TTLs EVERYWHERE — keys without TTL accumulate forever.
4) If working set exceeds one node, enable Cluster Mode and shard horizontally.
5) Alarm on DatabaseMemoryUsagePercentage before it hits 95%.

Q3: Scenario: You deployed a price change but customers still see the old price for 10 minutes. Where do you look?

Four caches between DB and customer:
1) ElastiCache/Redis — check the price key's TTL.
2) API Gateway stage cache.
3) CloudFront edge cache.
4) Browser cache (Cache-Control header). Immediate: flush the specific Redis key and invalidate CloudFront. Permanent: on a price WRITE, actively delete the Redis key (write-through/explicit invalidation) rather than waiting out TTL. Never cache volatile business data at CloudFront/API Gateway — cache static assets there.

Q4: Scenario: Your Redis cluster (Cluster Mode Disabled) has 1 primary and 5 read replicas. You need more than 1 node's worth of memory. What do you do?

Enable Cluster Mode (requires creating a new cluster — you cannot convert in place). Cluster Mode shards data across up to 500 shards, each with its own primary and replicas. Total memory = number of shards x node memory. Your app client must be cluster-aware, and multi-key operations only work if keys are in the same slot (use hash tags like {user123}). Enable Cluster Mode when the dataset exceeds one node's RAM or write throughput exceeds one primary.

Q5: Scenario: Your app uses Lazy Loading (cache-aside). Every cache miss costs 3 trips (check cache, read DB, write cache). First users after cache restart see slow performance. How do you solve the cold cache problem?

1) Cache warming: on deployment/restart, run a script that pre-loads the most popular keys from the database into Redis before traffic hits.
2) Use Write-Through for the critical hot data — every DB write also writes to cache, so the cache is always warm.
3) For cache misses during normal operation, Lazy Loading with a TTL is still correct — the first user pays the 3-trip cost, everyone after gets the cached value.
4) Reduce TTL impact: add jitter to TTLs so all keys do not expire simultaneously.

Q6: Scenario: Thousands of requests hit your database simultaneously when a popular cache key expires. What is this called and how do you prevent it?

Cache Stampede (thundering herd). The key expires, thousands of concurrent requests all miss cache and hit the DB at once — often killing it. Prevention:
1) Mutex/lock — only the FIRST request refreshes the value while others wait or serve stale data.
2) Add jitter to TTLs (300s ± random 60s) so keys do not expire at the same time.
3) Probabilistic early refresh — refresh a key slightly before expiry.
4) Stale-while-revalidate: return old value and refresh in background.

Q7: Scenario: Your team is choosing between Redis and Memcached. What are the deciding factors?

Redis for 95% of use cases: rich data structures (lists, sets, sorted sets, hashes), persistence, replication, automatic failover, pub/sub, Lua scripting, geospatial queries. Memcached: simple key-value only, multi-threaded (uses multiple cores per node), no persistence, no replication.

Choose Memcached only for the simplest possible object cache where you want to use multiple CPU cores per node and can tolerate data loss on node failure.

Q8: Scenario: Your Redis primary node in AZ-a fails. Your app goes down because there is no failover. How should this have been configured?

Enable Multi-AZ with automatic failover. Place replicas in different AZs. If the primary fails, ElastiCache automatically promotes a replica to primary and updates the DNS endpoint — usually within a couple of minutes. Your app keeps using the same endpoint. Without Multi-AZ, a primary failure means your cache is gone and every request stampedes to the database — this is how a cache outage becomes a full site outage.

Q9: Scenario: Your Lazy Loading cache uses a fixed TTL of 3600 seconds for all keys. Product prices update every 5 minutes but user profile settings change once a month. What is wrong?

One TTL does not fit all data. Prices with 3600s TTL means customers see stale prices for up to an hour — unacceptable for e-commerce. Profile settings with 3600s TTL means unnecessary cache misses and DB reads every hour for data that rarely changes. Fix: use different TTLs per data type. Prices: 60-120 seconds (low staleness tolerance). Profiles: 86400 seconds (24 hours — rarely changes). Static reference data (country list): 604800 seconds (1 week). TTL should match your tolerance for staleness.

Q10: Scenario: Your application uses Redis for session storage. During a scaling event, the new EC2 instances cannot connect to Redis. The existing instances work fine. What is wrong?

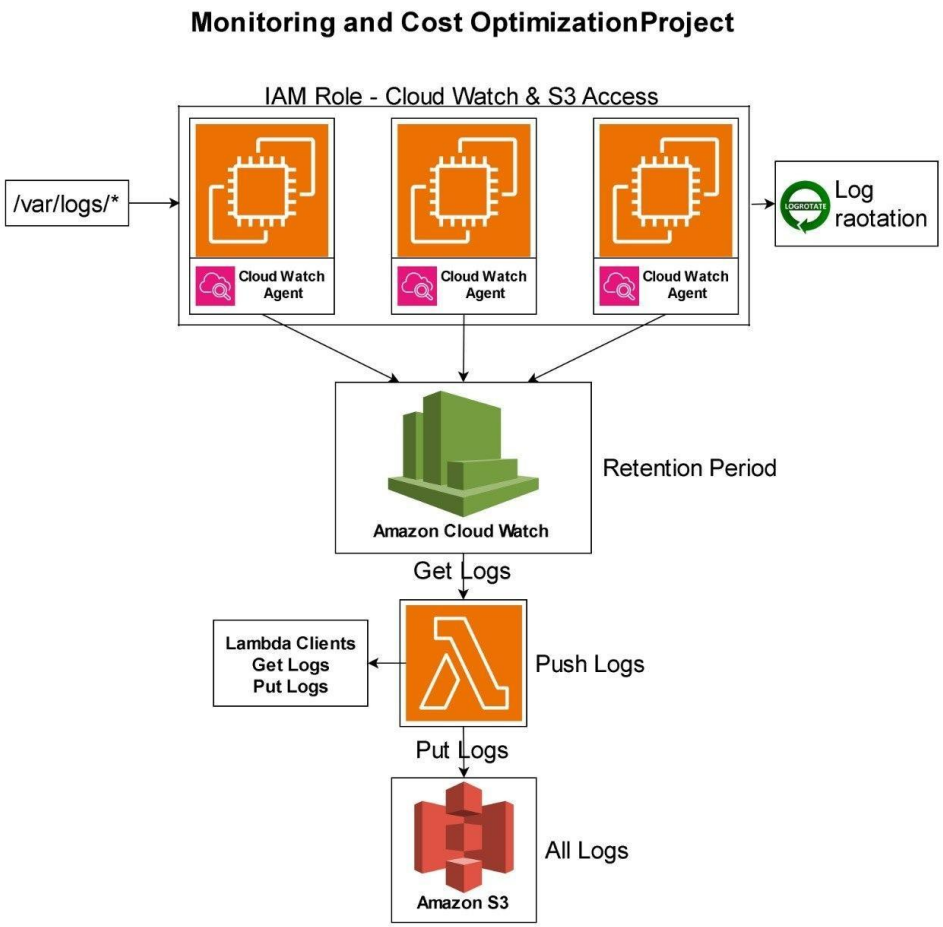
The Redis Security Group allows inbound from the OLD instances' Security Group, but the new instances may be in a different SG or the ASG is using a different SG than expected. Fix: ensure the Redis SG allows inbound on port 6379 from the App-SG (the security group attached to ALL app instances, including those launched by the ASG). SG-to-SG referencing handles this automatically when new instances get the same SG. Also check: the new instances are in the same VPC, the subnets have routes to the Redis subnet, and the Redis endpoint is correctly configured in the app (not a hardcoded IP from an old node).

15. Monitoring & Observability

You cannot manage what you cannot measure. The three pillars of observability: METRICS (numbers over time — what's happening), LOGS (detailed event records — what happened), TRACES (request tracking — how a request flows through multiple services).

15.1 Amazon CloudWatch

CloudWatch is AWS's native monitoring hub. Every AWS service automatically publishes metrics and logs here. You build dashboards, set alarms, and trigger automated actions.



CloudWatch: Metrics → Alarms → Actions (ASG scale / SNS alert / Lambda remediate / EC2 stop)

Metrics

- Every AWS service auto-publishes metrics: EC2 (CPU, network, disk), ALB (requests, latency, HTTP 5xx), RDS (CPU, connections, storage), Lambda (invocations, errors, duration).
- Default resolution: 1-minute data points. High-resolution: 1-second (extra cost).
- Custom Metrics: Use PutMetricData API. Example: active_users, queue_depth, cache_hit_ratio.
- CloudWatch Agent: Install on EC2 to collect OS metrics NOT available by default: memory %, disk space %, swap, running processes, app-specific metrics.

Alarms

- Monitor one metric. Define threshold + period + evaluation count.

- Three states: OK (normal), ALARM (threshold breached), INSUFFICIENT_DATA (not enough data points yet).
- ALARM actions: trigger ASG policy (scale out), publish to SNS (notify team), invoke Lambda (auto-fix), stop/restart EC2.

Real-World Example
Production alarm: Average 5xx error rate > 1% for 2 consecutive 1-minute periods → ALARM → SNS notification to team Slack channel + Lambda auto-restarts the unhealthy instances + PagerDuty creates incident.

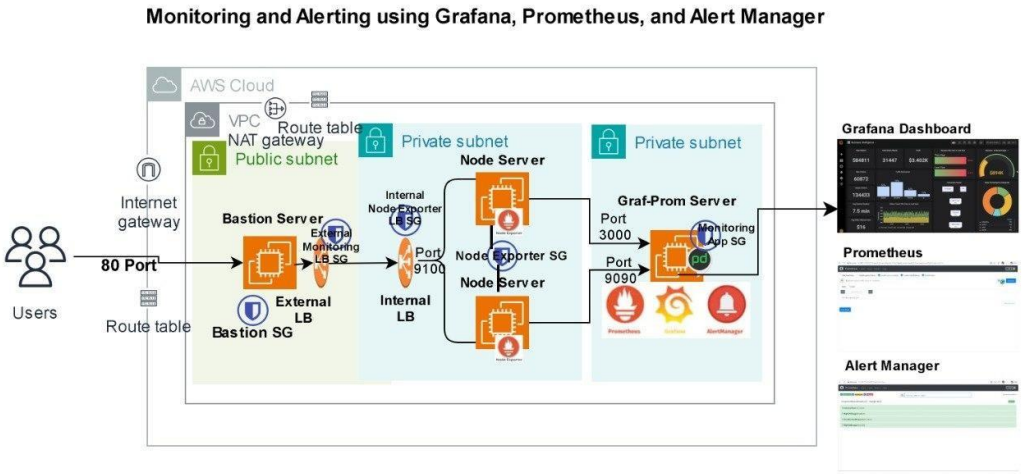
CloudWatch Logs

- Collect logs from: EC2 (via CloudWatch Agent), Lambda (automatic), RDS, ECS, API Gateway, VPC Flow Logs.
- Log Groups: Container per service (/aws/lambda/my-function, /var/log/nginx/access.log).
- Log Streams: Individual sequences within a group. Each Lambda invocation = one log stream.
- Logs Insights: SQL-like interactive query language. Example: find all errors in last hour, calculate P99 latency from logs.
- Metric Filters: Extract a number from logs and turn it into a CloudWatch metric. Example: count ERROR lines per minute → alert if > 10.
- Retention: Default = never expire. Always set retention policy (30 or 90 days) to control costs.

VEERA

15.2 Prometheus — Open Source Metrics

Prometheus is an open-source time-series monitoring system. It SCRAPES (pulls) metrics from targets at configurable intervals. Widely used for Kubernetes, Docker, and EC2 environments.

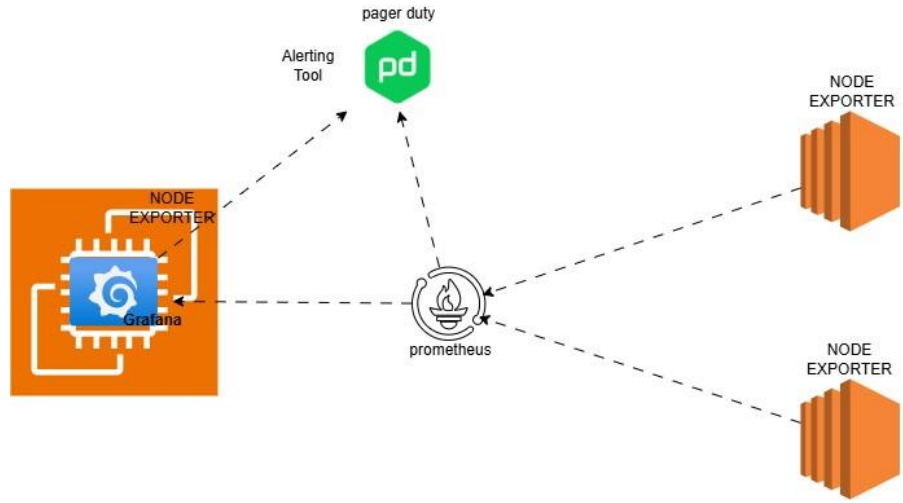


Prometheus + Grafana: Scrape metrics → Time-series storage → PromQL queries → Dashboards

- Pull Model: Prometheus contacts each target (default every 15 seconds) and fetches metrics from a /metrics HTTP endpoint. Targets expose their own metrics.
- Node Exporter: Agent installed on Linux servers. Reads /proc and /sys filesystems, exposes CPU, memory, disk, network metrics at :9100/metrics for Prometheus to scrape.
- Time-Series: Data stored as (metric_name + labels) → timestamps + values. Example: cpu_usage{instance='10.0.1.5',job='ec2'} → [(1706000000, 45.2), (1706000015, 47.8), ...].

```
# PromQL examples rate(http_requests_total[5m])                    #
requests/second over 5 min avg by (instance)(node_cpu_seconds_total)       # avg
CPU per instance histogram_quantile(0.99, http_duration_bucket)  # P99 response
time count(up == 0)                                  # count of down targets
```

15.3 Grafana — Dashboards & Visualization

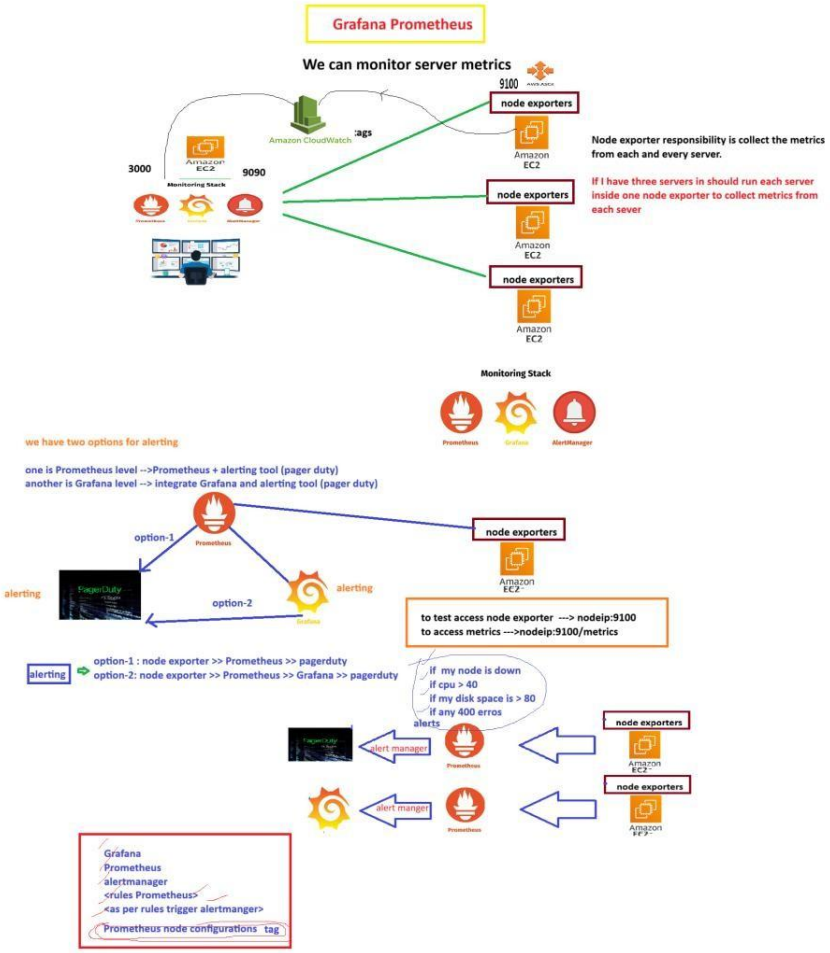


Grafana Dashboard: Multiple panels — CPU graph, request rate, error rate, latency

- Grafana connects to data sources: Prometheus, CloudWatch, Elasticsearch, InfluxDB, MySQL, and 50+ others.
- Dashboard: Collection of panels. Each panel = one chart, graph, gauge, bar chart, or table.
- Alerting: Grafana can send alerts to Slack, PagerDuty, email, OpsGenie, webhooks when metric thresholds are breached.
- Variables: Dynamic dashboards with dropdowns. Switch between environments, instances, regions without rebuilding.
- Provisioning: Define dashboards as JSON files, deploy via GitOps — dashboards as code.

VEERA

15.4 Alertmanager — Alert Routing & Deduplication



Alertmanager: Prometheus alerts → Group/Deduplicate/Silence/Route → PagerDuty/Slack

- Prometheus FIRES alerts when PromQL rules trigger. Alertmanager RECEIVES and MANAGES those alerts.
- Grouping: Multiple related alerts bundled into ONE notification. Prevents alert storm flooding Slack.
- Deduplication: Same alert firing 20 times → only 1 notification. No spam.
- Silencing: Suppress alerts during planned maintenance windows.
- Inhibition: If 'server-down' critical fires, suppress 'high-cpu' warning for same server — root cause is obvious.
- Routing tree: Database alerts → DBA team. Infrastructure alerts → Ops team. Security alerts → Security team. Different receivers, different urgency.

15.5 PagerDuty — On-Call & Incident Management

- PagerDuty is the on-call management platform. Receives alerts from Prometheus Alertmanager, CloudWatch, Grafana, Dynatrace, and 700+ integrations.
- On-Call Schedules: Who is on-call for each service right now. Automatic weekly rotation.
- Escalation Policies: Primary doesn't ack in 5 minutes? Page secondary. Secondary doesn't ack? Page manager. Automatic escalation chain.
- Notification methods: Push notification → SMS → phone call. Escalates until someone responds.
- Incident lifecycle: Triggered → Acknowledged (someone is working it) → Resolved. Full timeline and audit trail.
- Runbooks: Link runbooks to alerts so on-call engineer knows exactly what to do.

15.6 Dynatrace — Full-Stack APM

Dynatrace is an enterprise APM platform with AI-powered root cause analysis. OneAgent autoinstruments your entire stack with zero configuration.

Architecture Flow

Dynatrace Architecture: User Request → App on EC2 → OneAgent intercepts every method call, DB query, HTTP call → sends traces+metrics+logs to Dynatrace SaaS → Davis AI analyzes → detects: High CPU on EC2 → slow DB connection pool → user error rate spike → creates one Problem ticket with full root cause chain automatically → PagerDuty alert sent.

- OneAgent: Single agent. Auto-discovers all processes, services, dependencies on installation. No manual instrumentation.
- ActiveGate: Proxy/aggregator for OneAgents. Required for secure environments. Enables cloud platform monitoring.
- Davis AI: AWS calls it 'causal AI'. Groups related symptoms into one 'Problem'. Shows exact root cause. 'DB slow query caused by connection pool exhaustion caused by memory leak in service X'.
- Smartscape: Live topology map — how everything connects. Instantly see impact radius of any issue.
- Real User Monitoring (RUM): Track actual user sessions, click paths, page load times, JavaScript errors.
- Synthetic Monitoring: Simulates user requests from 30+ global locations every minute. Detects issues before real users do.

Scenario-Based Interview Questions — Monitoring & Observability

Q1: Scenario: You get 200 alert emails a night. The team ignores all of them. A real outage went unnoticed for 2 hours. Fix this.

Alert fatigue. Fixes:

1) Every alert must be ACTIONABLE — if nobody acts on it, delete it or make it a dashboard metric.
2) Alert on SYMPTOMS (error rate, p99 latency, checkout success) not CAUSES (CPU 81%). High CPU with happy customers is not an incident.
3) Use composite alarms — one root cause fires ONE alert, not 40 child alarms.
4) Route by severity: P1 to PagerDuty phone call, P2 to Slack, P3 to ticket. Email is where alerts die.
5) Review every page weekly — was it actionable? Target: <2 pages per on-call shift.

Q2: Scenario: Users say the site is slow but CloudWatch shows CPU 25%, memory 40%, all health checks green. Where do you look?

Green infrastructure with unhappy users means you are measuring the wrong things.

1) Check ALB TargetResponseTime p99 (not average — p50 200ms with p99 8s is a disaster that averages to fine).
2) Check database — Performance Insights for slow queries, lock waits. A DB waiting on a lock uses zero CPU.
3) Check downstream dependencies (X-Ray) — a third-party API timing out shows zero server load.
4) Check from the USER side — CloudWatch RUM/synthetic canaries for actual page load times. Server metrics cannot see DNS, TLS, CDN misses, or bloated JS bundles.

Q3: Scenario: Your CloudWatch Logs bill is Rs 4 lakh/month. Reduce it without losing debug ability.

1) Set RETENTION on every log group — default is Never Expire. Most need 7-30 days of hot logs, not years.
2) Stop logging noise — DEBUG level in production, health check logs every 30s, full request bodies.
3) Ingestion costs more than storage — filter at the SOURCE.
4) Export to S3+Glacier for compliance, query with Athena when needed (order of magnitude cheaper).
5) Use metric filters to turn log patterns into cheap metrics instead of storing every line.
6) Check for runaway loops — a recursive Lambda can generate terabytes overnight.

Q4: Scenario: Which important EC2 metrics does CloudWatch NOT collect by default? Your instance ran out of memory with no alarm.

Memory usage and disk space. AWS only sees the hypervisor view — CPU, network, disk I/O, status checks. It cannot see inside your OS. To get RAM and disk: install the CloudWatch Agent and publish them as custom metrics. This is the most common production surprise: 'my instance ran out of memory but there was no alarm' — because nobody installed the agent.

Q5: Scenario: Your EC2 instance failed a StatusCheckFailed_System check. What does this mean and what is the fix?

System status check failure means the problem is on AWS's side: the underlying host hardware, its network, or power. The fix: STOP and START the instance (not reboot). This migrates it to a new healthy physical host. Rebooting stays on the same host and does not fix hardware issues. Instance status check failure means YOUR problem — corrupted filesystem, exhausted memory, bad kernel. Knowing which check fired tells you whether to call AWS or check your own logs.

Q6: Scenario: Your microservices architecture has 15 services. A user request takes 8 seconds but each service's individual latency looks normal. How do you find the bottleneck?

Use AWS X-Ray (distributed tracing). X-Ray assigns a trace ID to the request and follows it across all 15 services, producing a service map and timing waterfall. You will see exactly which service/call is responsible for the 8 seconds. Without tracing, you are guessing across 15 log files. Common finding: one service has a 3-second call to an external API that nobody knew about, or a fan-out pattern where 5 sequential calls could be parallelized.

Q7: Scenario: Your team uses CloudWatch for AWS metrics and Prometheus+Grafana for Kubernetes app metrics. Do you need both?

Yes, they serve different strengths. CloudWatch: best for AWS infrastructure metrics (ALB latency, RDS CPU, Lambda duration), alarms, and integration with AWS services. Prometheus: pull-based, open-source, standard for Kubernetes, rich query language (PromQL), application-level metrics. Grafana: visualization layer that queries BOTH — CloudWatch and Prometheus — in a single pane of glass. This is the standard setup. AWS offers managed versions (AMP for Prometheus, AMG for Grafana) to reduce operational burden.

Q8: Scenario: After a production incident, you need to find out exactly who changed a Security Group rule at 3 AM last Tuesday. Where do you look?

CloudTrail. Every AWS API call is logged: the identity (who), the action (AuthorizeSecurityGroupIngress), the parameters (what rule was added/removed), the source IP, and the timestamp. Search CloudTrail event history or query Athena over the S3 trail for the SecurityGroup events on that date. CloudTrail answers WHO did WHAT and WHEN. This is separate from Config, which answers WHAT the resource looks like now vs then.

Q9: Scenario: You want to be alerted when your application error rate exceeds 5% of total requests. How do you set this up?

Use a CloudWatch Math Expression alarm. Create two metrics: count of 5xx responses (from ALB HTTPCode_Target_5XX) and total requests (RequestCount). Create a metric math expression: error_rate = (errors / total) * 100. Set an alarm on error_rate > 5 for 3 consecutive 1-minute periods (M-out-of-N evaluation prevents flapping on single blips). Action: SNS to PagerDuty. This is a symptom-based alarm (customer impact) rather than a cause-based alarm (CPU) — the correct approach.

Q10: Scenario: Your team has no monitoring dashboards. The CEO says build one. What are the 5 most important metrics to put on it?

1) Error rate — percentage of 5xx responses (the most direct measure of customer impact).
2) Latency p99 — the 99th percentile response time (not average — average hides everything).
3) Request count — traffic volume (is it growing, spiking, or dropping to zero?).
4) Availability — synthetic canary success rate (is the site actually up from the outside?).

5) Active alarms — how many P1/P2 incidents are currently open. All of these measure the CUSTOMER experience, not the infrastructure. CPU and memory belong on a deeper technical dashboard, not the executive dashboard.

16. AWS Messaging — SNS, SQS & EventBridge

Modern cloud apps are event-driven. Services communicate asynchronously through messages rather than synchronous direct calls. This decoupling makes systems more resilient (failures don't cascade), scalable (each service scales independently), and maintainable (services change without impacting each other).

16.1 Amazon SNS — Simple Notification Service

SNS implements publish-subscribe. One publisher, many subscribers. One message to a topic → ALL subscribers receive it simultaneously. Fan-out at scale.

- Topic: A named channel. Like a TV broadcast channel — all subscribers tuned in receive the broadcast.
- Publisher: Your app, CloudWatch Alarm, S3 event, CodeDeploy, or any AWS service.
- Subscriber: Lambda, SQS queue, HTTP/S endpoint, email address, SMS number, mobile push (iOS/Android).

Real-World Example
Fan-out example: User uploads a document. 1. S3 event → SNS topic 'document-uploaded' 2. SNS simultaneously triggers: → Lambda (extract text, run OCR) → SQS queue (virus scan) → SNS email (notify admin) → Lambda (update search index) All four happen in parallel. One S3 upload, four actions, zero coupling.

SNS Type	Ordering	Delivery	Throughput	Use Case
Standard Topic	Best-effort	At-least-once (may duplicate)	Unlimited	High-volume notifications, fan-out
FIFO Topic	Strict FIFO	Exactly-once	3,000/sec	Order-critical: financial events, inventory

16.2 Amazon SQS — Simple Queue Service

SQS decouples producers from consumers. Messages persist in the queue until consumed. Even if consumers are down for hours, messages wait safely. Critical for building resilient systems.

Feature	Standard Queue	FIFO Queue
Throughput	Nearly unlimited	300/sec (3,000 with batching)
Ordering	Best-effort (not guaranteed)	Strict FIFO — guaranteed
Delivery	At-least-once (may deliver twice)	Exactly-once — no duplicates
Message size	Up to 256 KB	Up to 256 KB
Retention	Default 4 days, max 14 days	Same
Use case	High volume, order unimportant	Finance, inventory, orders — order matters

- **Visibility Timeout:** After a consumer reads a message, it's hidden from other consumers for this duration (default 30s, max 12h). Prevents double-processing. If consumer crashes without deleting, message reappears after timeout.
- **Dead Letter Queue (DLQ):** After maxReceiveCount failed attempts, message moves to DLQ. Inspect DLQ to debug. Prevents bad messages from blocking the queue forever.
- **Delay Queue:** Messages invisible for 0–15 minutes after arrival. Example: send follow-up email 5 min after order.
- **Long Polling:** Consumer waits up to 20 seconds for a message before returning empty. Reduces empty responses and API cost vs short polling.

> 🕮 **Theory Explained**
> Why SQS Saves Systems: Web server: 1000 orders/min. Payment processor: handles 100/min. Without SQS: 900 orders lost or web server crashes. With SQS: 1000 messages queued. Payment processor reads 100/min. Zero messages lost. System never overwhelmed. Horizontal scaling of processors is simple.

16.3 Amazon EventBridge

- Serverless event bus connecting 200+ AWS services, SaaS apps, and your own apps with rich routing.
- Event Bus types: Default (AWS service events), Custom (your own events), Partner (SaaS providers).
- Rules: JSON pattern matching. Route matching events to up to 5 targets each.
- Schema Registry: Auto-discovers event schemas. Generate typed code from schemas.

Real-World Example

Order fulfillment via EventBridge: Order service publishes 'order.created' → EventBridge routes to: 1. Lambda (reserve inventory) 2. Lambda (charge payment) 3. SNS (send confirmation email) 4. Step Functions (coordinate fulfillment workflow) All parallel, all automatic, all loosely coupled.

Scenario-Based Interview Questions — AWS Messaging — SNS, SQS & EventBridge

Q1: Scenario: Your order-processing consumer charged some customers twice. Explain how and prevent it.

Two causes:

1) SQS Standard guarantees at-least-once — duplicates are normal, not a bug.

2) Visibility timeout shorter than processing time — message reappears mid-processing, second consumer picks it up. Fix: IDEMPOTENCY. Record the orderId in DynamoDB with a conditional write (attribute_not_exists) BEFORE charging. If key exists, skip. Pass the idempotency key to the payment provider too. Set visibility timeout to 6x processing time. Never assume exactly-once delivery.

Q2: Scenario: When a user uploads a photo, you must: make a thumbnail, run moderation, send email, update analytics. If email fails, others must still work. Design it.

SNS fan-out pattern. S3 Event -> SNS topic -> 4 separate SQS queues (thumbnail, moderation, email, analytics). Each queue has its own Lambda consumer, own retry policy, own DLQ. Email failure only fills the email DLQ — other paths never know. If S3 triggered one Lambda doing all four, an email failure would retry and re-run everything.

Q3: Scenario: Your SQS queue has 2 million messages backed up and growing. What do you do?

1) Is the consumer DOWN or TOO SLOW? Check ApproximateAgeOfOldestMessage.

2) If down: check errors and DLQ for a poison-pill or bad deploy.

3) If slow: scale consumers — for Lambda raise concurrency and batch size; for EC2 use target tracking on queue depth.

4) Check downstream bottleneck — database throttling makes scaling consumers worse.

5) Enable batching (10 messages per receive) and long polling.

6) Raise message retention to 14 days (default

4) so messages are not silently lost.

Q4: Scenario: You are using short polling on SQS. You make millions of empty ReceiveMessage calls. What is the fix?

Enable Long Polling: set ReceiveMessageWaitTimeSeconds to 20. SQS holds the connection until a message arrives or timer expires. This eliminates empty responses, reduces cost dramatically, reduces latency (message returned instantly on arrival), and eliminates false-empty responses from short polling. There is essentially no reason to use short polling.

Q5: Scenario: A poisoned message in your FIFO queue blocks all subsequent messages in that MessageGroupId. How do you handle it?

Configure a Dead Letter Queue with maxReceiveCount (e.g., 3). After 3 failures, the bad message moves to the DLQ, unblocking the MessageGroupId for subsequent messages. Alarm on DLQ depth. Inspect the failed message, fix the bug, then use SQS redrive to send DLQ messages back to the source queue. For Standard queues, use ReportBatchItemFailures for partial batch response so one bad message in a batch of 10 does not force all 10 to retry.

Q6: Scenario: You need to route events based on content: only orders over Rs 1 lakh should trigger fraud review. SNS cannot filter on amount. What do you use?

EventBridge. It supports content-based filtering with rich rules on the event JSON structure: { 'detail': { 'amount': [{ 'numeric': ['>', 100000] }] } }. SNS filtering is limited to message attributes (not body content). EventBridge also has schema registry, archive/replay, a scheduler, and native SaaS integrations. Use SNS for high-throughput simple fan-out; EventBridge for smart content-based routing.

Q7: Scenario: Your Lambda processes SQS messages. The function timeout is 30 seconds but the visibility timeout is 20 seconds. You see duplicate processing. Why?

The message reappears at 20 seconds while Lambda is still processing it (takes up to 30 seconds). Another invocation picks it up and processes it again. Fix: set visibility timeout to at LEAST 6x the function timeout — 180 seconds minimum. This gives enough buffer for retries and prevents the message from reappearing during processing.

Q8: Scenario: You need to process messages in strict order. SQS Standard does not guarantee order. What do you use?

SQS FIFO queue. It guarantees strict ordering and exactly-once processing within a MessageGroupId. Requirements: queue name must end with .fifo, you must provide MessageGroupId (ordering scope) and MessageDeduplicationId. Throughput: 300 msg/sec (3,000 with batching). Use Standard (unlimited throughput, at-least-once) when order does not matter. Use FIFO only when order genuinely matters — e.g., bank transactions on one account.

Q9: Scenario: Your app publishes order events to SNS. Three subscribers: email service, analytics, and warehouse. Email service is down. What happens to those messages?

If the subscriber is a Lambda or HTTP endpoint: SNS retries with exponential backoff for hours (for Lambda) or as configured (for HTTP). If retries exhaust, the message is lost unless you configured an SNS DLQ (redrive policy) for that subscription. If the subscriber is an SQS queue: the message is durably stored in the queue regardless of whether the consumer is running — it waits until the consumer comes back. This is why the SNS->SQS pattern is preferred: SQS provides durability that direct SNS->Lambda/HTTP does not.

Q10: Scenario: You need to run a task every day at 6 AM IST to generate reports. Which messaging service has built-in scheduling?

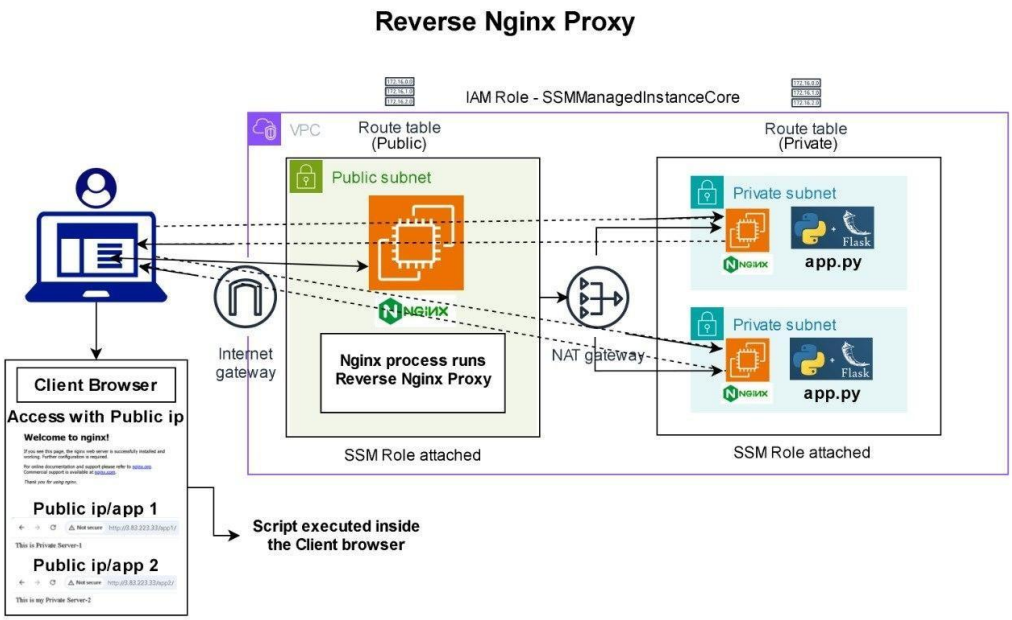
EventBridge Scheduler. Create a schedule with a cron expression (cron(30 0 * * ? *) for 6 AM IST/0:30 UTC) targeting a Lambda function, Step Functions, SQS, or other AWS services. EventBridge Scheduler replaced CloudWatch Events for this use case and supports one-time

schedules, rate-based schedules, and cron schedules with timezone support. It is purpose-built for scheduled tasks — no need to run a cron server.

17. Nginx — Web Server & Reverse Proxy

Nginx (Engine-X) is a high-performance open-source web server and reverse proxy. In AWS architectures, Nginx runs on EC2 instances to serve as the gateway to backend applications — handling SSL termination, request routing, load balancing, and static file serving.

17.1 Nginx Reverse Proxy Architecture



Nginx Reverse Proxy: Browser → Frontend EC2 (Nginx) → Backend EC2 (App Server) via Private IP

Architecture Flow

Complete Nginx Reverse Proxy Flow: 1. User browser → ALB (public, HTTPS:443) → Nginx EC2 (private subnet, HTTP:80) 2. Nginx reads the URL path and decides where to forward 3. Nginx → Backend App Server via PRIVATE IP (10.0.2.50:3000) 4. Backend processes request, returns JSON response to Nginx 5. Nginx forwards response back through ALB to user Key benefit: Backend NEVER has a public IP. Only Nginx's security group allows traffic. Internet cannot directly reach the application.

17.2 Nginx Configuration Examples

Basic Reverse Proxy

```
# /etc/nginx/conf.d/myapp.conf server {     listen 80;     server_name
myapp.com;      location / {         proxy_pass http://10.0.2.50:3000;        #
backend private IP         proxy_set_header Host $host;         proxy_set_header
X-Real-IP $remote_addr;  # pass real client IP         proxy_set_header
XForwarded-For $proxy_add_x_forwarded_for;         proxy_connect_timeout 30s;
proxy_read_timeout 60s;     } }
```

Path-Based Routing

```
server {     listen 80;     server_name myapp.com;      location /api/ {
proxy_pass http://10.0.2.51:5000;   # API backend     }     location /static/ {
root /var/www/html;                  # serve static files from disk
expires 7d;                          # browser cache 7 days     }     location /
{         proxy_pass http://10.0.2.52:3000;   # main app     } }
```

17.3 Security Group Configuration for Nginx

Instance	Security Group Rule	Direction	Reason
Frontend EC2 (Nginx)	TCP 80 from ALB-SG	Inbound	ALB forwards HTTP to Nginx
Frontend EC2 (Nginx)	TCP 443 from 0.0.0.0/0	Inbound	Direct HTTPS if no ALB
Backend EC2 (App)	TCP 3000/5000 from Frontend-SG	Inbound	Only Nginx EC2 can reach app
Backend EC2 (App)	TCP 22 from Bastion-SG	Inbound	SSH only via bastion

Key Tip
ALWAYS reference Security Group IDs (not IP ranges) as sources for backend security groups. When ASG launches new Nginx instances they get new IPs — but they all have the same SG ID. Backend automatically accepts them without any rule changes.

Scenario-Based Interview Questions — Nginx — Web Server & Reverse Proxy

Q1: Scenario: Users get 502 Bad Gateway from Nginx. Debug it systematically.

502 means Nginx could not get a valid response from the backend. Check:

1) Is the backend running? curl http://127.0.0.1:3000 from the server.
2) Read /var/log/nginx/error.log — it names the cause: connection refused (app down), no live upstreams (all health checks failed), upstream prematurely closed connection (app crashed, check for OOM).
3) SELinux on RHEL/CentOS blocks Nginx network connections by default: setsebool -P httpd_can_network_connect 1.
4) Wrong proxy_pass port or missing trailing slash.
5) Socket permissions for Unix sockets.

Q2: Scenario: Behind an ALB, Nginx access logs show every request from 10.0.1.x. You cannot see real user IPs. Fix it.

The ALB is a reverse proxy, so Nginx sees the ALB's ENI IP. Real client IP is in X-Forwarded-For header. Fix: add set_real_ip_from 10.0.0.0/16 (VPC CIDR), real_ip_header X-Forwarded-For, real_ip_recursive on. Now $remote_addr becomes the true client IP. Security-critical: set_real_ip_from restricts trust to ALB's subnet only — a client can forge X-Forwarded-For. Your app must also read X-Forwarded-For for rate limiting and fraud detection.

Q3: Scenario: Users uploading 50 MB files get 413 Request Entity Too Large. Fix it fully.

1) Nginx: client_max_body_size 100M (default is only 1 MB).
2) Also raise client_body_timeout and proxy_read_timeout for slow uploads.
3) There are usually THREE limits in the chain: ALB, Nginx, and your app framework (Express body-parser, multer). Fix all three.
4) If on API Gateway, hard limit is 10 MB — cannot be raised.
5) Architectural answer: use S3 pre-signed URLs so the browser uploads directly to S3, bypassing Nginx entirely.

Q4: Scenario: You changed Nginx config and ran systemctl restart nginx. The site was down for 2 seconds. How do you avoid this?

Never restart a production Nginx for config changes. Use nginx -t to validate the config first (a syntax error on restart means your site stays down). Then nginx -s reload (or systemctl reload nginx). Reload gracefully spawns new workers with the new config while OLD workers finish in-flight requests before exiting — ZERO downtime, no dropped connections. Restart kills everything immediately — every in-flight request dies.

Q5: Scenario: Your Nginx reverse proxy shows 504 Gateway Timeout when users generate large reports. The report takes 90 seconds but Nginx times out at 60 seconds.

proxy_read_timeout defaults to 60 seconds. Your 90-second report exceeds it. Fix: increase proxy_read_timeout to 120 seconds in the relevant location block. Also review proxy_connect_timeout and proxy_send_timeout. But the better architectural fix: make report generation asynchronous — return 202 Accepted immediately, process in the background, let the client poll for completion. Long-running synchronous requests are fragile.

Q6: Scenario: You configured Nginx as a load balancer with round-robin across 3 backend servers. One server is much slower than the others. Users hitting that server have bad experience. What should you change?

Switch from round-robin to least_conn in the upstream block. least_conn sends new requests to the backend with the fewest active connections. The slow server naturally accumulates connections, so new requests go to the faster servers. Round-robin blindly distributes equally regardless of server speed. For unequal server sizes, use weight= to assign proportional traffic.

Q7: Scenario: Your Nginx upstream has 3 backends. One dies. Nginx keeps sending traffic to it for 30 seconds before marking it down. Users see errors. How do you detect failures faster?

Configure passive health checks: max_fails=2 fail_timeout=10s in the upstream server directive. After 2 failed requests within 10 seconds, Nginx marks the server as unavailable for 10 seconds before retrying. Tune max_fails and fail_timeout for your tolerance. For active health checks (proactive, not waiting for failures), you need Nginx Plus or use the open-source nginx_upstream_check_module. The ALB in front handles this better — but if Nginx IS your load balancer, passive health checks are essential.

Q8: Scenario: Your Nginx serves both static files and proxies API requests. Static file performance is poor. How do you optimize it?

1) Enable sendfile on — uses kernel-level file transfer, bypassing userspace copying.
2) Enable tcp_nopush and tcp_nodelay.
3) Enable gzip compression for text/html, text/css, application/javascript.
4) Set expires headers (expires 30d) for static assets so browsers cache them.
5) Consider serving static files from S3+CloudFront instead of Nginx — CDN is faster and offloads Nginx entirely. Nginx should focus on proxying dynamic requests.

Q9: Scenario: You see 'Too many open files' errors in Nginx under high traffic. What is the fix?

Each connection uses a file descriptor. The OS limit is too low. Fix:

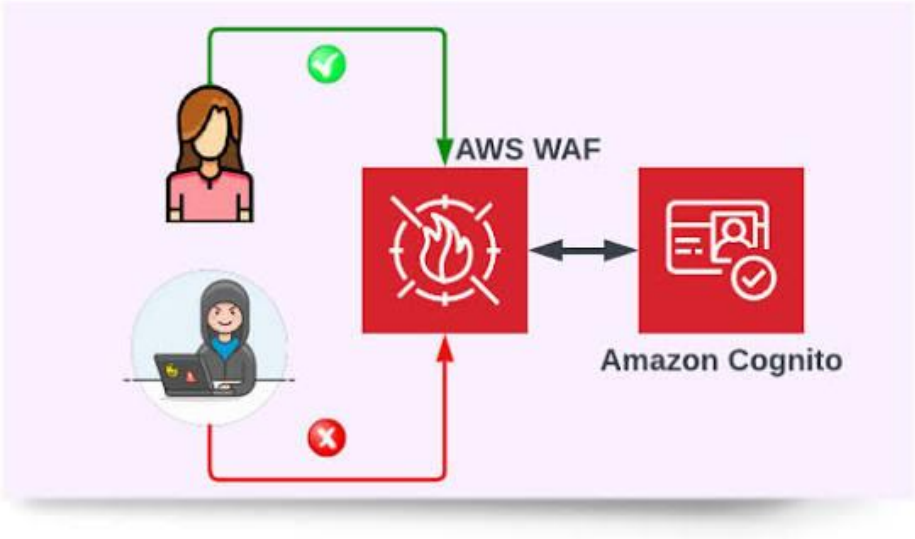
1) Increase the OS limit: add '* soft nofile 65535' and '* hard nofile 65535' to /etc/security/limits.conf.
2) In nginx.conf: set worker_rlimit_nofile 65535.
3) Increase worker_connections to 4096+ per worker. Maximum clients = worker_processes x worker_connections. For a reverse proxy, halve it because each client connection also needs a backend connection.

Q10: Scenario: Your Nginx config has location = /api/health, location /api/, and location ~ ^/api/v[0-9]+/. A request for /api/v2/users is matching the wrong block. Explain the matching order.

Nginx location matching order:

1) Exact match (=) — highest priority, stops immediately.
2) Prefix match with ^~ — stops regex evaluation.
3) Regex (~ or ~*) — evaluated in FILE ORDER, first match wins.
4) Plain prefix — longest match wins. For /api/v2/users: exact /api/health does not match. ^~ prefix is not defined. Regex ^/api/v[0-9]+/ matches, so it wins over the plain /api/ prefix. If you want /api/ to always win, change it to location ^~ /api/. This ordering trips up nearly everyone debugging wrong-block matching.

3. AWS Security — WAF, Config, KMS & Cognito



WAF + ALB: Every HTTP request inspected by WAF → block attacks → forward clean requests to ALB

WAF Component	Description	Example
Web ACL	Main WAF object. Contains ordered rules.	'MyAppWAF' attached to CloudFront
Rule	Condition + action. Inspects request.	Block if URL contains 'UNION SELECT' (SQLi)
Managed Rules	Pre-built rule groups from AWS	AWSManagedRulesCommonRuleSet (OWASP Top 10)
Rate Limit Rule	Block IP exceeding N requests/5 min	Block IP after 2000 requests in 5 minutes

3.1 AWS WAF — Web Application Firewall

WAF protects web applications from Layer 7 attacks by inspecting HTTP/HTTPS request content — URL path, query strings, headers, body, cookies. Attaches to CloudFront, ALB, or API Gateway.

Geo Match Rule	Block/allow by country	Block all traffic from specific countries
IP Set Rule	Block/allow specific IP list	Block known malicious IPs, allow office IPs

WAF Action	Effect on Request	When to Use
Allow	Permit request through to application	Explicitly whitelist trusted IPs
Block	Return HTTP 403 Forbidden immediately	Block known attackers and malicious patterns
Count	Log but don't block (test mode)	Test new rules in production before enabling Block
CAPTCHA	Return CAPTCHA challenge page	Suspected bot — let humans through
Challenge	Silent JS browser fingerprint check	Automated scraper detection

VEERA

18.2 AWS Config — Compliance Monitoring

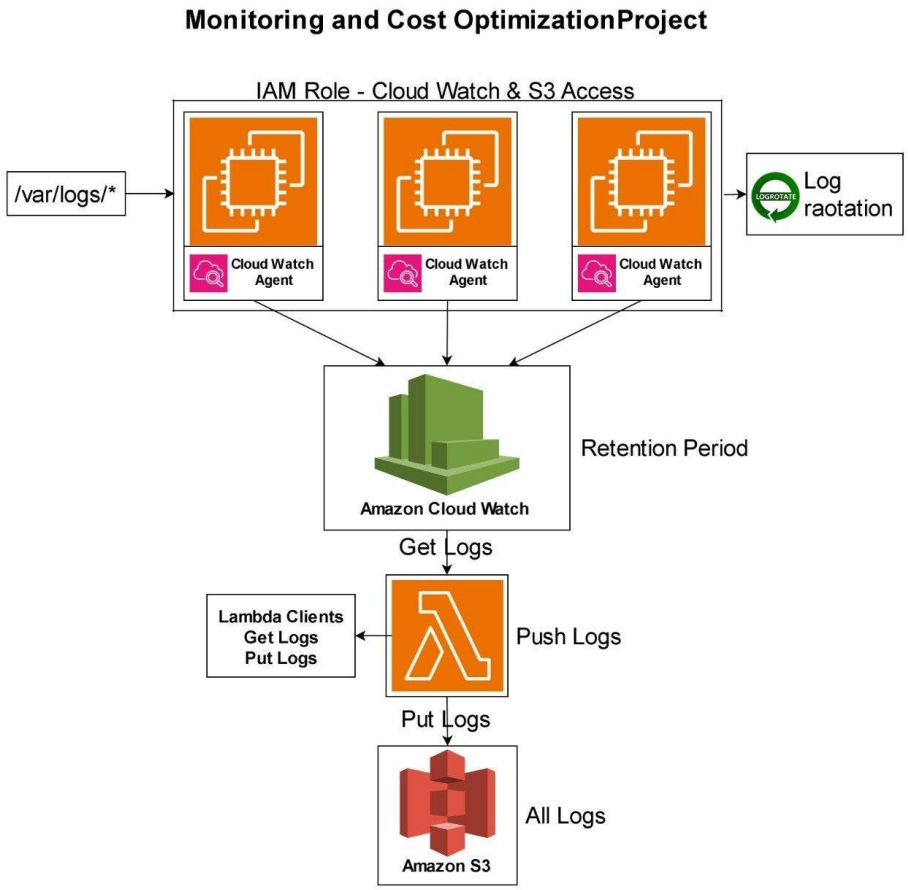
AWS Config continuously records every configuration change to your AWS resources and evaluates them against compliance rules. Answers the question: 'Is my infrastructure compliant right now, and what changed?'

- Config Recorder: Records every configuration change — EC2, S3, IAM, VPC, RDS, security groups, everything.
- Configuration History: See the full history of any resource. 'Show me this security group's config 3 days ago.'
- Config Rules: Compliance checks that run automatically on every change. Pass = Compliant. Fail = NON_COMPLIANT.

AWS Managed Config Rule	What It Checks	Severity
s3-bucket-public-readprohibited	S3 buckets NOT publicly readable	CRITICAL
ec2-instance-no-public-ip	EC2 instances have no public IP attached	HIGH
root-account-mfa-enabled	Root account has MFA enabled	CRITICAL

required-tags	Resources have all required tags	MEDIUM
encrypted-volumes	All EBS volumes encrypted	HIGH
rds-instance-public-accesscheck	RDS instances not publicly accessible	CRITICAL
iam-password-policy	IAM password policy meets requirements	MEDIUM

18.3 Automated Compliance — Config + EventBridge + Lambda + PagerDuty



Config → EventBridge → Lambda → PagerDuty: Automated compliance alerting pipeline

Architecture Flow

Step-by-Step Automated Compliance Alert: 1. Developer accidentally makes S3 bucket public. 2. AWS Config detects the change → evaluates rule s3-bucket-public-read-prohibited. 3. Result: NON_COMPLIANT → Config publishes compliance change event. 4. EventBridge rule matches: source=aws.config AND complianceType=NON_COMPLIANT. 5. EventBridge triggers Lambda function. 6. Lambda extracts resource info, builds PagerDuty payload, calls PagerDuty Events API v2. 7. PagerDuty creates Critical incident → on-call engineer paged within 60 seconds. 8. Engineer fixes bucket → Config re-evaluates → COMPLIANT → incident resolves.

```
# EventBridge rule pattern — matches Config NON_COMPLIANT events {   "source":
["aws.config"],   "detail-type": ["Config Rules Compliance Change"],   "detail":
{     "newEvaluationResult": {       "complianceType": ["NON_COMPLIANT"]     }
} }
```

```
# Lambda (Python) — send PagerDuty alert import json, requests  def
lambda_handler(event, context):     rule_name =
event['detail']['configRuleName']     resource_id =
event['detail']['resourceId']     account = event['account']          payload =
{         'routing_key': 'YOUR_PAGERDUTY_INTEGRATION_KEY',
'event_action': 'trigger',         'payload': {             'summary':
f'NON_COMPLIANT: {rule_name} | {resource_id} | Account: {account}',
'severity': 'critical',             'source': 'AWS Config',
'custom_details': {'rule': rule_name, 'resource': resource_id}         }     }
resp = requests.post('https://events.pagerduty.com/v2/enqueue', json=payload)
return {'statusCode': resp.status_code}
```

18.4 AWS KMS — Key Management Service

KMS creates and controls cryptographic keys used to encrypt your data at rest. Integrated with S3, EBS, RDS, DynamoDB, Lambda, Secrets Manager, and 100+ other AWS services.

- **Symmetric CMK:** Single 256-bit AES key. Encrypt and decrypt with the same key. Most common. Default for most services.
- **Asymmetric CMK:** RSA or ECC key pair. Public key encrypts, private key decrypts. Used for signing and public-key cryptography.
- **AWS Managed CMK:** AWS creates/manages on your behalf. Free. Service automatically uses when you enable encryption.
- **Customer Managed CMK:** You create and control. $1/month per key. Configure key rotation policy. Full access control. Full CloudTrail audit.
- **Envelope Encryption:** KMS pattern: generate a data key → encrypt your data with data key → encrypt data key with master key → store encrypted data + encrypted data key together. KMS master key never sees actual data.
- KMS is REGIONAL: keys cannot leave the region they were created in.
- All KMS API calls logged in CloudTrail — complete audit trail for compliance.

18.5 Amazon Cognito — User Authentication

Cognito provides user authentication, authorization, and user management for web and mobile apps. Add sign-up/sign-in to your app without building your own auth system.

Component	Purpose	What It Returns
User Pools	User directory: sign-up, sign-in, MFA, social IdP (Google/Facebook), SAML/AD federation	JWT tokens: ID token, Access token, Refresh token
Identity Pools	Exchange any token (Cognito, Google, SAML) for temporary AWS credentials	STS temporary credentials (AccessKey + SecretKey + SessionToken)

- OAuth 2.0, OpenID Connect (OIDC), and SAML 2.0 standards supported.
- User Pools integrate natively with ALB and API Gateway as built-in JWT authorizers.
- Scales to hundreds of millions of users automatically.

VEERA

Scenario-Based Interview Questions — AWS Security — WAF, Config, KMS & Cognito

Q1: Scenario: A compliance auditor asks: prove no S3 bucket has been public in 6 months and show who made every change.

Two services together:

1) AWS Config — records configuration state over time. Use managed rules s3-bucket-public-read-prohibited and check the compliance timeline for each bucket at every point. Config answers WHAT did it look like on 12 March?

2) CloudTrail — query for PutBucketAcl, PutBucketPolicy events with userIdentity field. Answers WHO changed WHAT and WHEN. Caveat: Config only has history from when it was enabled. Enable both on day one via Control Tower. Going forward: account-level Block Public Access + SCP + Config auto-remediation + Security Hub dashboard.

Q2: Scenario: Your company must encrypt all data at rest. Even AWS employees must not be able to read it. What do you implement?

Standard KMS is FIPS-validated and AWS states no access to key material, but for provable custody: use KMS External Key Store (XKS) or CloudHSM — key material lives in YOUR HSM outside AWS. AWS physically cannot decrypt without your HSM. For standard encryption: Customer Managed Keys (not AWS-managed) everywhere — S3 SSE-KMS, EBS default

encryption, RDS encryption, DynamoDB. Tight key policies with kms:ViaService conditions. SCPs denying unencrypted resource creation. Config rules with auto-remediation. TLS 1.2+ enforced via bucket policy aws:SecureTransport condition.

Q3: Scenario: A brute-force attack is hitting your login page at 5000 requests per minute from hundreds of IPs. How do you stop it?

AWS WAF with rate-based rules on the ALB/CloudFront. Create a rule: if requests to /login exceed 100 per 5 minutes from a single IP, block that IP automatically. For distributed attacks (hundreds of IPs): use WAF's managed Bot Control rule group which detects bot patterns beyond IP counting. Add a CAPTCHA action on suspicious requests. Combine with Cognito's built-in brute-force protection (account lockout after N failed attempts). WAF operates at Layer 7 — it can see the URL path; Security Groups cannot.

Q4: Scenario: Your developer stores database passwords in environment variables in the EC2 user data script, which is visible in the instance metadata and CloudTrail. What is the correct approach?

Use AWS Secrets Manager. Store the password in Secrets Manager. The application reads it at runtime using the SDK: secretsmanager:GetSecretValue. Benefits:

1) Automatic rotation — Secrets Manager can rotate the RDS password end-to-end with Lambda.
2) Encryption at rest with KMS.
3) Audit trail — every access logged in CloudTrail.
4) Cross-region replication for DR. For non-rotating config values (feature flags, endpoints), use Systems Manager Parameter Store (free for standard parameters). Never store secrets in code, user data, environment variables, or AMIs.

Q5: Scenario: You need to authenticate users for your web app with sign-up, login, MFA, and Google/Facebook login. Also need temporary AWS credentials for the mobile app to upload directly to S3. What AWS services do you use?

Two Cognito services:

1) Cognito User Pool for AUTHENTICATION — handles sign-up, login, MFA, password reset, social federation (Google/Facebook). Returns JWT tokens.
2) Cognito Identity Pool for AUTHORIZATION — exchanges the JWT token from the User Pool for temporary AWS credentials (via STS). The mobile app uses these credentials to upload directly to S3 with the AWS SDK. User Pool = who are you? Identity Pool = here are AWS keys for what you can touch.

Q6: Scenario: Your Security Group allows port 22 from 0.0.0.0/0, your S3 bucket is public, and your RDS is publicly accessible. A pen test finds all three. What is the fastest way to detect these issues automatically going forward?

Enable AWS Security Hub with the AWS Foundational Security Best Practices standard. It aggregates findings from Config rules, GuardDuty, Inspector, and Access Analyzer into one dashboard. Specific Config rules: restricted-ssh (SG check), s3-bucket-public-read-prohibited, rds-instance-public-access-check. Enable auto-remediation via Config+SSM/Lambda. Also enable GuardDuty for threat detection and Access Analyzer for external access analysis. The point: automate detection so you find these before the pen tester does.

Q7: Scenario: Explain the difference between CloudTrail and AWS Config with a real example.

CloudTrail = the CCTV camera. It records WHO did WHAT and WHEN: 'User arjun called AuthorizeSecurityGroupIngress at 3:14 PM from IP 203.0.113.5.' AWS Config = the inventory checklist. It records WHAT the resource looks like: 'Security Group sg-abc123 currently allows

port 22 from 0.0.0.0/0. Yesterday it only allowed from 10.0.0.0/16.' Config also evaluates COMPLIANCE: 'Is this resource following our rules?' Example scenario: Config tells you the SG is non-compliant NOW. CloudTrail tells you who made the change and when. Together they answer: what happened, who did it, and is it compliant.

Q8: Scenario: Your KMS key policy allows only the DBA team to decrypt the database credentials. A developer needs temporary access to debug a production issue. How do you grant it without changing the key policy?

Use a KMS Grant. The DBA team creates a grant on the KMS key allowing the developer's IAM role to use kms:Decrypt. The grant can include constraints (encryption context) and can be revoked immediately when debugging is done. Unlike key policy changes (which are permanent and require policy editing), grants are temporary and granular. They are the correct mechanism for temporary, delegated access to KMS keys without modifying the key policy.

Q9: Scenario: Your company processes credit card payments and needs PCI DSS compliance on AWS. What are the key requirements?

1) Network isolation: cardholder data in dedicated private subnets with NACLs and SGs restricting access.
2) Encryption: TLS 1.2 in transit, KMS CMK encryption at rest for all data stores.
3) Access control: least-privilege IAM, MFA on all admin accounts, no shared credentials.
4) Logging: CloudTrail enabled all regions, VPC Flow Logs, S3 access logs — retained and tamper-proof.
5) Vulnerability management: Amazon Inspector for EC2/containers, GuardDuty for threats.
6) AWS is PCI DSS Level 1 compliant for its infrastructure — but YOU are responsible for your application layer, IAM configuration, and data handling. Use AWS Artifact to download AWS's compliance reports.

Q10: Scenario: Your app uses a KMS CMK to encrypt S3 objects. An auditor asks how KMS encrypts a 5 GB file when KMS has a 4 KB size limit.

Envelope encryption. KMS does not encrypt the 5 GB file directly. Process:

1) KMS generates a Data Key and returns it in plaintext AND encrypted form.
2) Your app encrypts the 5 GB file locally with the plaintext data key (fast, no network calls for the bulk data).
3) You store the ENCRYPTED data key alongside the ciphertext and discard the plaintext key from memory.
4) To decrypt: send the encrypted data key to KMS, get the plaintext key back, decrypt locally. This gives KMS's key control with local-speed bulk encryption.

19. AWS CloudFormation — Infrastructure as Code

CloudFormation lets you define your ENTIRE AWS infrastructure in JSON or YAML text files. Deploy consistently, repeatedly, and automatically — treating infrastructure exactly like application code.

19.1 Why Infrastructure as Code (IaC)?

Theory Explained
WITHOUT IaC: Engineers click through AWS Console. Dev environment looks different from staging which looks different from prod. 'Works in dev, fails in prod' mysteries. Disaster recovery means manually re-clicking hundreds of settings. No history of what changed or who changed it. WITH CloudFormation: Define all resources in YAML. Commit to Git. CI/CD deploys identical infrastructure to every environment. DR = run the same template in a new region. Change review = pull request with diff. Rollback = git revert + redeploy.

- Repeatability: Dev, staging, prod all deployed from IDENTICAL templates. Zero configuration drift.
- Version Control: Infrastructure changes tracked in Git. PR reviews for infrastructure. Full history.
- Disaster Recovery: Entire VPC + EC2 + RDS + ALB recreated in another region in 15 minutes.
- Cost Control: `aws cloudformation delete-stack` tears down every resource instantly. No orphaned resources.
- Self-Documenting: The template IS the truth — always up to date, no outdated wikis.

VEERA

19.2 Core Concepts

- **Template:** JSON or YAML file. Sections: AWSTemplateFormatVersion, Description, Parameters (inputs), Mappings, Conditions, Resources (REQUIRED), Outputs.
- **Stack:** One deployed instance of a template. All resources in the template created/managed/deleted as one unit.
- **Change Set:** Preview of changes. 'Show me what will be ADDED, MODIFIED, or DELETED if I apply this update.' Always use Change Sets in production.
- **Stack Policy:** Prevents accidental updates/deletes of critical resources during stack update. Protection against mistakes.
- **StackSets:** Deploy one stack to multiple accounts and/or regions simultaneously. For multiaccount organizations.
- **Nested Stacks:** Parent template references child templates. VPCStack + EC2Stack + DBStack → MainAppStack. Reusable modules.
- **Drift Detection:** Compare current actual state of resources vs template definition. Shows what was manually changed outside CloudFormation.

19.3 Template Structure Example

```
AWSTemplateFormatVersion: "2010-09-09" Description: "Web application stack"
Parameters:   Environment:     Type: String     AllowedValues: [dev, staging,
prod]   InstanceType:     Type: String     Default: t3.micro  Resources:   #
Security Group   WebSecurityGroup:     Type: AWS::EC2::SecurityGroup
Properties:      GroupDescription: "Allow HTTP and HTTPS"      VpcId: !Ref MyVPC
SecurityGroupIngress:         - IpProtocol: tcp           FromPort:
80           ToPort: 80           CidrIp: 0.0.0.0/0    # EC2 Instance
WebServer:     Type: AWS::EC2::Instance     Properties:       ImageId: ami-
0abcdef1234567890       InstanceType: !Ref InstanceType       SecurityGroupIds:
- !Ref WebSecurityGroup       Tags:         - Key: Environment           Value:
!Ref Environment  Outputs:   WebServerPublicIP:     Description: "Public IP of
web server"     Value: !GetAtt WebServer.PublicIp     Export:       Name: !Sub
"${AWS::StackName}-PublicIP"
```

Key Tip

CloudFormation is FREE. You pay only for the AWS resources the template creates. If you create a template that builds 50 EC2 instances, you pay for 50 EC2 instances — not for CloudFormation.

VEERA

Scenario-Based Interview Questions — AWS CloudFormation — Infrastructure as Code

Q1: Scenario: A developer added a Name property to an RDS resource. The change set says Replace: True. Explain the danger.

Replace means CloudFormation will CREATE a new RDS instance and DELETE the old one — your production database with all its data, gone. DBInstanceIdentifier is immutable; CFN's only path is replacement. The developer thinks they are adding a label. DO NOT execute. Protect against this: DeletionPolicy: Retain, UpdateReplacePolicy: Retain on the RDS resource. Stack Policy with Deny on Update:Replace. RDS Deletion Protection ON. Put stateful resources (RDS, S

3) in a SEPARATE stack from stateless ones (EC2, ALB).

Q2: Scenario: Your CloudFormation stack update failed and is stuck in UPDATE_ROLLBACK_FAILED. You cannot do anything. Fix it.

Rollback itself failed — usually a resource was changed manually outside CFN. Steps:

1) Read Events tab for the exact resource and error.

2) Fix the cause manually (recreate a deleted resource, detach a stuck ENI).

3) Run ContinueUpdateRollback.

4) If a resource cannot be rolled back, use ContinueUpdateRollback with ResourcesToSkip — but the stack and reality are now out of sync.

5) Reconcile with drift detection and import. Prevention: never manually change stack-managed resources.

Q3: Scenario: You must deploy the same VPC+security baseline to 50 AWS accounts across 3 regions. How?

CloudFormation StackSets with AWS Organizations integration. Write ONE template, create a StackSet targeting the OU with 3 regions. Enable automatic deployment so new accounts get the baseline automatically. Parameterize CIDRs so each account gets non-overlapping ranges (use IPAM). Set MaxConcurrentPercentage and FailureTolerance so a bad template does not break all 50 accounts. Better: use AWS Control Tower which wraps StackSets, SCPs, and Organizations.

Q4: Scenario: Someone made manual changes to stack-managed resources via the console. Your next CloudFormation update may revert their changes or fail. How do you detect this?

Use Drift Detection. It compares the actual resource configuration against the template and shows MODIFIED, DELETED, or IN_SYNC for each resource. Run drift detection periodically or before any update. To fix drift: either import the manual change into the template (update the template to match reality) or revert the manual change. Prevention: remove console write access in production — the template is the only path to make changes.

Q5: Scenario: CloudFormation vs Terraform vs CDK. Your team is AWS-only, developer-heavy, and wants to use TypeScript. Which do you recommend?

AWS CDK with TypeScript. CDK lets you write infrastructure with real programming constructs — loops, conditionals, classes, type checking. It synthesizes to CloudFormation, so you get programming power with native AWS deployment, drift detection, and StackSets. For a developer-heavy, AWS-only team, CDK is the best fit. Terraform if you are multi-cloud. Raw CloudFormation for simple, stable stacks or teams that prefer declarative YAML.

Q6: Scenario: You need to pass the ALB DNS name from your network stack to your application stack. How?

Use CloudFormation Outputs with Export. In the network stack: Output the ALB DNS name with an Export name (e.g., ProdALBDNS). In the application stack: reference it with Fn::ImportValue: ProdALBDNS. This creates a cross-stack reference. Important: you cannot delete the exporting stack while other stacks import its values. Alternative: use SSM Parameter Store — the network stack writes the value, the app stack reads it.

Q7: Scenario: Your CloudFormation template has grown to 800 resources. Deployments are slow and hit the 500-resource limit. What do you do?

Use Nested Stacks. Break the template into logical modules: a VPC stack, an ALB stack, a database stack, an app stack. The parent template references each as an AWS::CloudFormation::Stack resource. Each nested stack has its own 500-resource limit. Benefits: reusability (the VPC stack can be used across projects), faster change detection (only changed stacks are updated), and separation of concerns.

Q8: Scenario: Your production RDS has DeletionPolicy: Delete (the default). Someone accidentally runs delete-stack. What happens?

The RDS instance is DELETED along with all its data — no snapshot taken. This is a production-destroying event. Fix immediately:

1) Set DeletionPolicy: Snapshot (takes a final snapshot before deletion) or DeletionPolicy: Retain (leaves the instance alive when removed from the stack).
2) Enable Stack Termination Protection so delete-stack is rejected outright.
3) Enable RDS Deletion Protection. All three layers should be on for every production stateful resource.

Q9: Scenario: You want to create an EC2 instance only in production, not in dev. How do you make the template environment-aware?

Use Parameters and Conditions. Add a parameter Environment with AllowedValues: [dev, prod]. Add a Condition: IsProd: !Equals [!Ref Environment, prod]. On the EC2 resource: add Condition: IsProd. The instance is created only when the template is deployed with Environment=prod. For dev, it is skipped entirely. Use this pattern for Multi-AZ (only in prod), larger instance types (only in prod), and expensive resources.

Q10: Scenario: Your stack update takes 45 minutes. You want to know exactly what will change before committing. How?

Create a Change Set. It shows exactly what CloudFormation WILL create, modify, or replace — without executing anything. Watch for Replace: True on stateful resources (RDS, S3, DynamoDB) — replacement means delete and recreate. Review the change set, then decide: execute or delete it. In CI/CD, make change set creation a mandatory PR gate. Never run update-stack directly on production without reviewing a change set first.

20. AWS Containers — ECS, EKS, ECR & Fargate

Containers package your application + all dependencies into a single portable image. 'It works on my machine' becomes 'It works everywhere.' AWS provides fully managed container orchestration services.

20.1 Amazon ECS — Elastic Container Service

ECS is AWS's native container orchestration service. It manages placement, scheduling, and scaling of Docker containers on a cluster.

ECS Concept	Definition	Analogy
Task Definition	JSON blueprint: container images, CPU/memory, ports, env vars, volumes, IAM role	Like a recipe: list of ingredients and instructions
Task	A running instance of a Task Definition. 1-10 containers together.	Like a cooking session: one execution of the recipe

Service	Maintains N running tasks. Replaces failed tasks. Integrates with ALB.	Like a restaurant: always keeps N dishes ready
Cluster	Logical group of tasks/services running on EC2 or Fargate.	Like a kitchen: the operational environment

20.2 AWS Fargate — Serverless Containers

Fargate is the serverless compute layer for ECS and EKS. Define task CPU and memory — Fargate provisions the underlying compute transparently. No EC2 instances to manage.

- No EC2 instances: no OS patching, no cluster node management, no capacity planning.
- Per-task isolation: each Fargate task runs in its own isolated micro-VM. No shared kernel with other tasks.
- Per-second billing: pay only for the CPU and memory your containers actually use.
- Works with both ECS and EKS workloads.

Key Tip

When to use Fargate vs EC2 launch type: Use Fargate for most workloads (simpler, no server management). Use EC2 launch type for: GPU workloads, Windows containers, specialized instance requirements, or when cost optimization via Reserved Instances is critical.

20.3 Amazon ECR — Elastic Container Registry

- Managed Docker container registry — store, version, scan, and deploy container images.
- Fully integrated with ECS, EKS, Lambda, CodeBuild. Pull images without external credentials — IAM handles auth.
- Vulnerability scanning: automatically scan images for known CVEs on push. View scan results in console.
- Lifecycle policies: auto-delete old/untagged images. Example: keep last 10 tagged images only.
- All images encrypted at rest (KMS). Transfers over HTTPS only.
- Cross-region replication: replicate images to other regions for multi-region deployment pipelines.

Scenario-Based Interview Questions — AWS Containers — ECS, EKS, ECR & Fargate

Q1: Scenario: Your ECS tasks keep stopping with OutOfMemoryError. Container killed due to memory usage. Fix it.

Container exceeded its hard memory limit and the OOM killer terminated it. Debug:

1) Check ECS Stopped Reason and Container Insights MemoryUtilization for real peak.
2) The classic JVM trap: JVM does not know about the container limit and sizes heap from HOST RAM. Fix with -XX:+UseContainerSupport -XX:MaxRAMPercentage=75. Node.js: --max-old-space-size.
3) Is it a memory leak (climbs steadily) or genuine need? Profile before raising limits.
4) Right-size: set memory to peak + 30% headroom. Alarm on MemoryUtilization at 80%.

Q2: Scenario: Your EKS pods cannot pull images from ECR. They are in a private subnet. Debug it.

Check:

1) Private subnet route table: 0.0.0.0/0 -> NAT Gateway? NAT in a PUBLIC subnet? Most common cause.
2) ECR pull needs THREE things: ecr.api endpoint, ecr.dkr endpoint, AND S3 (layers live in S3).
3) The cost-correct fix: VPC Interface Endpoints for ecr.api and ecr.dkr, plus FREE S3 Gateway Endpoint. Ensure endpoint SG allows 443 from node SG.
4) IAM: node role needs AmazonEC2ContainerRegistryReadOnly.
5) Use IRSA (IAM Roles for Service Accounts) instead of node role for pod-level permissions.

Q3: Scenario: Migrate a monolithic Java app from 10 EC2 instances to containers. Give a realistic plan.

Do NOT rewrite into microservices first.

1) Containerize as-is: Dockerfile with multi-stage build, slim base image, non-root user, JVM container flags.
2) Externalise state: sessions to Redis, files to S3, secrets to Secrets Manager, logs to stdout. A container that cannot be killed freely is not containerized.
3) Set up ECR with scanning and immutable tags.
4) Deploy to ECS Fargate behind ALB.
5) Migrate traffic gradually using weighted target groups: 10% -> 50% -> 100%. Keep EC2 fleet as rollback for a week.
6) Add autoscaling and observability. THEN consider splitting into microservices where there is a measured reason.

Q4: Scenario: ECS vs EKS. Your team is small, AWS-only, no Kubernetes experience. Which do you choose?

ECS. Simpler, no control-plane cost, deep AWS integration, small learning curve. Best when your team is small and AWS-only. Choose EKS only for: multi-cloud strategy, existing K8s skills, complex platform needs (service meshes, operators), or specific ecosystem tools. Honest guidance: if nobody on the team can explain what a Kubernetes operator is, choose ECS.

Q5: Scenario: Your developer sets only the Task Execution Role but the app cannot read from S3 at runtime. What is wrong?

Wrong role. Task EXECUTION Role is used by the ECS AGENT — it pulls images from ECR, fetches secrets, writes CloudWatch Logs. The Task ROLE is used by YOUR APPLICATION at runtime for S3, DynamoDB, etc. 'App cannot read S3' = fix the Task Role. 'Task will not start or cannot pull image' = fix the Execution Role. Two different problems, two different roles.

Q6: Scenario: You need to run a batch job that takes 4 hours. ECS Fargate has no timeout limit but Lambda does. Which do you choose?

ECS Fargate. Lambda has a hard 15-minute timeout. Fargate tasks can run for as long as needed with no timeout limit. Define a Task Definition with the batch job container, schedule it with EventBridge (one-time or recurring), and it runs to completion. For batch processing at scale, also consider AWS Batch which manages compute resources and job queues. Fargate tasks can be run standalone — they do not need a Service for one-off jobs.

Q7: Scenario: Your ECR repository has 2000 images accumulated over 2 years. Storage costs are climbing. How do you clean it up?

Enable ECR Lifecycle Policies. Example rules: keep only the last 10 tagged images, expire any untagged image older than 1 day, keep images matching the pattern 'prod-*' for 90 days. Lifecycle policies automatically delete matching images. Also enable: tag immutability (prevents overwriting tested tags), image scanning (find CVEs), and a VPC endpoint for private access. Without lifecycle policies, ECR grows unbounded forever.

Q8: Scenario: Your ECS service deployment replaces tasks but users see 503 errors for 30 seconds during each deployment. What is wrong?

New tasks are registered with the ALB before the app is actually ready. The ALB routes traffic to a container still starting up. Fixes:

1) Configure a proper health check path that returns 200 only when ALL dependencies are ready.

2) Increase the health check grace period in the ECS service.

3) Set the ALB target group deregistration delay high enough for old tasks to finish in-flight requests.

4) Use deployment configuration minimumHealthyPercent=100 so new tasks are added before old ones are removed.

5) For ECS rolling update: set deployment circuit breaker with auto-rollback.

Q9: Scenario: You want to give a specific Kubernetes pod (not the entire node) permission to access only its own S3 bucket. How?

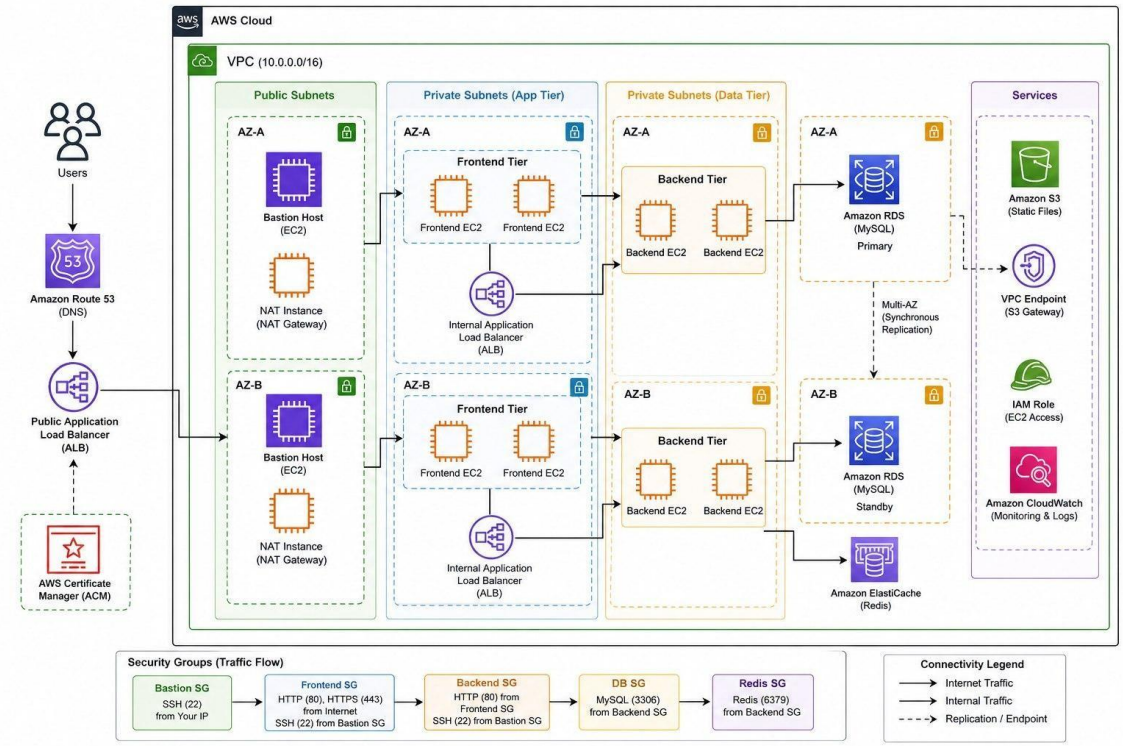
Use IRSA (IAM Roles for Service Accounts) or EKS Pod Identity. Create an IAM role with an S3 policy scoped to the specific bucket. Associate the role with a Kubernetes Service Account. Assign the service account to the pod. Now only that pod gets those credentials — other pods on the same node do not. Without IRSA, the node's instance role gives ALL pods the same permissions — a major security anti-pattern. IRSA provides pod-level least privilege.

Q10: Scenario: Your container needs 4 GPUs for ML inference. Can you use Fargate?

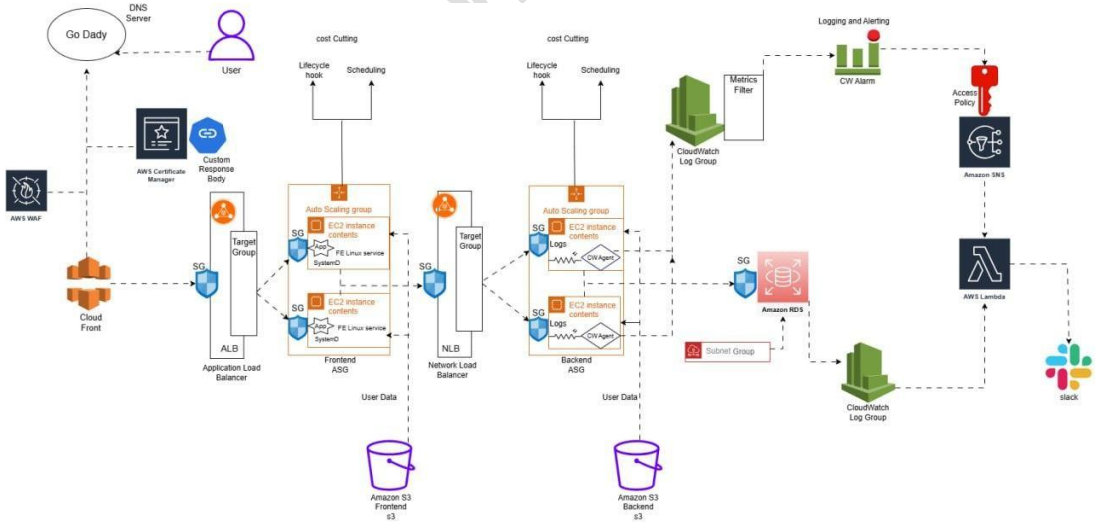
No. Fargate does not support GPU instances. You must use ECS on EC2 with GPU instance types (p3, g4dn, g5). Create a capacity provider with GPU instances, define the Task Definition with GPU resource requirements, and ECS schedules the task on a node with available GPUs. For managed GPU scaling, consider EKS with Karpenter which can provision GPU nodes on demand.

21. Full AWS Production Architecture

This is the complete picture — all services working together in a production-ready, highly available, scalable, and secure AWS architecture.



Complete Production Architecture: Route 53 → CloudFront → WAF → ALB → ASG/EC2 → RDS Multi-AZ



AWS Full Deployment Overview: All services integrated and connected

21.1 Complete Request Flow — Step by Step

- 1. User types www.myapp.com → Route 53 DNS resolves to CloudFront distribution endpoint.
- 2. Request routes to nearest CloudFront edge (400+ globally). Static assets (CSS/JS/images) → CACHE HIT → served in <10ms.
- 3. Dynamic API request: CloudFront forwards to ALB origin (cache miss for /api/* paths).
- 4. WAF inspects HTTP request: SQL injection? XSS? Known bad IP? Bot? → Block. Clean request? → Allow through.
- 5. ALB receives clean request → evaluates listener rules → routes to healthy EC2 target in Auto Scaling Group.
- 6. EC2 app processes → checks ElastiCache Redis (CACHE HIT → return in <1ms, zero DB query).
- 7. Cache MISS → EC2 queries RDS: SELECT queries to Read Replica, INSERT/UPDATE to Primary.
- 8. Response returns: EC2 → ALB → CloudFront → User. Full round trip < 200ms globally.

21.2 High Availability at Every Layer

Layer	Service	HA Mechanism	Recovery Time
DNS	Route 53	Health checks + automatic failover	<60 seconds
CDN	CloudFront	400+ global POPs, no single point of failure	Instant
Security	WAF + Shield	Edge-level, globally distributed	Always on
Load Balancer	ALB	Multi-AZ, auto-scales with traffic	Instant
Compute	EC2 + ASG	Min 2 instances across 2 AZs. Auto-replace unhealthy.	3-5 minutes
Cache	ElastiCache Redis	Primary + Multi-AZ replica, auto-failover	<60 seconds
Database	RDS Multi-AZ	Synchronous standby, auto DNS failover	~60 seconds
Object Storage	S3	11 nines durability, 99.99% availability	Always available

21.3 Security — Defense in Depth

- Edge Layer: CloudFront + AWS Shield Standard (DDoS protection). WAF blocks OWASP Top 10 at edge.
- Network: VPC with public/private subnets. RDS and EC2 app servers in private subnets. No public IPs on backend.
- Transport: ACM + HTTPS everywhere. ALB terminates SSL. All internal traffic over private IPs.
- Host: Security Groups as per-resource firewalls. Each tier allows traffic ONLY from tier above.
- Data: KMS encryption at rest (RDS, S3, EBS). SSL/TLS in transit for all DB connections.
- Identity: IAM roles (no hardcoded keys). MFA required for all admin users. Secrets Manager rotates passwords.
- Audit: CloudTrail logs all API calls. Config monitors compliance. CloudWatch detects anomalies.

21.4 Complete AWS Master Checklist

☑ AWS Overview: Cloud Computing (IaaS/PaaS/SaaS), Regions, AZs, Service Domains

☑ EC2: AMI, Instance Types, EBS vs Instance Store, Security Groups, Key Pairs, ENI, Nginx, Pricing

☑ IAM: Users, Groups, Roles, Policies, Allow/Deny, Role Switching, CLI, Programmatic Access, Best Practices

☑ VPC: CIDR, Subnets, IGW, NAT Gateway, Route Tables, SG vs NACL, Bastion, VPC Peering, Endpoints, Proxies

☑ Route 53: DNS, Hosted Zones, All Record Types, Alias vs CNAME, All 6 Routing Policies, Health Checks

☑ ELB: ALB (Layer 7 routing), NLB (Layer 4 performance), GWLB, Target Groups, Health Checks

☑ Auto Scaling: ASG, Launch Templates, All Scaling Policies, Instance Refresh, ASG+ALB Integration

☑ ACM + CloudFront: SSL/TLS Certificates, CDN, Distributions, Origins, TTL, Invalidations, OAC, Signed URLs

☑ S3: Buckets/Objects/Keys, All Storage Classes, Versioning, Lifecycle, Replication, Security, Static Site, Events

☑ RDS: Multi-AZ, Read Replicas, Aurora, PITR, Backup/Maintenance Windows, RDS Proxy, Secrets Manager, EC2 DB

☑ DynamoDB: Tables, Partition/Sort Keys, Indexes (GSI/LSI), Capacity Modes, DAX, Streams, Transactions

VEERA

☑ API Gateway: REST/HTTP/WebSocket APIs, Endpoint Types, Security Methods, Integration Types

☑ Lambda: Triggers, Execution Model, Cold/Warm Start, Layers, Monitoring, Provisioned Concurrency

☑ ElastiCache: Redis vs Memcached, Caching Pattern, Session Store, Rate Limiting, Leaderboards

☑ Monitoring: CloudWatch, Prometheus+Node Exporter+PromQL, Grafana, Alertmanager, PagerDuty, Dynatrace

☑ Messaging: SNS (Pub/Sub, Fan-out), SQS (Standard/FIFO, DLQ, Visibility Timeout), EventBridge

☑ Nginx: Reverse Proxy, Config, Path-Based Routing, Security Group Architecture

☑ Security: WAF, AWS Config Rules, Config+EventBridge+Lambda+PagerDuty Pipeline, KMS, Cognito

☑ CloudFormation: IaC, Templates, Stacks, Change Sets, StackSets, Nested Stacks, Drift Detection

☑ Containers: ECS, Fargate, ECR, EKS Overview

☑ Full Production Architecture: Request Flow, HA at Every Layer, Security Defense-in-Depth

VEERA

Scenario-Based Interview Questions — Full AWS Production Architecture

Q1: Scenario: Design the architecture for a food-delivery app expecting 10 lakh users with a 7-10 PM daily spike.

Edge: Route 53 -> CloudFront + WAF. API: ALB in public subnets across 3 AZs. Compute: ECS Fargate in private subnets with SCHEDULED scaling at 6:45 PM (the spike is predictable — reactive autoscaling is 4 minutes too late). Data: Aurora Multi-AZ for orders/payments, DynamoDB for tracking/sessions/cart (no connection limits), ElastiCache Redis for menu/geo cache. Async: order placement publishes to SNS -> SQS queues for restaurant notification, delivery assignment, invoicing — failure in one path never blocks an order. Key decisions: scheduled scaling for the known spike, SNS fan-out for path isolation.

Q2: Scenario: Your CTO says 'we need multi-region for high availability.' Argue for or against.

Push back.

1) Multi-AZ survives the realistic failure (data centre fire, power loss). Full region outages are rare.

2) Cost doubles.

3) Complexity explodes: replication lag, split-brain, cross-region consistency.
4) Most outages are caused by bad deploys — which replicate to both regions. Recommend instead: Multi-AZ everything, fix deployment safety (canary, auto-rollback), implement Pilot Light DR (data replicated, infra as code, tested quarterly). Multi-region active/active only when: regulatory requirement, contractual RTO in minutes, or global users where latency justifies presence.

Q3: Scenario: Black Friday is in 2 weeks. Last year the site crashed. What do you do?

Week 1:
1) Load test at 3x last year's peak — FIND the bottleneck.
2) Check every service quota NOW (EC2 vCPU limits, Lambda concurrency, NAT bandwidth). Quota increases take DAYS.
3) Review last year's post-mortem. Week 2:
4) Pre-scale — set high minimum capacity; ASG cannot boot faster than the spike arrives.
5) Cache aggressively: CloudFront for static, Redis for hot reads.
6) Queue non-critical work (emails, analytics) via SQS.
7) Add read replicas.
8) Freeze deploys 48 hours before.
9) Run a game day — kill an AZ deliberately.
1
0) War room with runbooks and dashboards on customer SLOs, not CPU.

Q4: Scenario: You inherit an AWS account: 200 instances, no documentation, Rs 30 lakh/month bill. What are your first 30 days?

Days 1-7 (see): Enable CloudTrail, Config, Cost Explorer, GuardDuty, Security Hub. Build inventory with Resource Explorer. Tag everything — owner, environment. Days 8-15 (security): Audit IAM (root MFA, delete unused keys), check public S3 buckets, open security groups, unencrypted databases. Test that backups actually restore. Days 16-30 (cost): Kill waste (unattached EBS, old snapshots, idle EIPs, dev instances 24/7, instances at <5% CPU). THEN buy Savings Plans for the steady baseline — never commit before right-sizing. Priority order: visibility -> security -> backups -> cost.

Q5: Scenario: Describe the standard 3-tier production architecture in one sentence each layer.

Route 53 -> CloudFront+WAF -> ALB in public subnets across 2+ AZs -> ASG/ECS app servers in PRIVATE subnets across 2+ AZs -> RDS Multi-AZ + ElastiCache in ISOLATED subnets. Static assets in S3 via CloudFront. Every tier spans 2+ AZs. Only the ALB is internet-facing. Security Groups chain: ALB-SG allows 443 from internet, App-SG allows 8080 from ALB-SG only, DB-SG allows 3306 from App-SG only. Database is mathematically unreachable from the internet.

Q6: Scenario: What is the difference between stateless and stateful, and why does it determine everything?

Stateless: no user data on the server — sessions in Redis, files in S3, secrets in Secrets Manager. Any server can handle any request. Stateful: sessions/files stored locally — user is stuck to one server. Why it matters: autoscaling, rolling deploys, Spot instances, containers, self-healing ALL require killing and replacing servers at any moment. If your server holds state, you cannot scale, deploy, or recover without user impact. Statelessness is the single precondition for cloud-native architecture.

Q7: Scenario: Explain Blue/Green vs Canary vs Rolling deployments with trade-offs.

Rolling: replace instances in batches. No extra cost, but both versions run simultaneously and rollback is slow. Blue/Green: stand up a complete new environment, switch all traffic at once. Instant rollback by switching back, but double infrastructure cost temporarily. Canary: send 5% of real traffic to the new version, watch metrics, then ramp up. Tiny blast radius, catches real-world bugs, but takes longer and needs good observability. Use Canary for risky user-facing changes, Blue/Green when instant rollback is critical.

Q8: Scenario: Your backend is fine (metrics green) but users say the site feels slow. Where do you look?

Green backend means the slowness is somewhere you are not measuring — usually the frontend or network.

1) Measure USER experience with CloudWatch RUM — Core Web Vitals (LCP, INP, CLS).
2) Check JS bundle size — a 4 MB React bundle on a mid-range Android on 4G takes seconds.
3) CloudFront cache hit ratio — low ratio means every user fetches from origin.
4) Check p99 latency, not averages — p50 200ms with p99 8s is a disaster that averages to 'fine'.
5) Check API call pattern — 60 sequential API calls to render one page (N+1 over network).
6) Third-party scripts (analytics, chat) routinely add seconds.

Q9: Scenario: Design the CI/CD pipeline for a 3-tier system.

Source: git with trunk-based dev, PR requires review, pipeline triggered on merge. Build: Frontend — npm ci, lint, tests, build with fingerprinted filenames, sync to S3, invalidate index.html. Backend — build Docker image, tag with GIT SHA (never 'latest'), scan for CVEs, push to ECR with immutable tags. Deploy: promote the SAME artifact through dev -> staging -> prod. Build once, deploy many — rebuilding per environment means prod runs untested code. Prod: manual approval gate, ECS blue/green with auto-rollback on 5xx alarm. Database: migrations run separately, backwards-compatible (expand-then-contract) so rollback works.

Q10: Scenario: A new engineer asks 'why is all this complexity needed? My Django app runs fine on one server.' Give an honest answer.

For many apps, one server IS correct. Complexity should be earned, not assumed. One EC2 + managed DB is fine for internal tools, MVPs, and low-traffic apps. But: start stateless from day one (sessions in Redis, files in S3, secrets in Secrets Manager) because statelessness is the ONE thing hard to retrofit. Then add each piece when you can name the problem it solves: ALB when you need zero-downtime deploys, ASG when traffic varies, Redis when DB is the bottleneck, CloudFront when users are far away. Complexity you can justify in one sentence is architecture; complexity you cannot is just cost.

24. Production System — Frontend + Backend + Database

This section brings everything together — showing the complete 3-tier production architecture exactly as taught in class, from user browser all the way to the database, with all failure scenarios and solutions.

24.1 The Data Consistency Problem

When multiple EC2 instances serve your app behind a load balancer, they must ALL connect to the SAME database. Without a shared database, each instance has its own local data — causing data inconsistency and redundancy.

Scenario	Without Shared DB	With Shared RDS
User signs up via EC2-1	User stored only in EC2-1 local DB	User stored in central RDS — all instances see it
User logs in via EC2-2	EC2-2: user doesn't exist → login fails	EC2-2 queries RDS → user found → login succeeds
Update profile on EC21	Only EC2-1 knows the update	All instances read updated data from RDS
Data inconsistency	10 EC2s = 10 different copies of data	Single source of truth — always consistent
Scale horizontally	Add EC2 = add another DB copy problem	Add EC2, point to same RDS — instant scale

Architecture Flow

THE FIX — Stateless EC2 + Shared RDS: All EC2 instances are STATELESS — they process requests but store NO user data locally. ALL data lives in RDS. ANY EC2 can handle ANY user's request. Frontend LB → Multiple EC2s (stateless) → Backend LB → Multiple EC2s (stateless) → Single RDS (shared)

24.2 Complete 3-Tier Production Architecture

Tier	Components	Subnet Type	Responsibility
Tier 1 — Presentation	External LB + EC2 (HTML/CSS/React/Mobile)	LB: Public, EC2: Private	Serve UI, handle HTTP/HTTPS, SSL termination
Tier 2 — Application	Internal LB + EC2 (Python/Java/Node.js)	Private	Business logic, API processing, DB queries
Tier 3 — Data	RDS MySQL Multi-AZ + Read Replicas	Isolated Private	Store and retrieve all application data

Theory Explained

Why 3 separate layers? 1. Independent Scaling: Traffic spike on frontend? Scale only frontend EC2. DB getting slow? Add Read Replicas only. Backend slow? Scale backend EC2. Each layer scales independently. 2. Security: Frontend never talks directly to DB. Only backend can reach DB. • Frontend SG: Allow 80/443 from internet only • Backend SG: Allow 8080/5000 from Frontend-SG only • DB SG: Allow 3306 from Backend-SG only 3. Fault Isolation: Frontend bug doesn't crash the DB. Backend issue doesn't corrupt data. Clear separation of concerns.

Security Group Rules for 3-Tier Architecture

Instance	Allow Port	Allow From	Block From
Frontend EC2	80, 443	0.0.0.0/0 (internet)	Everything else
Backend EC2	5000/8080	Frontend Security Group ID ONLY	Internet, other SGs
RDS Database	3306	Backend Security Group ID ONLY	Everything else including internet

VEERA

Scenario-Based Interview Questions — Production System — Frontend + Backend + Database

Q1: Scenario: Walk through a user request end-to-end in a 3-tier production system.

1) User types app.company.com; Route 53 resolves via Alias to CloudFront.
2) CloudFront serves React bundle from edge (S3 origin on miss); WAF inspects first.
3) React app calls api.company.com which resolves to ALB.
4) ALB terminates TLS (ACM certificate), forwards to healthy target.
5) App server (private subnet) checks ElastiCache first; on miss queries RDS (isolated subnet).
6) Slow work (email, invoicing) pushed to SQS.
7) Response returns; logs to CloudWatch, traces to X-Ray. Security: internet touches only CloudFront and ALB; nothing else is reachable.

Q2: Scenario: Why should the frontend be on S3+CloudFront instead of on the EC2 app servers?

A React build is just static files. Serving them from servers you patch, scale, and pay for 24/7 is waste. S3+CloudFront gives: infinite scale with no ASG, global edge delivery (20ms in Delhi vs 300ms from Mumbai), near-zero cost, no patching, and it cannot go down the way an EC2 fleet

can. Bonus: a backend outage does not mean a blank page — the app shell still loads and shows a graceful error.

Q3: Scenario: Where do sessions, files, and secrets belong in this architecture? Why?

Sessions -> ElastiCache Redis (not local memory — or scale-in logs users out). Files -> S3 via pre-signed URL (not local disk — or the file vanishes when the instance is replaced). Secrets -> Secrets Manager (not code, AMI, Docker image, or git — those leak). Config -> Parameter Store. This makes the app STATELESS — the precondition for autoscaling, rolling deploys, and Spot instances.

Q4: Scenario: How does zero-downtime deployment work for each tier?

Frontend: build with fingerprinted filenames (app.a3f9b2.js), sync to S3, invalidate only /index.html. Backend: new AMI/container tagged with git SHA. ASG Instance Refresh or ECS rolling update with MinHealthy=100%. Essentials: deep /health readiness endpoint, deregistration delay for in-flight requests, ALB slow start for cold instances, auto-rollback on CloudWatch alarm. Database: migrations must be backwards-compatible (expand-then-contract) so old and new code run simultaneously.

Q5: Scenario: The site is down. Users see a blank page. Walk through your triage in 10 minutes.

Minute 0-2: Check alarms — the FIRST alarm usually names the root cause. Is it everyone or some users? Check from different network/region. Minute 2-5: Browser devtools — is index.html loading (CloudFront fine, backend issue) or nothing loading (DNS/CDN issue)? This splits the search space in half. Check ALB target group — zero healthy targets is the classic total outage cause. Minute 5-10: What CHANGED? CloudTrail + deploy history. The majority of outages are caused by a deploy or config change in the last hour. If a deploy correlates: ROLL BACK FIRST, investigate after. Two highest-yield questions: what fired first? and what changed?

Q6: Scenario: Top 10 production mistakes in a 3-tier system.

1) App servers in public subnets with public IPs.
2) Database with Publicly Accessible=Yes.
3) Security Groups referencing 0.0.0.0/0 instead of another SG.
4) Sessions in local memory forcing sticky sessions.
5) Only one AZ.
6) Health checks returning 200 without checking dependencies.
7) No connection pooling or RDS Proxy.
8) Secrets in environment variables committed to git.
9) No caching — RDS carries load Redis should absorb.
1
0) Alarming on CPU instead of customer experience.

Q7: Scenario: How do the three tiers Security Groups chain together and why is this critical?

ALB-SG: inbound 443 from 0.0.0.0/0. App-SG: inbound 8080 from ALB-SG only (SG ID, not CIDR). DB-SG: inbound 3306 from App-SG only. Why SG references not IPs:

1) Works automatically with autoscaling — no IP updates.
2) Database is mathematically unreachable from internet — no path from 0.0.0.0/0 to DB-SG exists.
3) Self-documenting: 'only app servers reach the DB.'

Q8: Scenario: Your backend is fine but users say the site feels slow. Where do you look?

Green backend = slowness is somewhere you are not measuring. Check:
1) CloudWatch RUM for actual user experience (LCP, INP, CLS).
2) JS bundle size — 4 MB on 4G Android is seconds.
3) CloudFront cache hit ratio.
4) p99 latency, not averages.
5) API call pattern — 60 sequential calls to render one page.
6) Third-party scripts adding seconds.
7) Geography — user in Chennai hitting origin in Virginia. Your green dashboard measures servers; the user measures the browser.

Q9: Scenario: Design the CI/CD pipeline for this system.

Source -> Build -> Test -> Deploy. Frontend: npm ci, lint, test, build fingerprinted, sync S3, invalidate /index.html. Backend: Docker image tagged GIT SHA (never latest), CVE scan, push ECR immutable. Deploy: promote SAME artifact dev -> staging -> prod (build once, deploy many). Prod: approval gate, ECS blue/green, auto-rollback on 5xx alarm. DB: separate migrations step, backwards-compatible. Secrets from Secrets Manager. OIDC instead of long-lived keys. Deploy freeze window before major events.

Q10: Scenario: A new engineer asks why not just use one server. Give an honest answer.

For many apps, one server IS correct — internal tools, MVPs, low-traffic apps. Adding ALBs, ASG, Redis, Multi-AZ to that is unnecessary complexity. But: start stateless from day one (sessions in Redis, files in S3, secrets in Secrets Manager) because statelessness is the ONE thing hard to retrofit. Then add each piece when you can name the problem it solves. The failure mode to warn about: the first time the CEO calls at 2 AM because a disk died and there is no backup. Start simple, stay stateless, add complexity only when you can justify it in one sentence.